

Data Warehouse

Judul Proses Bisnis 1 : Pengambilan kelas mata kuliah ke FRS

Penjelasan proses bisnis & granularitas (5W 1 H)

- What : Mahasiswa menambahkan mata kuliah Departemen ke FRS
- Who :
 - Mahasiswa → Mengajukan penambahan mata kuliah.
 - Sistem Akademik → Memverifikasi prasyarat mata kuliah dan ketersediaan kuota kelas
- When :

pada masa pengisian FRS (Menyesuaikan jadwal yang diberikan SIAKAD ITS)
- Why :

Agar mahasiswa dapat memenuhi jumlah SKS yang diperlukan untuk kelulusan dan mengikuti perkuliahan sesuai kurikulum.
- Where :

Secara online melalui sistem SIAKAD ITS

How (bagaimana) :

Mahasiswa mengakses Formulir Rencana Studi (FRS) di Sistem Informasi Akademik (SIAKAD) ITS melalui perangkat. mahasiswa melihat informasi yang diberikan sistem. Mahasiswa memilih mata kuliah dari dropdown, mengisi captcha sebagai validasi akhir. Data FRS tersimpan sebagai satu entri per matkul yang diambil.

Granularitas :

Satu baris pada tabel fakta = merepresentasikan satu pengambilan mata kuliah oleh seorang mahasiswa pada satu semester

Dimensi (tips gunakan 4w1h)

- What
Mahasiswa menambahkan kelas mata kuliah ke FRS
- Who

Mahasiswa
Sistem

- When
Pada masa pengisian FRS (menyesuaikan jadwal SIAKAD ITS)
- Where
Online melalui sistem SIAKAD ITS
- How
Melalui perangkat & browser
- Dimensi :

Dim_Mahasiswa:

NRP (pk) | INT | SCD Type 0
Nama | VARCHAR | SCD Type 1
IPK_terakhir | Decimal | SCD Type 2
Program_Studi | VARCHAR | SCD Type 2
Departemen | VARCHAR | SCD Type 2
Jenis kelamin | VARCHAR | SCD Type 0
Fakultas | VARCHAR | SCD Type 1
Angkatan | INT | SCD Type 1
Semester_saat_ini | INT | SCD Type 1
Ips_terakhir | Decimal | SCD Type 2
Jenjang | VARCHAR | SCD Type 1
Tanggal masuk | DATE | SCD Type 1
NIP_Dosen | VARCHAR | SCD Type 2

Dim_Kelas_MK:

ID_kelas_MK (pk) | INT | SCD Type 0
Kode_mk | VARCHAR | SCD Type 0
Kelas | VARCHAR | SCD Type 1
Dosen pengampu 1 | VARCHAR | SCD Type 1
Dosen pengampu 2 | VARCHAR | SCD Type 1
Dosen pengampu 3 | VARCHAR | SCD Type 1
Nama_MK | VARCHAR | SCD Type 2
SKS | INT | SCD Type 2
Semester_mk | VARCHAR | SCD Type 2
Ruangan | VARCHAR | SCD Type 1
Hari | VARCHAR | SCD Type 1
Jam_mulai | TIME | SCD Type 1
Jam_selesai | TIME | SCD Type 1
Kapasitas | INT | SCD Type 2

Dim_Waktu:

ID_Waktu | INT | SCD Type 0

Tanggal | DATE | SCD Type 0

Hari | VARCHAR | SCD Type 0

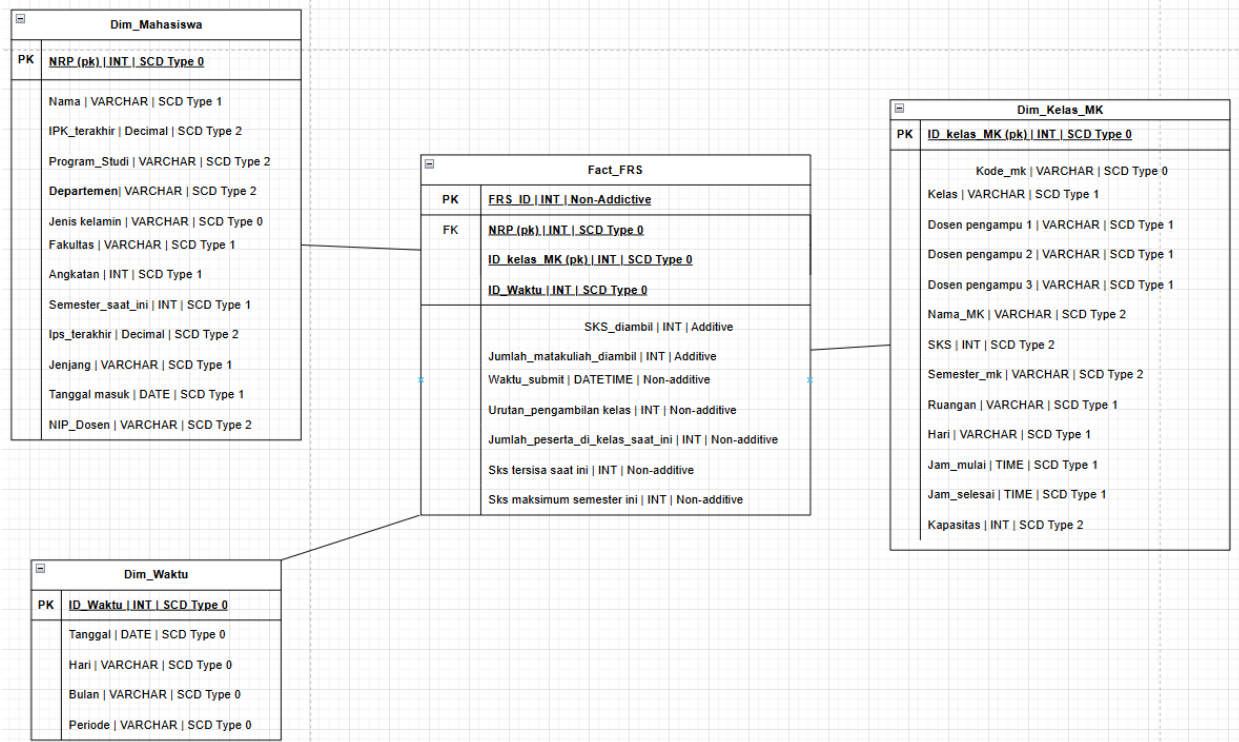
Bulan | VARCHAR | SCD Type 0

Periode | VARCHAR | SCD Type 0

Fakta (bukan tabel fakta)

- Detect facts
 - Termasuk jenisnya, additive, semi additive , non additive
 - SKS_diambil | INT | Additive
 - Jumlah_matakuliah_diambil | INT | Additive
 - Waktu_submit | DATETIME | Non-additive
 - Urutan_pengambilan_kelas | INT | Non-additive
 - Jumlah_peserta_di_kelas_saat_ini | INT | Non-additive
 - Sks tersisa saat ini | INT | Non-additive
 - Sks maksimum semester ini | INT | Non-additive

Star Schema (sampai physical diagram) ada keterangan scd dari dimensi, tipe fakta nya (additive / non additive / semi addivite) , misalnya ada degenerate dimension



Proses Bisnis 2 : Pembayaran Uang Kuliah Tunggal (UKT)

Penjelasan proses bisnis & granularitas (5W 1 H)

- What:
Mahasiswa melakukan pembayaran UKT sesuai dengan tagihan yang diberikan oleh Sistem Akademik (SIKAD) ITS.
- Who:
Mahasiswa: Melakukan pembayaran
Sistem Akademik: Memberikan informasi tagihan dan mencatat status pembayaran
- When:
Pada periode pembayaran UKT yang telah ditentukan oleh ITS (setiap awal semester)
- Where:
Dilakukan secara offline melalui mesin ATM berbagai bank mitra (misalnya Bank BNI, BRI, Mandiri, dll)
- Why:
Untuk memenuhi kewajiban administrasi keuangan dan mempertahankan status mahasiswa aktif.

- How:
Melalui ATM menggunakan virtual account yang disediakan oleh sistem, lalu data dibaca sistem akademik melalui integrasi sistem perbankan.
- Granularitas
Satu baris pada tabel fakta = satu transaksi pembayaran UKT oleh seorang mahasiswa pada satu semester.

Dimensi

Dim_Mahasiswa:

NRP (pk) | INT | SCD Type 0
Nama | VARCHAR | SCD Type 1
IPK_terakhir | Decimal | SCD Type 2
Program_Studi | VARCHAR | SCD Type 2
Departemen | VARCHAR | SCD Type 2
Jenis kelamin | VARCHAR | SCD Type 0
Fakultas | VARCHAR | SCD Type 1
Angkatan | INT | SCD Type 1
Semester_saat_ini | INT | SCD Type 1
Ips_terakhir | Decimal | SCD Type 2
Jenjang | VARCHAR | SCD Type 1
Tanggal masuk | DATE | SCD Type 1
NIP_Dosen | VARCHAR | SCD Type 2

Dim_Waktu:

ID_Waktu | INT | SCD Type 0
Tanggal | DATE | SCD Type 0
Hari | VARCHAR | SCD Type 0
Bulan | VARCHAR | SCD Type 0
Periode | VARCHAR | SCD Type 0

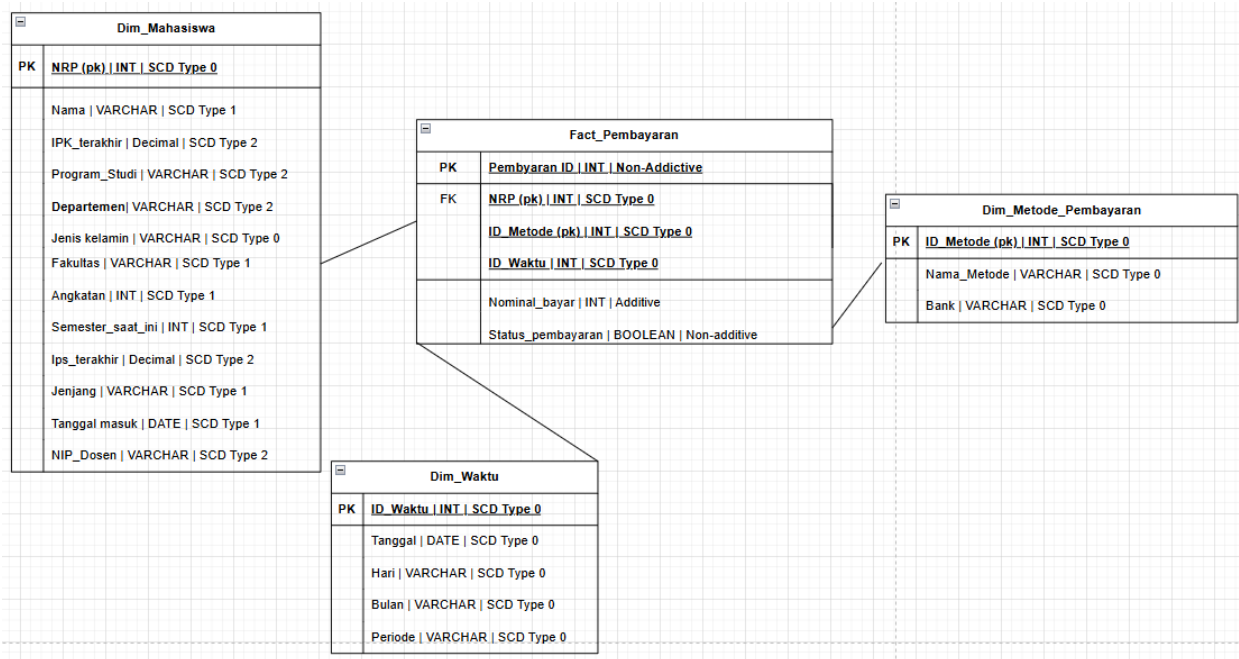
Dim_Metode_Pembayaran

ID_Metode (pk) | INT | SCD Type 0
Nama_Metode | VARCHAR | SCD Type 0 (misal: ATM, M-Banking, Teller)
Bank | VARCHAR | SCD Type 0

Fakta

Nominal_bayar | INT | Additive
 Status_pembayaran | BOOLEAN | Non-additive

Star Schema



Proses Bisnis 3 : Pengajuan Permohonan Drop Mata Kuliah dalam Formulir Rencana Studi (FRS)

Penjelasan proses bisnis & granularitas (5W 1 H)

- **What:**
Mahasiswa mengajukan permohonan drop mata kuliah dalam formulir rencana studi (FRS).
- **Who:**
 - Mahasiswa → Mengajukan permohonan drop mata kuliah.
 - Sistem Akademik → Memvalidasi perubahan (mencatat permohonan drop dan statusnya).
 - Dosen Wali → Menyetujui atau menolak permohonan drop mata kuliah berdasarkan pertimbangan akademik.
- **When:**
Pada masa perubahan dan drop FRS yang disesuaikan dengan jadwal yang diberikan oleh SIAKAD ITS (biasanya di awal atau tengah semester).
- **Why:**
Agar mahasiswa dapat menyesuaikan beban studi atau memperbaiki kesalahan pemilihan mata kuliah.

- Where:
Secara online melalui sistem SIAKAD ITS.
- How:
Mahasiswa mengakses FRS melalui perangkat dan sistem SIAKAD ITS, kemudian memilih mata kuliah yang ingin di-drop. Permohonan akan diverifikasi oleh dosen wali dan disetujui atau ditolak melalui sistem.
- Granularitas
Satu baris pada tabel fakta = satu permohonan drop mata kuliah yang diajukan oleh seorang mahasiswa pada satu semester.

Dimensi

Dim_Mahasiswa:

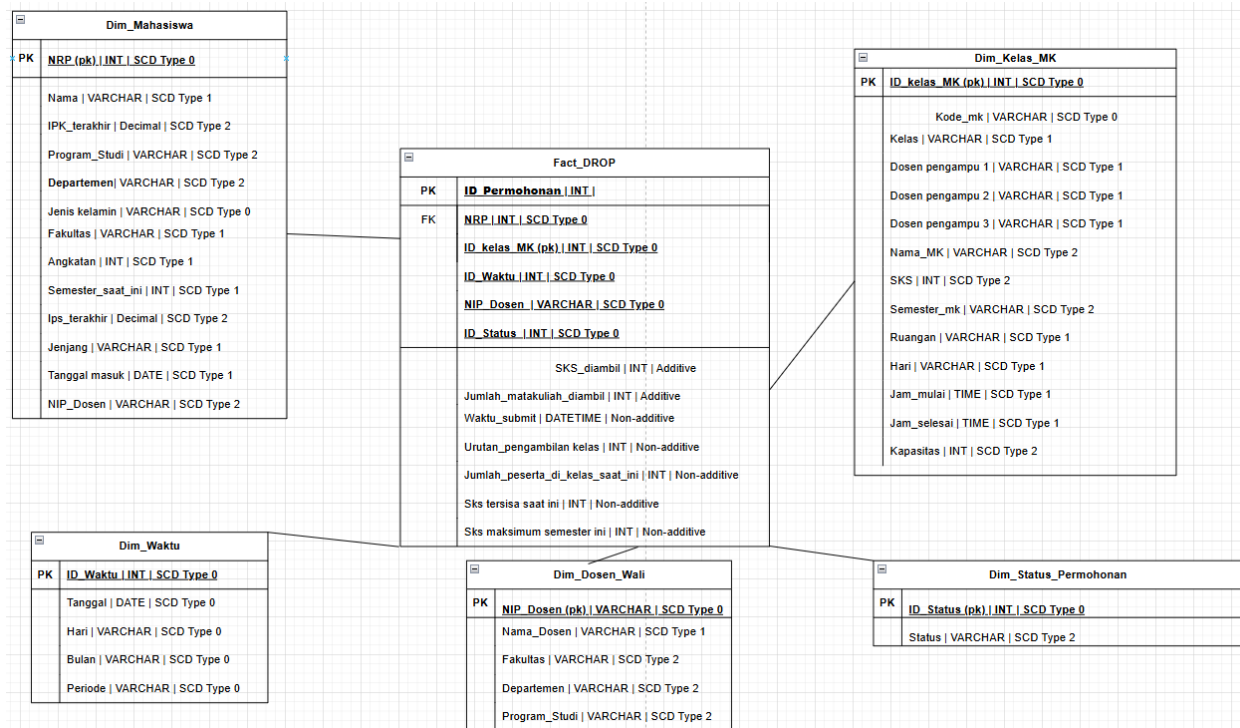
NRP (pk) | INT | SCD Type 0
Nama | VARCHAR | SCD Type 1
IPK_terakhir | Decimal | SCD Type 2
Program_Studi | VARCHAR | SCD Type 2
Departemen | VARCHAR | SCD Type 2
Jenis kelamin | VARCHAR | SCD Type 0
Fakultas | VARCHAR | SCD Type 1
Angkatan | INT | SCD Type 1
Semester_saat_ini | INT | SCD Type 1
Ips_terakhir | Decimal | SCD Type 2
Jenjang | VARCHAR | SCD Type 1
Tanggal masuk | DATE | SCD Type 1
NIP_Dosen | VARCHAR | SCD Type 2

Dim_Kelas_MK:

ID_kelas_MK (pk) | INT | SCD Type 0
Kode_mk | VARCHAR | SCD Type 0
Kelas | VARCHAR | SCD Type 1
Dosen pengampu 1 | VARCHAR | SCD Type 1
Dosen pengampu 2 | VARCHAR | SCD Type 1
Dosen pengampu 3 | VARCHAR | SCD Type 1
Nama_MK | VARCHAR | SCD Type 2
SKS | INT | SCD Type 2
Semester_mk | VARCHAR | SCD Type 2
Ruangan | VARCHAR | SCD Type 1
Hari | VARCHAR | SCD Type 1
Jam_mulai | TIME | SCD Type 1
Jam_selesai | TIME | SCD Type 1
Kapasitas | INT | SCD Type 2

Dim_Waktu:**ID_Waktu** | INT | SCD Type 0**Tanggal** | DATE | SCD Type 0**Hari** | VARCHAR | SCD Type 0**Bulan** | VARCHAR | SCD Type 0**Periode** | VARCHAR | SCD Type 0**Dim_Status_Permohonan:****ID_Status** (pk) | INT | SCD Type 0**Status** | VARCHAR | SCD Type 2**Dim_Dosen_Wali:****NIP_Dosen** (pk) | VARCHAR | SCD Type 0**Nama_Dosen** | VARCHAR | SCD Type 1**Fakultas** | VARCHAR | SCD Type 2**Departemen** | VARCHAR | SCD Type 2**Program_Studi** | VARCHAR | SCD Type 2**Fakta****Jumlah_SKS_terdrop** | INT | Additive**Jumlah_SKS_Before_drop** | INT | Additive**Jumlah_SKS_After_drop** | INT | Additive**Jumlah_Mata_Kuliah_Terdaftar_Before_Drop)** | INT | Non-additive**Jumlah_Mata_Kuliah_Terdaftar_After_Drop)** | INT | Non-additive

Star Schema



Proses Bisnis 4 : Setujui Formulir Rencana Studi (FRS)

Penjelasan proses bisnis & granularitas (5W 1 H)

- **What:**
Dosen wali menyetujui Formulir Rencana Studi (FRS) yang telah diajukan oleh mahasiswa bimbingannya untuk satu semester.
- **Who:**
Dosen Wali → Menyetujui atau memberikan catatan pada FRS.
Sistem Akademik → Memvalidasi dan mencatat status persetujuan.
- **When:**
Pada masa pengisian FRS, menyesuaikan jadwal akademik yang ditetapkan oleh SIAKAD ITS
- **Why:**
Untuk memastikan bahwa mahasiswa memilih mata kuliah sesuai dengan kurikulum, kapasitas akademik, dan kelulusan tepat waktu.
- **Where:**
Melalui sistem informasi akademik ITS (SIAKAD), secara online.
- **How:**

Mahasiswa mengisi FRS secara online. Setelah itu, dosen wali akan menerima notifikasi, lalu memeriksa dan menyetujui atau memberikan catatan melalui sistem.

- Granularitas
Satu baris dalam tabel fakta = satu persetujuan FRS oleh dosen wali untuk satu mahasiswa pada satu semester.

Dimensi

Dim_Mahasiswa:

NRP (pk) | INT | SCD Type 0
Nama | VARCHAR | SCD Type 1
IPK_terakhir | Decimal | SCD Type 2
Program_Studi | VARCHAR | SCD Type 2
Departemen | VARCHAR | SCD Type 2
Jenis kelamin | VARCHAR | SCD Type 0
Fakultas | VARCHAR | SCD Type 1
Angkatan | INT | SCD Type 1
Semester_saat_ini | INT | SCD Type 1
Ips_terakhir | Decimal | SCD Type 2
Jenjang | VARCHAR | SCD Type 1
Tanggal masuk | DATE | SCD Type 1
NIP_Dosen | VARCHAR | SCD Type 2

Dim_Waktu:

ID_Waktu | INT | SCD Type 0
Tanggal | DATE | SCD Type 0
Hari | VARCHAR | SCD Type 0
Bulan | VARCHAR | SCD Type 0
Periode | VARCHAR | SCD Type 0

Dim_Status_Permohonan:

ID_Status (pk) | INT | SCD Type 0
Status | VARCHAR | SCD Type 2

Dim_Dosen_Wali:

NIP_Dosen (pk) | VARCHAR | SCD Type 0
Nama_Dosen | VARCHAR | SCD Type 1
Fakultas | VARCHAR | SCD Type 2
Departemen | VARCHAR | SCD Type 2
Program_Studi | VARCHAR | SCD Type 2

Fakta

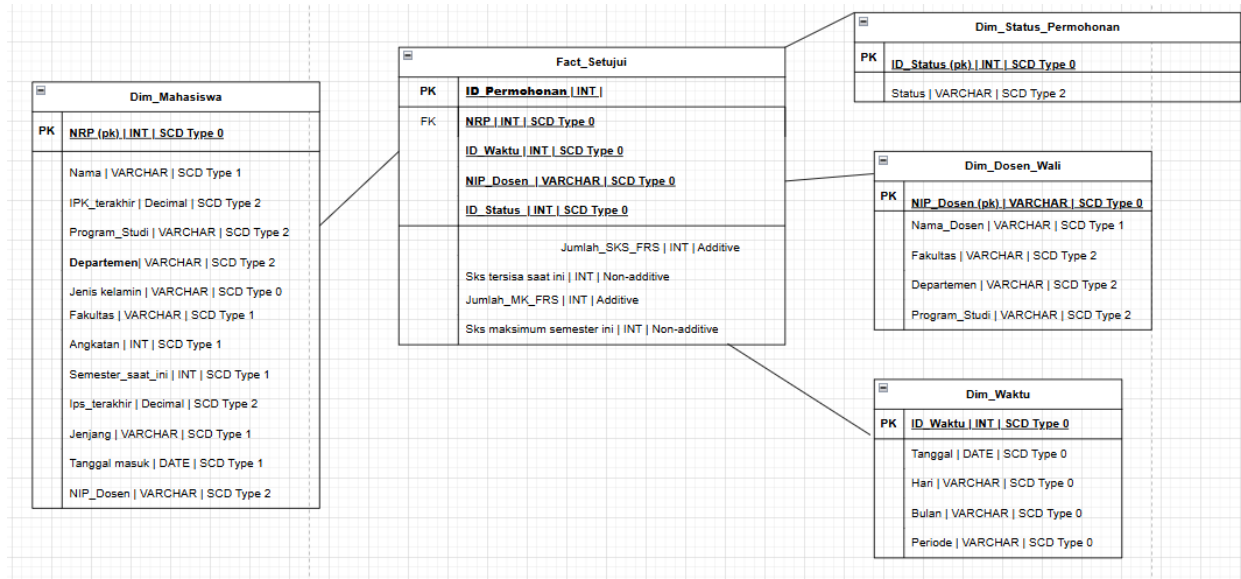
Jumlah_SKS_FRS | INT | Additive

Jumlah_MK_FRS | INT | Additive

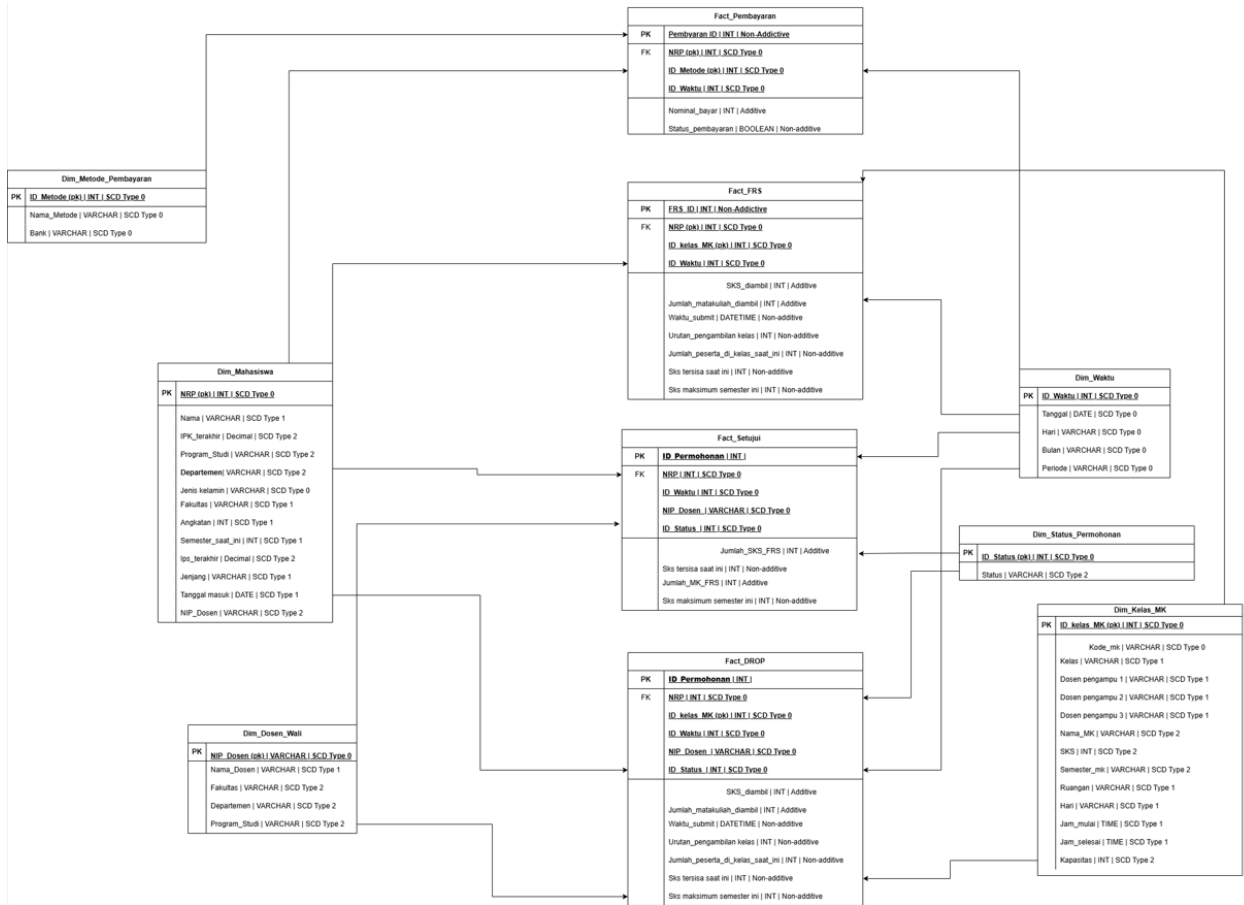
Sks tersisa saat ini | INT | Non-additive

Sks maksimum semester ini | INT | Non-additive

Star Schema



Star Schema Final



DW Bus Architecture

Proses bisnis	D.mahasiswa	D.Met.Pem	D.Dos.Wal	D.Kel.MK	D.Stat.Per	D.Waktu
Ambil kelas MK	✓	X	X	✓	X	✓
Bayar UKT	✓	✓	X	X	X	✓
Drop MK	✓	X	✓	✓	✓	✓
Setujui MK	✓	X	✓	X	✓	✓

Detailed DW Bus

Fact Table / OLAP Cube	Granularity	Facts	Mahasiswa	Kelas_MK	Waktu	Metode_Pembayaran	Status_Permohonan	Dosen_Wali
Pengambilan Kelas Mata Kuliah ke FRS	1 row per pengambilan matkul oleh mahasiswa per semester	SKS_diambil, Jumlah_matakuliah_diambil, Waktu_submit, Urutan_pengambilan_kelas, Jumlah_peserta_di_kelas_saat_ini, Sks_tersisa_saat_ini, Sks_maksimum_semester_ini	X	X	X			
Pembayaran Uang Kuliah Tunggal (UKT)	1 row per transaksi pembayaran UKT oleh mahasiswa per semester	Nominal_bayar, Status_pembayaran	X		X	X		
Pengajuan Permohonan Drop Mata Kuliah	1 row per permohonan drop matkul oleh mahasiswa per semester	Jumlah_SKS_terdrop, Jumlah_SKS_Before_drop, Jumlah_SKS_After_drop, Jumlah_Mata_Kuliah_Terdaftar_Before_Drop, Jumlah_Mata_Kuliah_Terdaftar_After_Drop	X	X	X		X	X
Persetujuan FRS oleh Dosen Wali	1 row per persetujuan FRS oleh dosen wali untuk mahasiswa per semester	Jumlah_SKS_FRS, Jumlah_MK_FRS, Sks_tersisa_saat_ini, Sks_maksimum_semester_ini	X		X		X	X

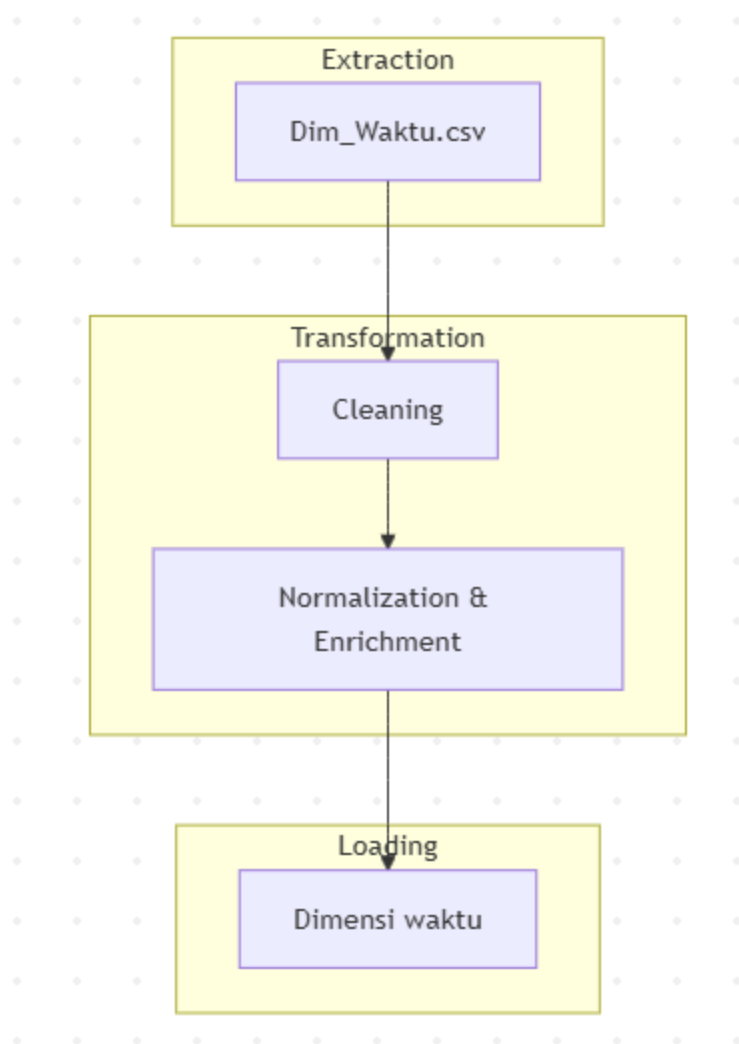
Stakeholder . business process DW Bus

Proses Bisnis / Fungsi Bisnis	Akademik	Keuangan	Dosen Wali	Mahasiswa	IT/SIAKAD	Bank Mitra
Pengambilan Kelas Mata Kuliah ke FRS	X		X	X	X	
Pembayaran Uang Kuliah Tunggal (UKT)	X	X		X	X	X
Pengajuan Permohonan Drop Mata Kuliah	X		X	X	X	
Persetujuan FRS oleh Dosen Wali	X		X	X	X	

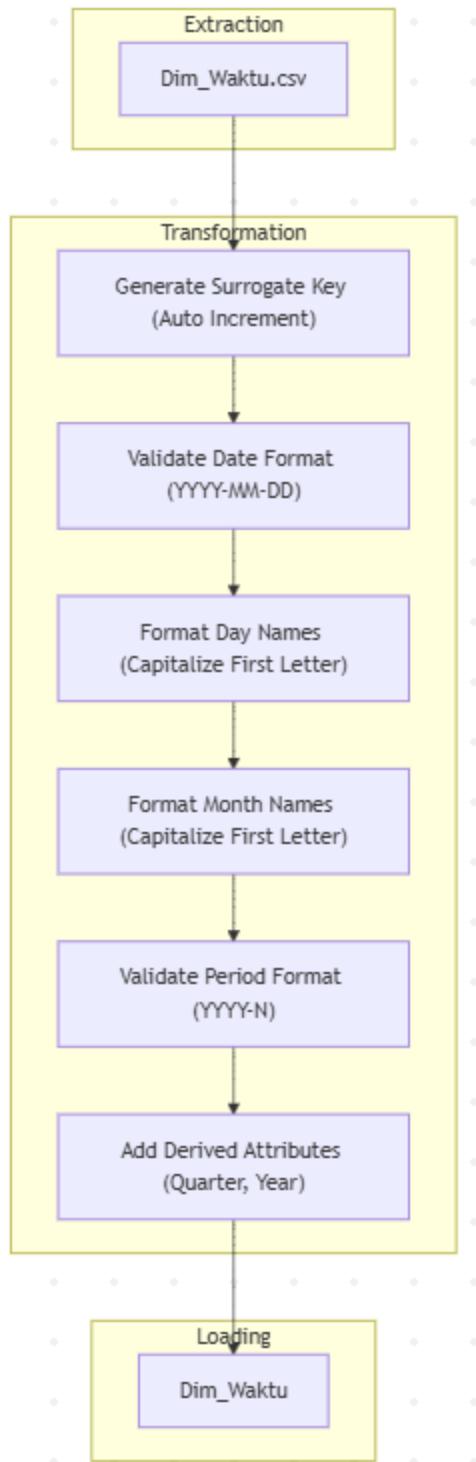
ETL

Table dimensi Waktu / Dim_waktu

High level plan



Detailed Level Plan



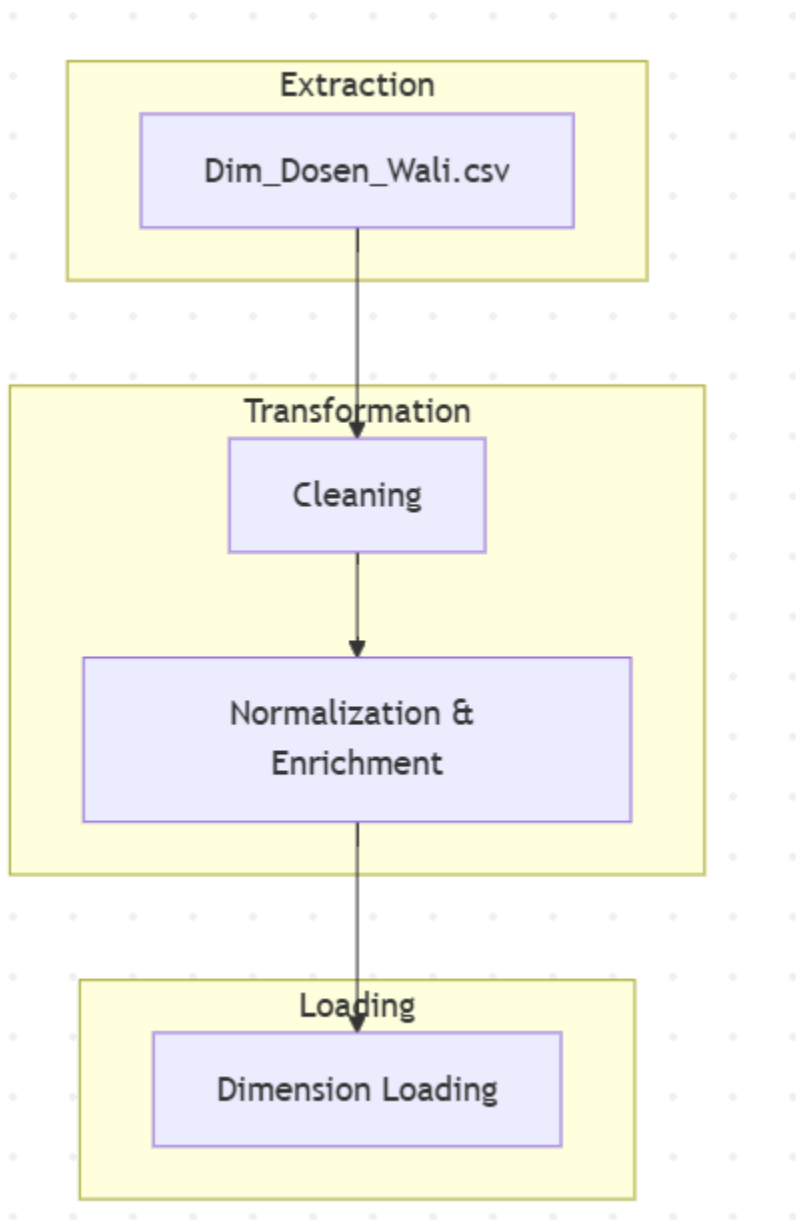
Source to target attribute mapping

Source Column	Target Column	Transformation Description
ID_Waktu	ID_Waktu_SK	Generate surrogate key (auto increment)

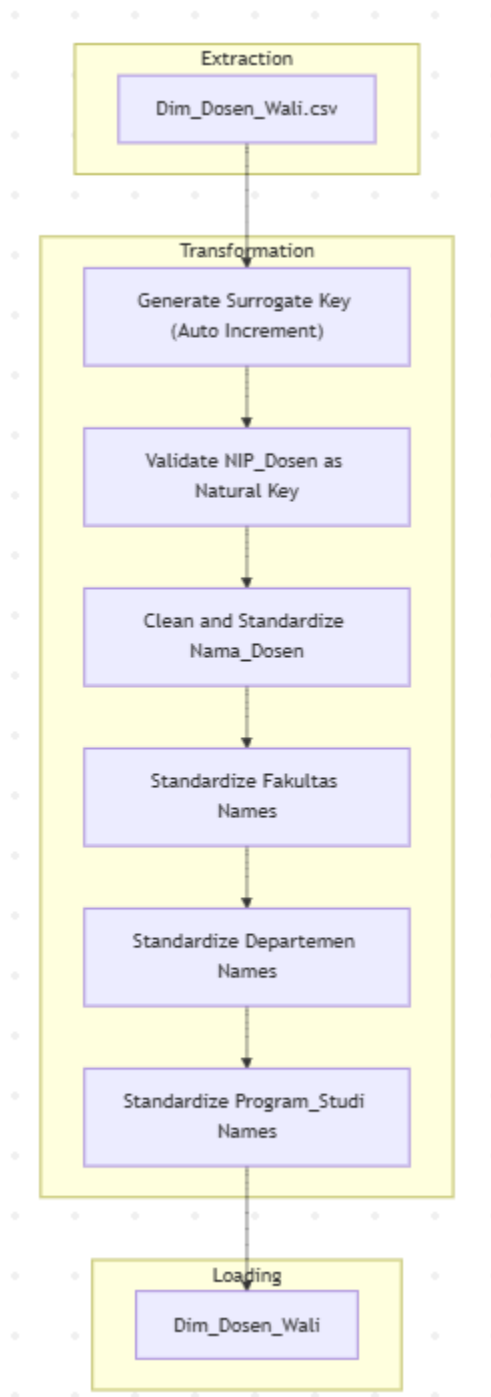
ID_Waktu	ID_Waktu_NK	Copy original ID as natural key
Tanggal	Tanggal	Validate date format (YYYY-MM-DD)
Hari	Hari	Standardize day names (capitalize first letter)
Bulan	Bulan	Standardize month names (capitalize first letter)
Periode	Periode	Validate period format (YYYY-N)
N/A	Tahun	Extract year from Tanggal
N/A	Kuartal	Derive quarter from Bulan

Tabel Dosen wali / Dim_Dosen_wali

High level plan



Detailed Level Plan



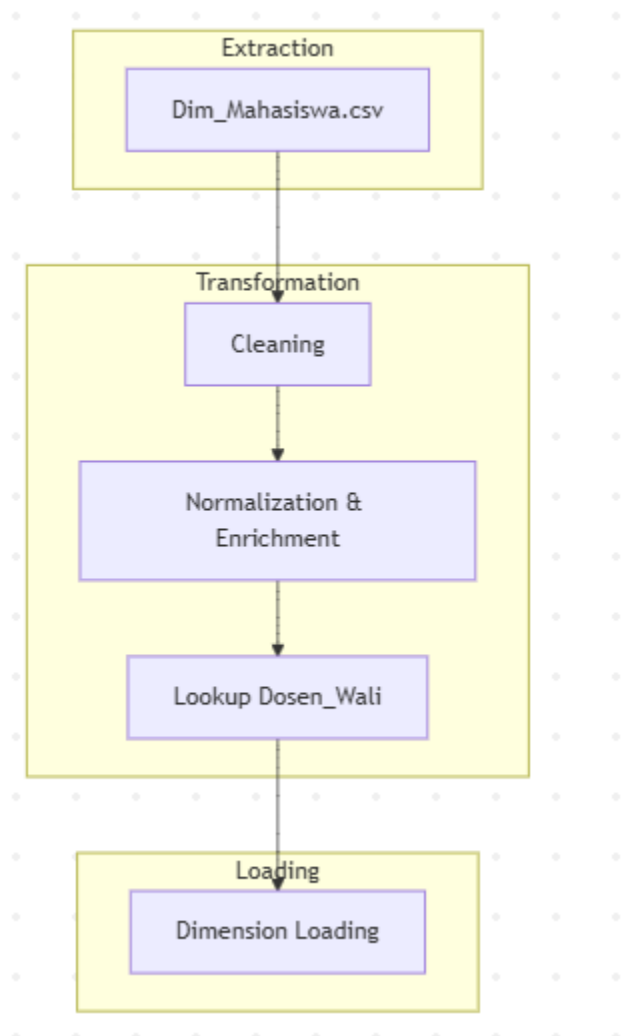
Source to target attribute mapping

Source Column	Target Column	Transformation Description
N/A	Dosen_SK	Generate surrogate key (auto increment)
NIP_Dosen	NIP_Dosen	Validate as natural key (ensure uniqueness)

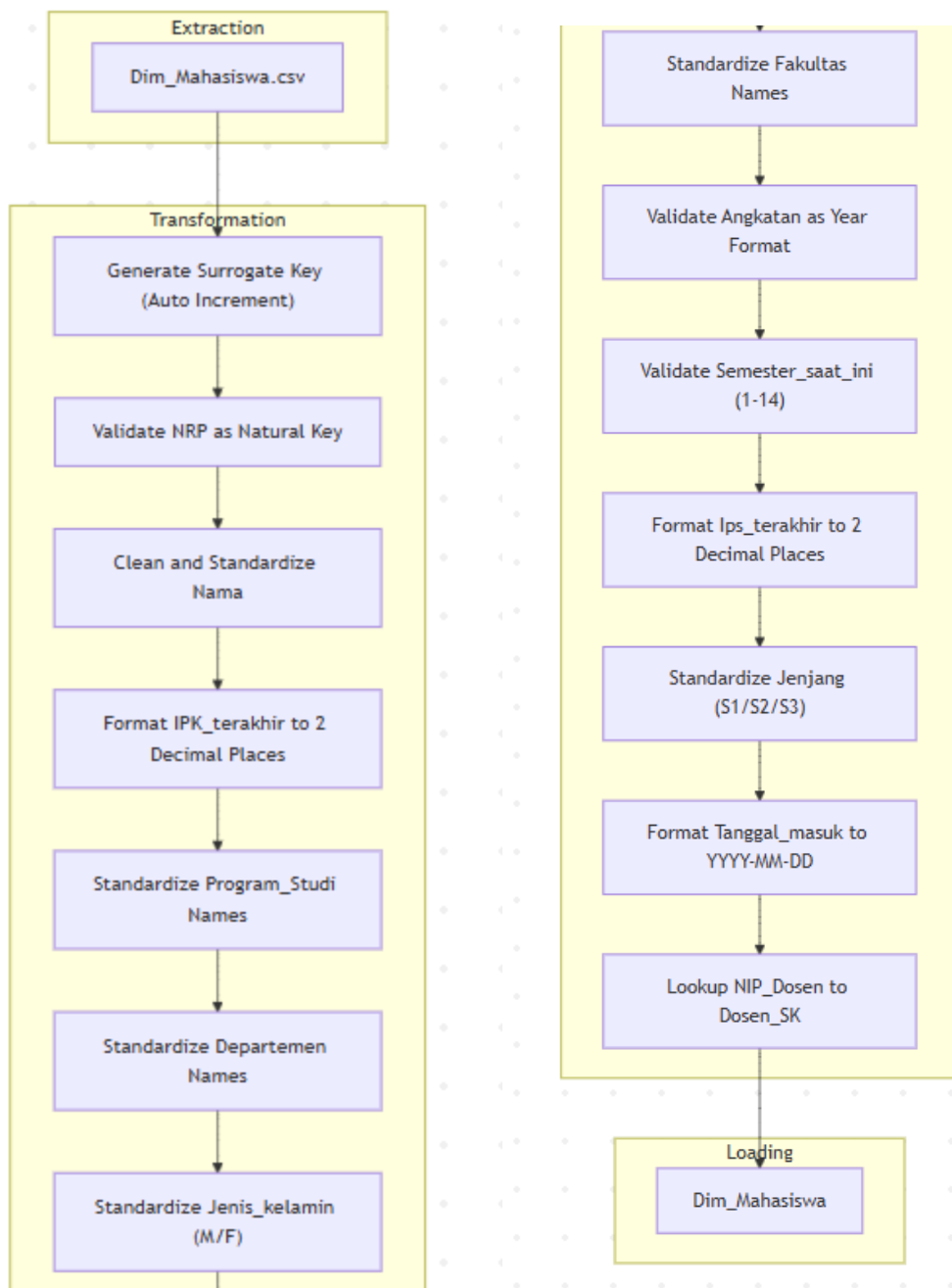
Nama_Dosen	Nama_Dosen	Clean and standardize name format
Fakultas	Fakultas	Standardize faculty names (consistent abbreviations)
Departemen	Departemen	Standardize department names
Program Studi	Program Studi	Standardize program names

Tabel dimensi Mahasiswa / Dim_Mahasiswa

High level plan



Detailed Level Plan



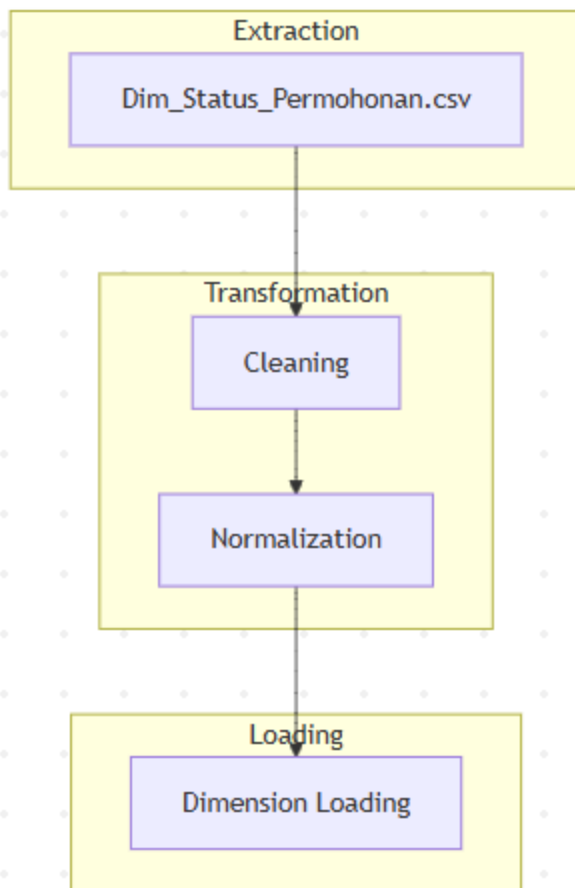
Source to target attribute mapping

Source Column	Target Column	Transformation Description
N/A	Mahasiswa_SK	Generate surrogate key (auto increment)
NRP	NRP	Validate as natural key (ensure uniqueness)
Nama	Nama	Clean and standardize name format
IPK_terakhir	IPK_terakhir	Format to 2 decimal places
Program_Studi	Program_Studi	Standardize program names

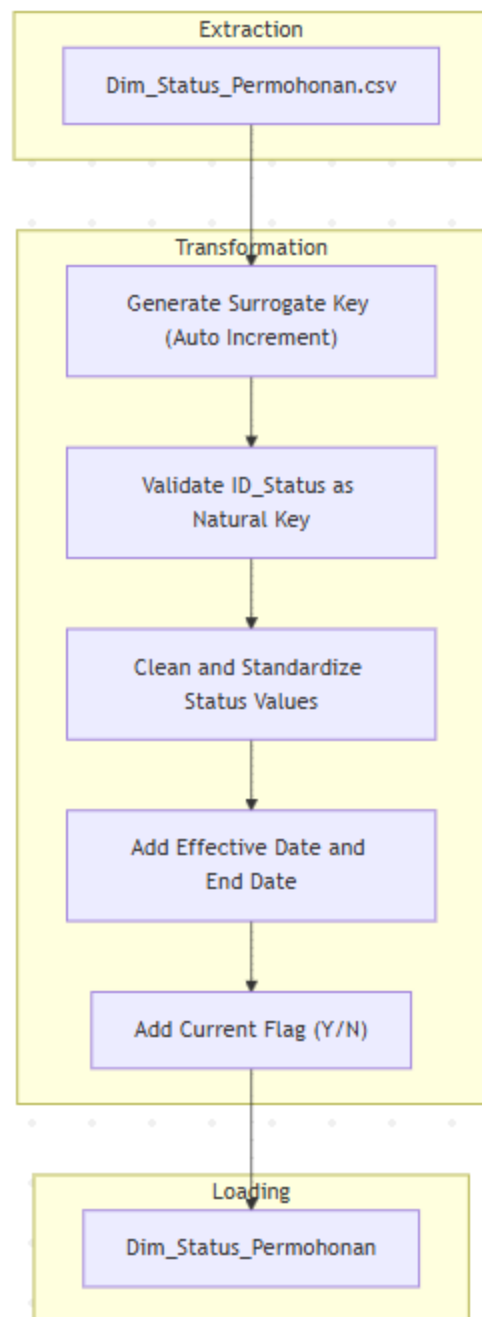
Departemen	Departemen	Standardize department names
Jenis kelamin	Jenis_kelamin	Standardize to "Laki-laki" or "Perempuan"
Fakultas	Fakultas	Standardize faculty names
Angkatan	Angkatan	Validate as year format (YYYY)
Semester_saat_ini	Semester_saat_ini	Validate range (1-14)
Ips_terakhir	Ips_terakhir	Format to 2 decimal places
Jenjang	Jenjang	Standardize to S1, S2, or S3
Tanggal masuk	Tanggal_masuk	Format to YYYY-MM-DD
NIP_Dosen	Dosen_SK	Lookup to Dim_Dosen_Wali for surrogate key

Table dimensi status permohonan / Dim_status_permohonan

High level plan



Detailed Level Plan



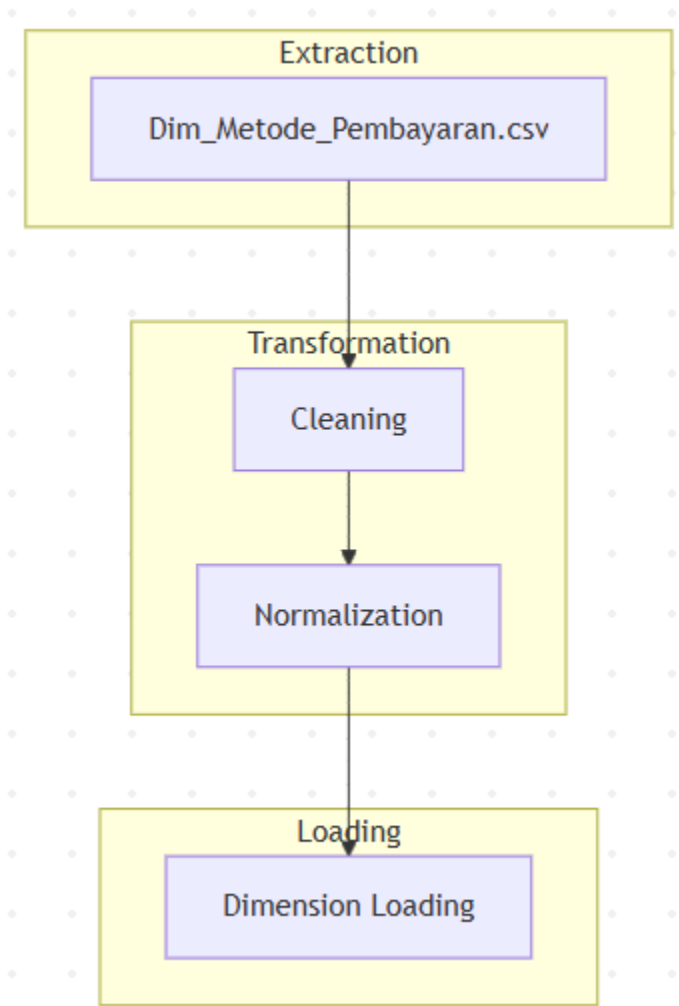
Source to target attribute mapping

Source Column	Target Column	Transformation Description
N/A	Status_SK	Generate surrogate key (auto increment)
ID_Status	ID_Status	Validate as natural key (ensure uniqueness)
Status	Status	Clean and standardize status values
N/A	Effective_Date	Add current date as effective date

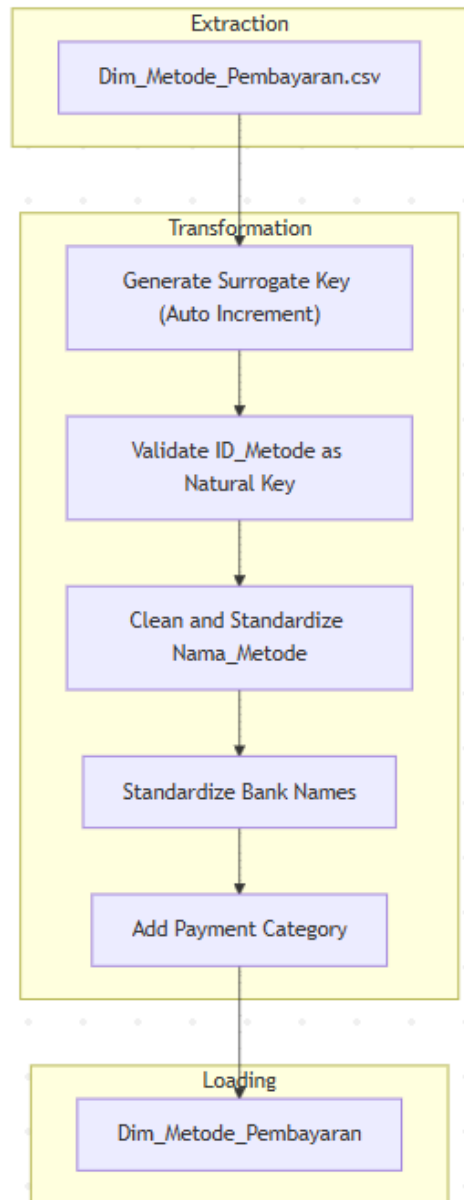
N/A	End_Date	Add default end date (9999-12-31)
N/A	Current_Flag	Add current flag (Y)

Tabel dimensi metode pembayaran / Dim_Metode_pembayaran

High level plan



Detailed Level Plan

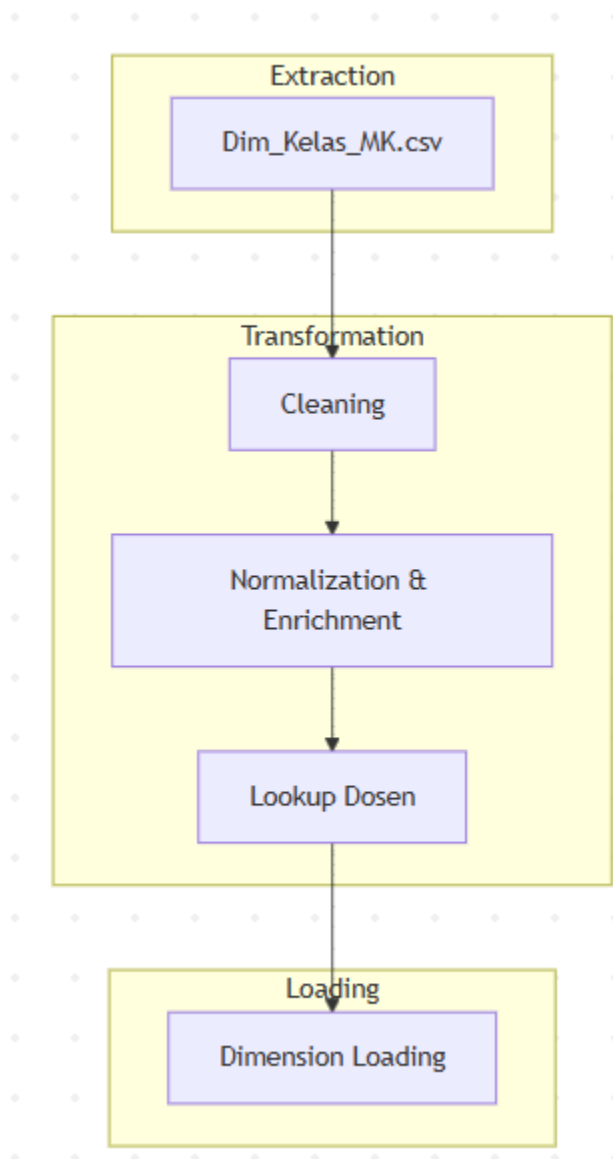


Source to target attribute mapping

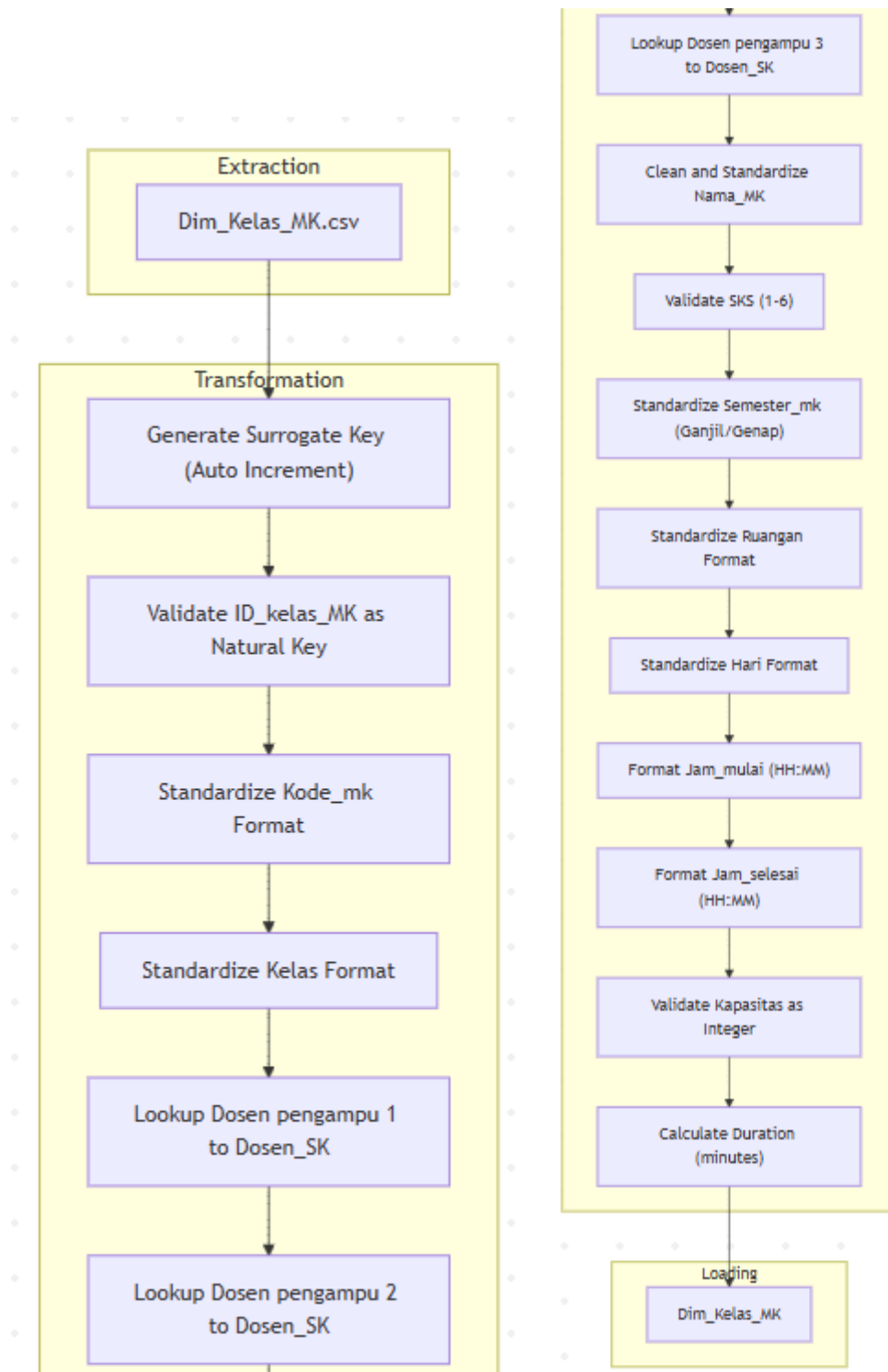
Source Column	Target Column	Transformation Description
N/A	Metode_SK	Generate surrogate key (auto increment)
ID_Metode	ID_Metode	Validate as natural key (ensure uniqueness)
Nama_Metode	Nama_Metode	Clean and standardize method names
Bank	Bank	Standardize bank names
N/A	Kategori_Pembayaran	Derive from Nama_Metode (Online/Offline)

Tabel dimensi Kelas_MK / Dim_Kelas_MK

High level plan



Detailed Level Plan



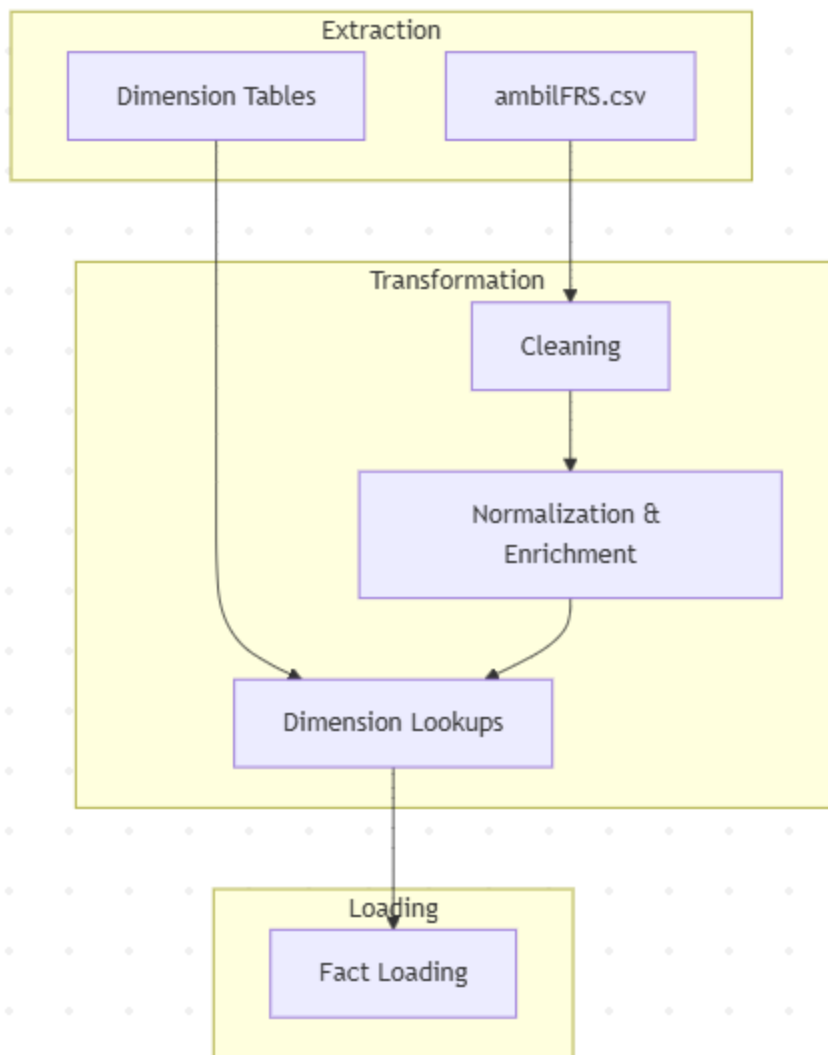
Source to target attribute mapping

Source Column	Target Column	Transformation Description
N/A	Kelas_MK_SK	Generate surrogate key (auto increment)
ID_kelas_MK	ID_kelas_MK	Validate as natural key (ensure uniqueness)

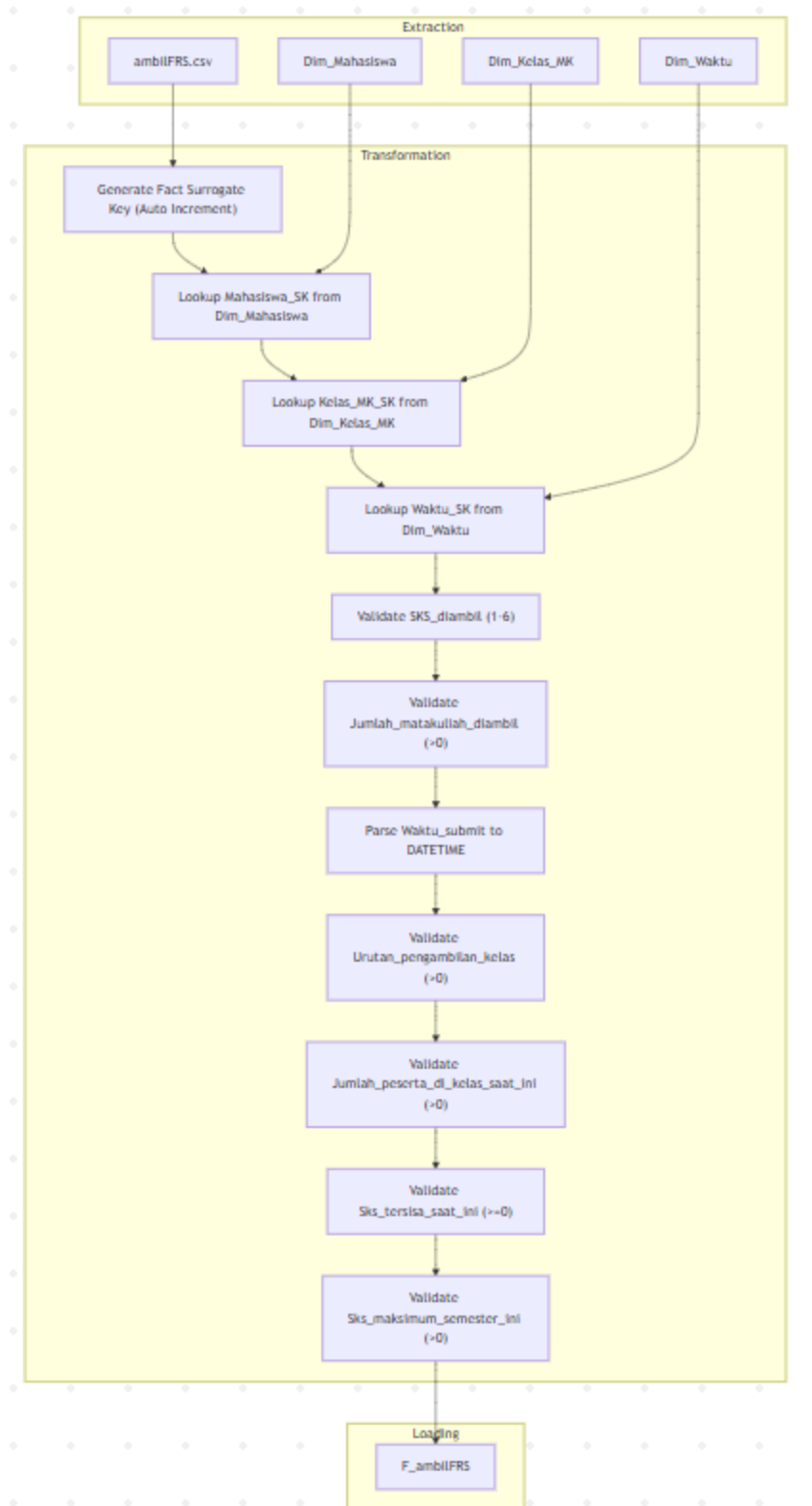
Kode_mk	Kode_mk	Standardize course code format
Kelas	Kelas	Standardize class format (single letter)
Dosen pengampu 1	Dosen_SK_1	Lookup to Dim_Dosen_Wali for surrogate key
Dosen pengampu 2	Dosen_SK_2	Lookup to Dim_Dosen_Wali for surrogate key
Dosen pengampu 3	Dosen_SK_3	Lookup to Dim_Dosen_Wali for surrogate key
Nama_MK	Nama_MK	Clean and standardize course name
SKS	SKS	Validate range (1-6)
Semester_mk	Semester_mk	Standardize to "Ganjil" or "Genap"
Ruangan	Ruangan	Standardize room format
Hari	Hari	Standardize day format
Jam_mulai	Jam_mulai	Format to HH:MM
Jam_selesai	Jam_selesai	Format to HH:MM
Kapasitas	Kapasitas	Validate as integer
N/A	Durasi	Calculate from Jam_mulai and Jam_selesai (minutes)

Tabel fakta / ambilFRS

High level plan



Detailed Level Plan



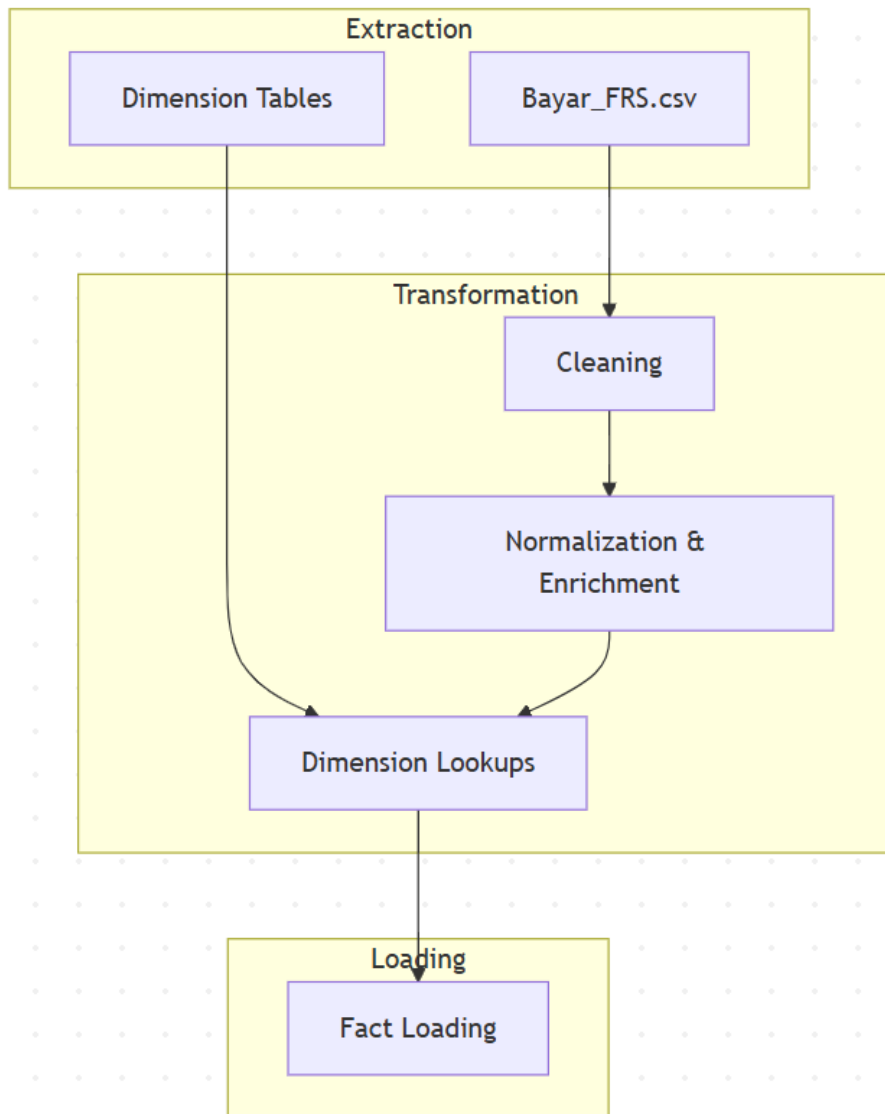
Source to target attribute mapping

Source Column	Target Column	Transformation Description
---------------	---------------	----------------------------

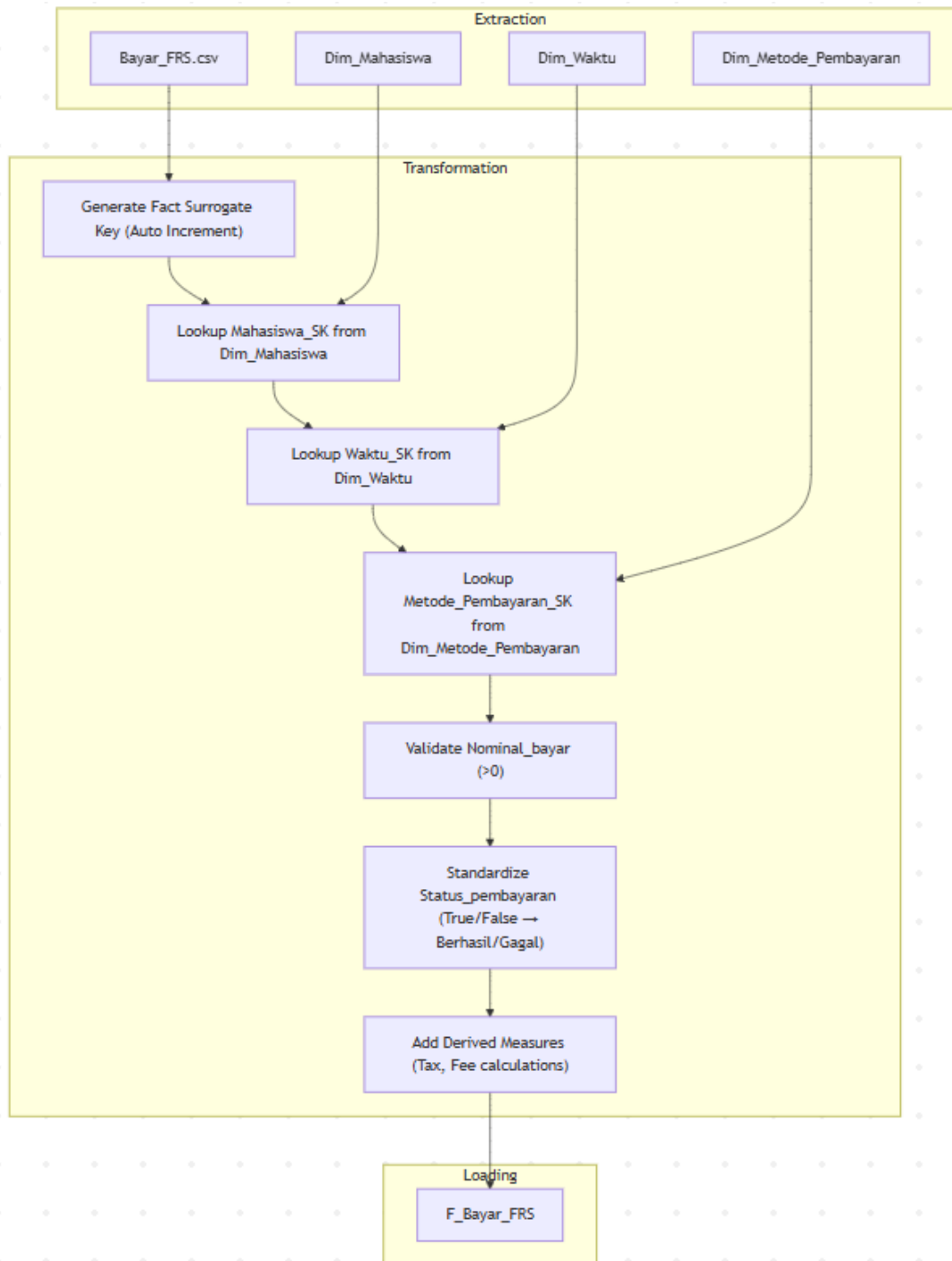
N/A	ambilFRS_SK	Generate fact surrogate key (auto increment)
Mahasiswa_SK	Mahasiswa_SK	Lookup from Dim_Mahasiswa
Kelas_MK_SK	Kelas_MK_SK	Lookup from Dim_Kelas_MK
Waktu_SK	Waktu_SK	Lookup from Dim_Waktu
SKS_diambil	SKS_diambil	Validate range (1-6)
Jumlah_matakuliah_diambil	Jumlah_matakuliah_diambil	Validate positive integer
Waktu_submit	Waktu_submit	Parse to DATETIME format
Urutan_pengambilan_kelas	Urutan_pengambilan_kelas	Validate positive integer
Jumlah_peserta_di_kelas_saat_ini	Jumlah_peserta_di_kelas_saat_ini	Validate positive integer
Sks_tersisa_saat_ini	Sks_tersisa_saat_ini	Validate non-negative integer
Sks_maksimum_semester_ini	Sks_maksimum_semester_ini	Validate positive integer

Table fakta Bayar_FRS

High level plan



Detailed Level Plan



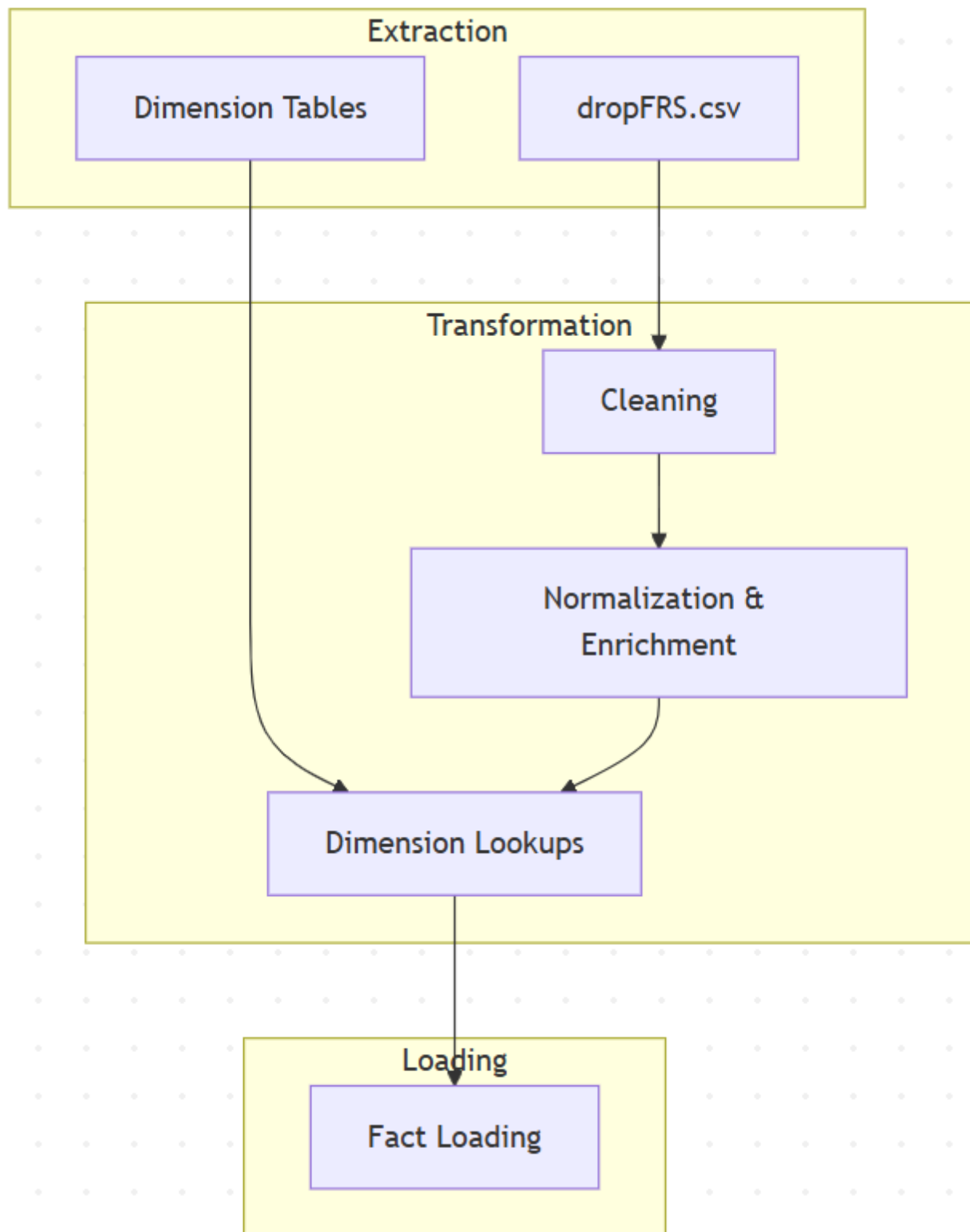
Source to target attribute mapping

Source Column	Target Column	Transformation Description
N/A	Bayar_FRS_SK	Generate fact surrogate key (auto increment)
Mahasiswa_SK	Mahasiswa_SK	Lookup from Dim_Mahasiswa
Waktu_SK	Waktu_SK	Lookup from Dim_Waktu

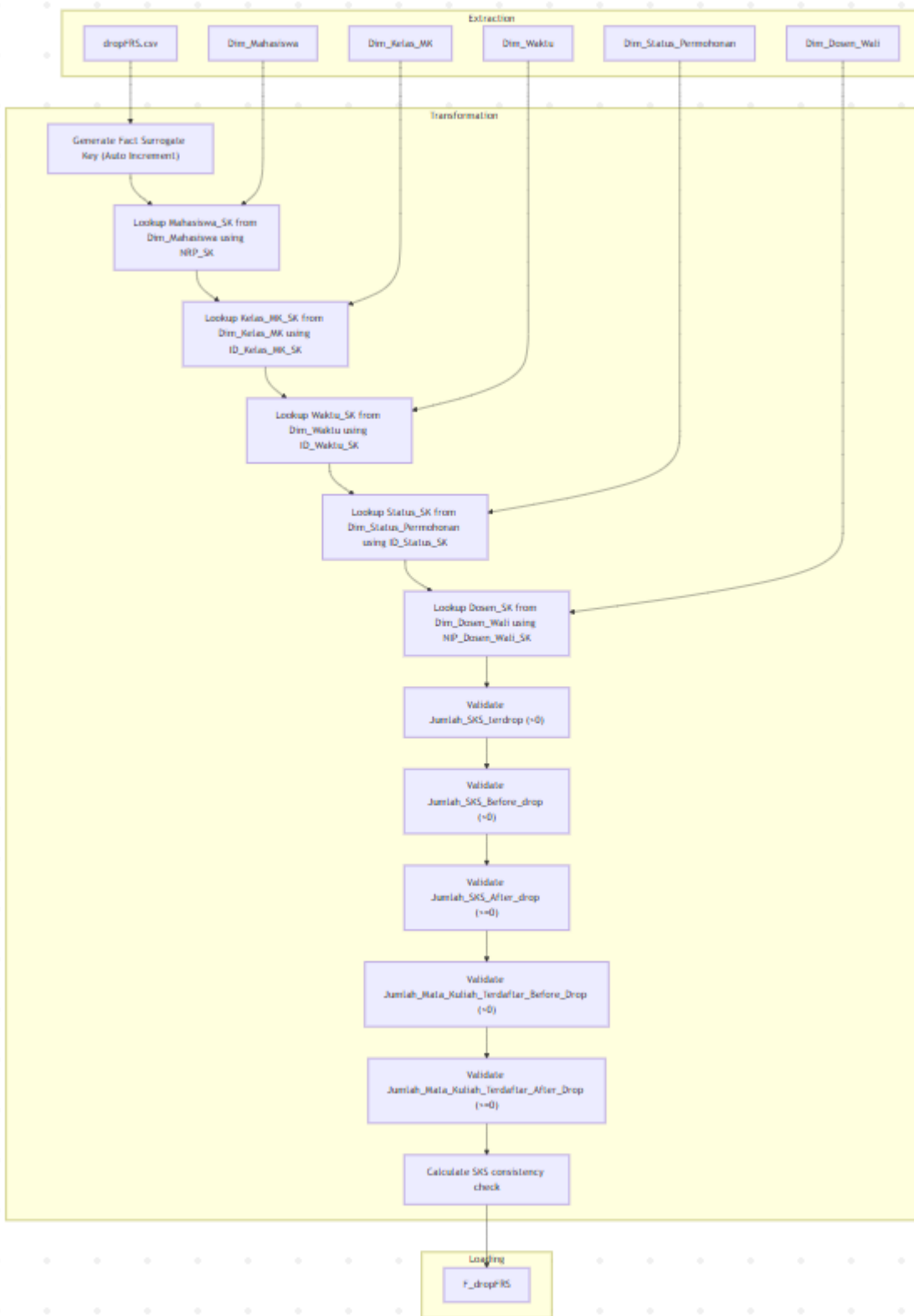
Metode_Pembayaran_S K	Metode_Pembayaran_S K	Lookup from Dim_Metode_Pembayaran
Nominal_bayar	Nominal_bayar	Validate positive amount
Status_pembayaran	Status_pembayaran	Standardize: True → 'Berhasil', False → 'Gagal'

Tabel fakta Drop_FRS

High level plan



Detailed Level Plan



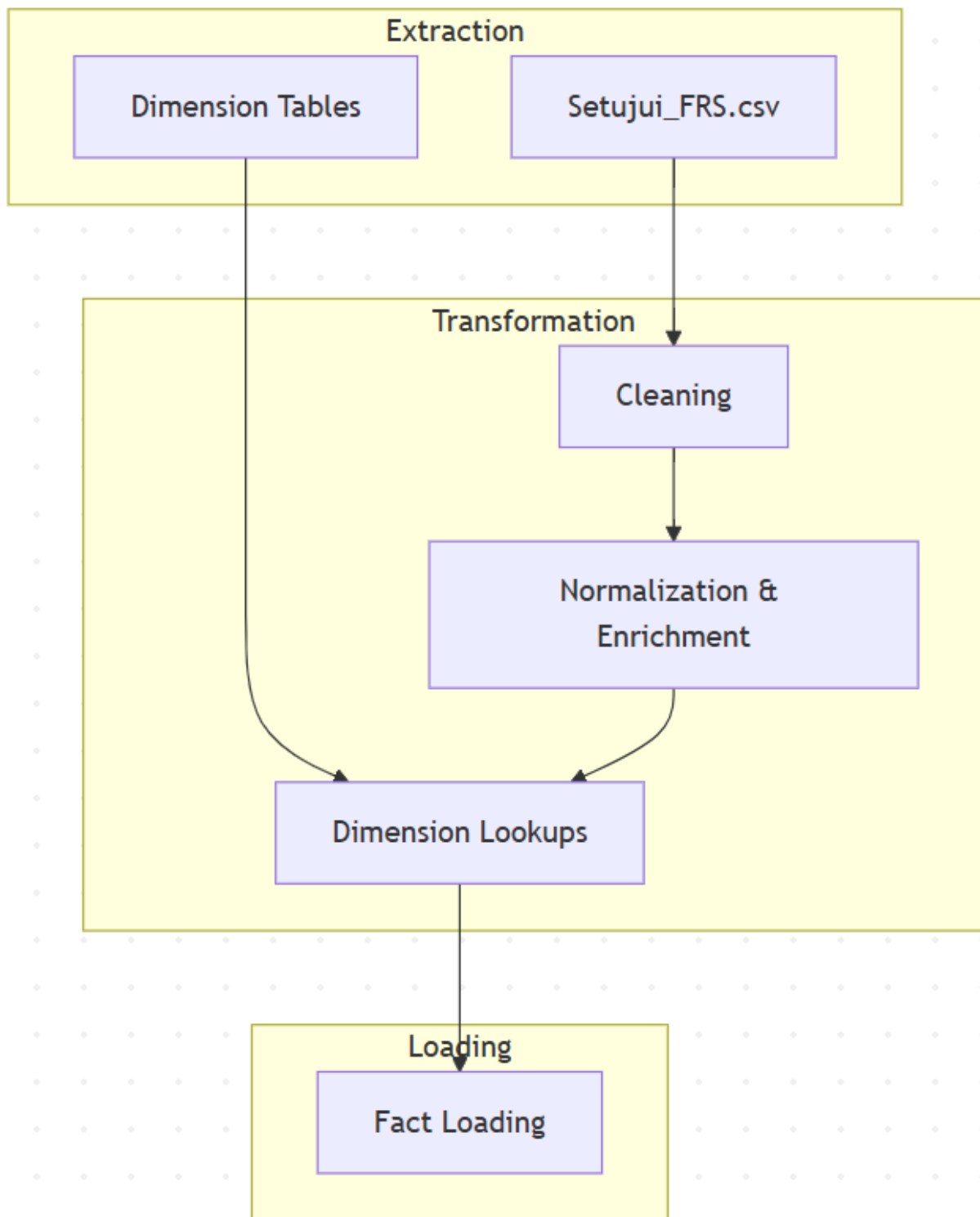
Source to target attribute mapping

Source Column	Target Column	Transformation Description
---------------	---------------	----------------------------

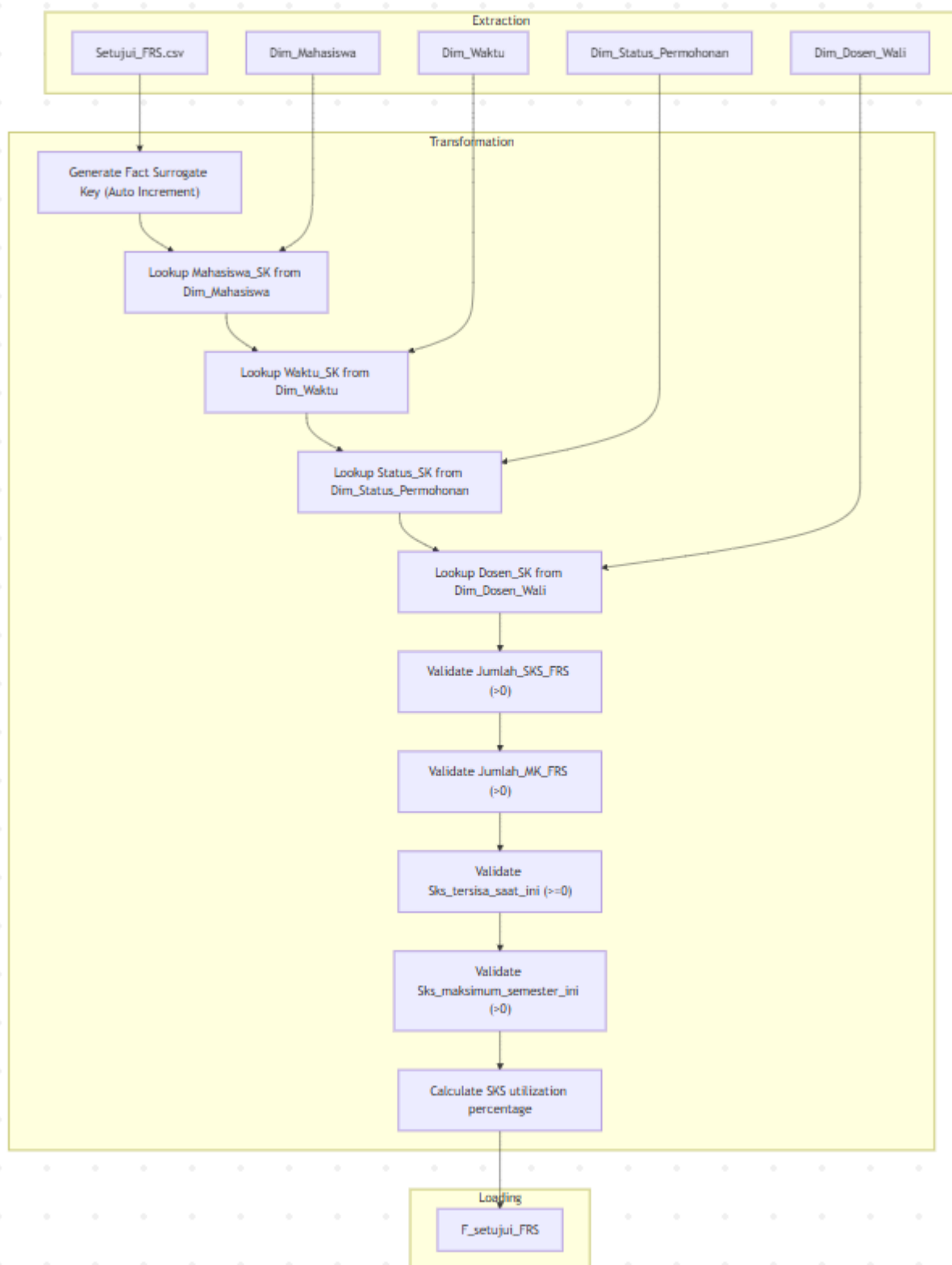
N/A	dropFRS_SK	Generate fact surrogate key (auto increment)
NRP_SK	Mahasiswa_SK	Lookup from Dim_Mahasiswa using NRP
ID_Kelas_MK_SK	Kelas_MK_SK	Lookup from Dim_Kelas_MK
ID_Waktu_SK	Waktu_SK	Lookup from Dim_Waktu
ID_Status_SK	Status_SK	Lookup from Dim_Status_Permohonan
NIP_Dosen_Wali_SK	Dosen_SK	Lookup from Dim_Dosen_Wali
Jumlah_SKS_terdrop	Jumlah_SKS_terdrop	Validate positive integer
Jumlah_SKS_Before_drop	Jumlah_SKS_Before_drop	Validate positive integer
Jumlah_SKS_After_drop	Jumlah_SKS_After_drop	Validate non-negative integer
Jumlah_Mata_Kuliah_Terdaftar_Before_Drop	Jumlah_MK_Before_Drop	Validate positive integer
Jumlah_Mata_Kuliah_Terdaftar_After_Drop	Jumlah_MK_After_Drop	Validate non-negative integer

Table fakta Setujui_FRS

High level plan



Detailed Level Plan



Source to target attribute mapping

Source Column	Target Column	Transformation Description
N/A	setujui_FRS_SK	Generate fact surrogate key (auto increment)
Mahasiswa_SK	Mahasiswa_SK	Lookup from Dim_Mahasiswa
Waktu_SK	Waktu_SK	Lookup from Dim_Waktu
Status_Permohonan_SK	Status_SK	Lookup from Dim_Status_Permohonan
Dosen_Wali_SK	Dosen_SK	Lookup from Dim_Dosen_Wali
Jumlah_SKS_FRS	Jumlah_SKS_FRS	Validate positive integer
Jumlah_MK_FRS	Jumlah_MK_FRS	Validate positive integer
Sks_tersisa_saat_ini	Sks_tersisa_saat_ini	Validate non-negative integer
Sks_maksimum_semester_ini	Sks_maksimum_semester_ini	Validate positive integer

ETL Implementation

One time - Historical ETL

Dimensi waktu

```
DLH > Dim_waktuHistorical_V2.js > ...
287
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[Running] node "c:\Users\micle\Desktop\DLH\Dim_waktuHistorical_V2.js"
[2025-05-27T20:16:28.049Z] [START] Starting ETL process for Dim_Waktu
[2025-05-27T20:16:28.051Z] [EXTRACTION] Fetching data from Dim_Waktu CSV
[2025-05-27T20:16:28.779Z] [EXTRACTION] Extracted 100 records
[2025-05-27T20:16:28.779Z] [PROFILING] Starting data profiling for Dim_Waktu
Dim_Waktu Profiling Results:
Total Records: 100
Null Values: { ID_Waktu: 0, Tanggal: 0, Hari: 0, Bulan: 0, Periode: 0 }
Unique Values: { ID_Waktu: 100, Tanggal: 91, Hari: 7, Bulan: 12, Periode: 5 }
Data Types: {
  ID_Waktu: 'numeric',
  Tanggal: 'date',
  Hari: 'string',
  Bulan: 'string',
  Periode: 'string'
}
Date Range: { min: '2023-05-23', max: '2025-05-12' }
[2025-05-27T20:16:28.782Z] [TRANSFORMATION] Starting transformation for Dim_Waktu
[2025-05-27T20:16:28.784Z] [TRANSFORMATION] Transformed 100 records
[2025-05-27T20:16:28.784Z] [LOADING] Starting loading for Dim_Waktu
[2025-05-27T20:16:28.784Z] [LOADING] Loaded 100 records successfully, 0 errors
[2025-05-27T20:16:28.784Z] [EXPORT] Exporting transformed data to CSV
Output written to Dim_Waktu_Output.csv
[2025-05-27T20:16:28.787Z] [COMPLETE] ETL process for Dim_Waktu completed successfully

ETL Process Result: {
  status: 'success',
  extracted: 100,
  transformed: 100,
  loaded: 100,
  errors: 0,
  transformedData: [
    {
      Waktu_SK: 1,
      ID_Waktu: '1',
      Tanggal: '2023-11-12',
      Tahun: 2023,
```



```
DLH > Dim_Waktu_Output.csv
1 Waktu_SK_ID_Waktu,Tanggal,Tahun,Bulan_Angka,Hari_Angka,Kuartal,Minggu_Tahun,Hari_Dalam_Tahun,Hari,Bulan,Periode,Tahun_Akademik,Semester,Created_Date,ETL_Batch_ID
2 1,1,2023-11-12,2023,11,12,4,45,316,Jumat,Agustus,2023-2,2023,Genap,2025-05-27,BATCH_20250527_001
3 2,2,2023-07-30,2023,7,30,3,30,211,Senin,Agustus,2023-2,2023,Genap,2025-05-27,BATCH_20250527_001
4 3,3,2024-01-24,2024,1,24,1,4,24,Rabu,Maret,2024-1,2024,Ganjil,2025-05-27,BATCH_20250527_001
5 4,4,2023-08-19,2023,8,19,3,33,231,Jumat,Desember,2023-2,2023,Genap,2025-05-27,BATCH_20250527_001
6 5,5,2023-05-23,2023,5,23,2,21,143,Selasa,Juli,2023-1,2023,Ganjil,2025-05-27,BATCH_20250527_001
7 6,6,2023-12-13,2023,12,13,4,50,347,Senin,April,2023-2,2023,Genap,2025-05-27,BATCH_20250527_001
8 7,7,2024-03-27,2024,3,27,1,13,87,Sabtu,September,2024-1,2024,Ganjil,2025-05-27,BATCH_20250527_001
9 8,8,2023-06-01,2023,6,1,2,22,152,Rabu,Juli,2023-1,2023,Ganjil,2025-05-27,BATCH_20250527_001
10 9,9,2024-05-06,2024,5,6,2,19,127,Kamis,April,2024-1,2024,Ganjil,2025-05-27,BATCH_20250527_001
11 10,10,2024-07-19,2024,7,19,3,29,201,Senin,Januari,2024-2,2024,Genap,2025-05-27,BATCH_20250527_001
12 11,11,2025-01-25,2025,1,25,1,4,25,Jumat,Oktober,2025-1,2025,Ganjil,2025-05-27,BATCH_20250527_001
13 12,12,2023-09-15,2023,9,15,3,37,258,Jumat,September,2023-2,2023,Genap,2025-05-27,BATCH_20250527_001
14 13,13,2023-08-28,2023,8,28,3,35,240,Selasa,Februari,2023-2,2023,Genap,2025-05-27,BATCH_20250527_001
15 14,14,2024-11-23,2024,11,23,4,47,328,Minggu,Maret,2024-2,2024,Genap,2025-05-27,BATCH_20250527_001
16 15,15,2025-02-09,2025,2,9,1,6,40,Minggu,Oktober,2025-1,2025,Ganjil,2025-05-27,BATCH_20250527_001
17 16,16,2024-07-30,2024,7,30,3,31,212,Selasa,Februari,2024-2,2024,Genap,2025-05-27,BATCH_20250527_001
18 17,17,2025-02-24,2025,2,24,1,9,55,Kamis,April,2025-1,2025,Ganjil,2025-05-27,BATCH_20250527_001
19 18,18,2024-08-13,2024,8,13,3,33,226,Minggu,Februari,2024-2,2024,Genap,2025-05-27,BATCH_20250527_001
20 19,19,2024-12-03,2024,12,3,4,49,338,Senin,Januari,2024-2,2024,Genap,2025-05-27,BATCH_20250527_001
21 20,20,2025-05-06,2025,5,6,2,19,126,Selasa,Desember,2025-1,2025,Ganjil,2025-05-27,BATCH_20250527_001
```

```
const { parse } = require("csv-parse/sync");
```

```
const { createObjectCsvWriter } = require("csv-writer");
```

```
const fetch = (...args) => import('node-fetch').then(({ default: fetch }) => fetch(...args));
```

```
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}
```

```
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}
```

```
async function exportToCSV(data, outputPath) {
  if (!data || data.length === 0) {
    logStep("EXPORT", "No data to write to CSV.")
    return
  }
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))
```

```
const csvWriter = createObjectCsvWriter({
  path: outputPath,
  header: headers,
```

```

    })

    await csvWriter.writeRecords(data)
    console.log(`Output written to ${outputPath}`)
  }

function profileDimWaktu(data) {
  logStep("PROFILING", "Starting data profiling for Dim_Waktu")

  const metrics = {
    totalRecords: data.length,
    nullValues: {},
    uniqueValues: {},
    dataTypes: {},
    dateRange: { min: null, max: null },
  }

  const columns = Object.keys(data[0] || {})
  columns.forEach((col) => {
    metrics.nullValues[col] = 0
    metrics.uniqueValues[col] = new Set()
  })

  data.forEach((record) => {
    columns.forEach((col) => {
      const value = record[col]

      if (value === null || value === undefined || value === "") {
        metrics.nullValues[col]++
      } else {
        metrics.uniqueValues[col].add(value)

        if (col === "Tanggal" && /^\d{4}-\d{2}-\d{2}$/.test(value)) {
          metrics.dataTypes[col] = "date"
          const dateValue = new Date(value)
          if (!metrics.dateRange.min || dateValue < metrics.dateRange.min) {
            metrics.dateRange.min = dateValue
          }
          if (!metrics.dateRange.max || dateValue > metrics.dateRange.max) {
            metrics.dateRange.max = dateValue
          }
        } else if (!isNaN(value)) {
          metrics.dataTypes[col] = "numeric"
        } else {

```

```

        metrics.dataTypes[col] = "string"
    }
}
})
})

```

```

Object.keys(metrics.uniqueValues).forEach((col) => {
    metrics.uniqueValues[col] = metrics.uniqueValues[col].size
})

```

```

console.log("Dim_Waktu Profiling Results:")
console.log("Total Records:", metrics.totalRecords)
console.log("Null Values:", metrics.nullValues)
console.log("Unique Values:", metrics.uniqueValues)
console.log("Data Types:", metrics.dataTypes)
console.log("Date Range:", {
    min: metrics.dateRange.min?.toISOString().split("T")[0],
    max: metrics.dateRange.max?.toISOString().split("T")[0],
})

```

```

return metrics
}

```

```

function getWeekNumber(date) {
    const d = new Date(Date.UTC(date.getFullYear(), date.getMonth(), date.getDate()))
    const dayNum = d.getUTCDay() || 7
    d.setUTCDate(d.getUTCDate() + 4 - dayNum)
    const yearStart = new Date(Date.UTC(d.getUTCFullYear(), 0, 1))
    return Math.ceil(((d - yearStart) / 86400000 + 1) / 7)
}

```

```

function transformDimWaktu(data) {
    logStep("TRANSFORMATION", "Starting transformation for Dim_Waktu")

```

```

    return data.map((record, index) => {
        const transformed = {}

```

```

        transformed.Waktu_SK = index + 1

```

```

        transformed.ID_Waktu = record.ID_Waktu

```

```

        if (record.Tanggal && /\d{4}-\d{2}-\d{2}$/.test(record.Tanggal)) {
            transformed.Tanggal = record.Tanggal
            const date = new Date(record.Tanggal)

```

```

if (isNaN(date.getTime())) {
    transformed.Tanggal = null
    transformed.Tahun = null
    transformed.Bulan_Angka = null
    transformed.Hari_Angka = null
    transformed.Kuartal = null
    transformed.Minggu_Tahun = null
    transformed.Hari_Dalam_Tahun = null
} else {
    transformed.Tahun = date.getFullYear()
    transformed.Bulan_Angka = date.getMonth() + 1
    transformed.Hari_Angka = date.getDate()
    transformed.Kuartal = Math.ceil((date.getMonth() + 1) / 3)
    transformed.Minggu_Tahun = getWeekNumber(date)
    transformed.Hari_Dalam_Tahun = Math.floor((date - new Date(date.getFullYear(), 0, 0)) /
(1000 * 60 * 60 * 24))
}
} else {
    transformed.Tanggal = null
    transformed.Tahun = null
    transformed.Bulan_Angka = null
    transformed.Hari_Angka = null
    transformed.Kuartal = null
    transformed.Minggu_Tahun = null
    transformed.Hari_Dalam_Tahun = null
}

transformed.Hari = record.Hari ? record.Hari.charAt(0).toUpperCase() +
record.Hari.slice(1).toLowerCase() : ""

transformed.Bulan = record.Bulan ? record.Bulan.charAt(0).toUpperCase() +
record.Bulan.slice(1).toLowerCase() : ""

if (record.Periode && /^d{4}-d$/.test(record.Periode.toString())) {
    transformed.Periode = record.Periode.toString()
    const [year, semester] = transformed.Periode.split("-")
    transformed.Tahun_Akademik = parseInt(year)
    transformed.Semester = parseInt(semester) === 1 ? "Ganjil" : "Genap"
} else {
    transformed.Periode = record.Periode ? record.Periode.toString() : ""
    transformed.Tahun_Akademik = null
    transformed.Semester = ""
}

```

```

    transformed.Created_Date = new Date().toISOString().split("T")[0]
    transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-]/g,
    "").split("T")[0]}_001`

    return transformed
  })
}

```

```

function loadDimWaktu(transformedData) {
  logStep("LOADING", "Starting loading for Dim_Waktu")

```

```

  const loadResults = {
    inserted: transformedData.length,
    errors: 0,
    errorRecords: [],
  }

```

```

  transformedData.forEach((record, index) => {
    if (!record.Tanggal || !record.Hari || !record.Bulan) {
      loadResults.errors++
      loadResults.errorRecords.push({
        index: index,
        record: record,
        error: "Missing required fields",
      })
      loadResults.inserted--
    }
  })

```

```

  logStep("LOADING", `Loaded ${loadResults.inserted} records successfully,
  ${loadResults.errors} errors`)

```

```

  if (loadResults.errors > 0) {
    console.log("Error Records:", loadResults.errorRecords)
  }

```

```

  return loadResults
}

```

```

async function etlDimWaktu() {
  try {
    logStep("START", "Starting ETL process for Dim_Waktu")

    logStep("EXTRACTION", "Fetching data from Dim_Waktu CSV")

```

```

const rawData = await fetchCSVData(
  "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Waktu%20%282%29-bBN4wnuL2mryLiopDAgmhsKt8i90a2.csv"
)
logStep("EXTRACTION", `Extracted ${rawData.length} records`)

const profilingResults = profileDimWaktu(rawData)

const transformedData = transformDimWaktu(rawData)
logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`)

const loadResults = loadDimWaktu(transformedData)

logStep("EXPORT", "Exporting transformed data to CSV")
await exportToCSV(transformedData, "Dim_Waktu_Output.csv")

logStep("COMPLETE", "ETL process for Dim_Waktu completed successfully")

return {
  status: "success",
  extracted: rawData.length,
  transformed: transformedData.length,
  loaded: loadResults.inserted,
  errors: loadResults.errors,
  transformedData: transformedData.slice(0, 5),
}
} catch (error) {
  logStep("ERROR", `ETL process failed: ${error.message}`)
  console.error(error)
  return {
    status: "error",
    message: error.message,
  }
}
}

(async () => {
  const result = await etlDimWaktu()
  console.log("\nETL Process Result:", result)
})();

```

Dimensi Dosen_wali

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[Running] node "c:\Users\micle\Desktop\DLH\Dim_dosen_wali_historical.js"
[2025-05-27T20:25:45.945Z] [START] Starting ETL process for Dim_Dosen_Wali
[2025-05-27T20:25:45.947Z] [EXTRACTION] Fetching data from Dim_Dosen_Wali CSV
[2025-05-27T20:25:46.936Z] [EXTRACTION] Extracted 20 records
[2025-05-27T20:25:46.937Z] [PROFILING] Starting data profiling for Dim_Dosen_Wali
Dim_Dosen_Wali Profiling Results:
Total Records: 20
Null Values: {
  NIP_Dosen: 0,
  Nama_Dosen: 0,
  Fakultas: 0,
  Departemen: 0,
  Program_Studi: 0
}
Unique Values: {
  NIP_Dosen: 20,
  Nama_Dosen: 20,
  Fakultas: 1,
  Departemen: 1,
  Program_Studi: 1
}
DLH > Dim_Dosen_Wali_Output.csv
1 Dosen_SK,NIP_Dosen>Nama_Dosen,Fakultas,Departemen,Program_Studi,Has_Academic_Title,Display_Name,Faculty_Department,Effective_Date,End_Date,Current_Flag,Version_Number,
2 1,95822412,"Ozy Haryanto, S.I.Kom",FTSPK,Informatika,S1 Teknik Mesin,N,"Ozy Haryanto, S.I.Kom",FTSPK - Informatika,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATC
3 2,24942683,Digdaya Narpati,FTSPK,Informatika,S1 Sistem Informasi,N,Digdaya Narpati,FTSPK - Informatika,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_0
4 3,13356886,Tiara Anggriawan,FTEIC,Elektro,S1 Teknik Mesin,N,Tiara Anggriawan,FTEIC - Elektro,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
5 4,46913810,"Dr. Rafid Irawan, M.TI.",FTM,Elektro,S1 Teknik Mesin,Y,"Rafid wan, .",FTM - Elektro,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
6 5,42868828,Asmadi Mayasari,FTSPK,Mesin,S1 Sistem Informasi,N,Asmadi Mayasari,FTSPK - Mesin,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
7 6,39958838,Sutan Prima Habibi,FTI,Kelautan,S1 Sistem Informasi,N,Sutan Prima Habibi,FTI - Kelautan,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
8 7,28728463,H. Malik Pradana,FTI,Kelautan,S1 Teknik Sipil,N,H. Malik Pradana,FTI - Kelautan,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
9 8,23756669,Dwi Wahyuni,FTSPK,Informatika,S1 Teknik Elektro,N,Dwi Wahyuni,FTSPK - Informatika,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
10 9,83197857,"H. Daryani Mayasari, S.Kom",FTI,Elektro,S1 Informatika,N,"H. Daryani Mayasari, S.Kom",FTI - Elektro,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_2
11 10,21668732,Tiara Yulianti,FTM,Kelautan,S1 Sistem Informasi,N,Tiara Yulianti,FTM - Kelautan,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
12 11,89254563,Maman Palastri,FTSPK,Sipil,S1 Informatika,N,Maman Palastri,FTSPK - Sipil,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
```

Source code :

```
const { parse } = require("csv-parse/sync");
const { createObjectCsvWriter } = require("csv-writer");
const fetch = require("node-fetch");

function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`);
}

async function fetchCSVData(url) {
  try {
    const response = await fetch(url);
    const csvText = await response.text();
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    });
  } catch (error) {
```

```
    throw new Error(`Failed to fetch CSV data: ${error.message}`);
  }
}
```

```
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }));

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  });

  await csvWriter.writeRecords(data);
  console.log(`Output written to ${outputPath}`);
}
```

```
function profileDimDosenWali(data) {
  logStep("PROFILING", "Starting data profiling for Dim_Dosen_Wali");
```

```
  const metrics = {
    totalRecords: data.length,
    nullValues: {},
    uniqueValues: {},
    dataTypes: {},
  };

```

```
  const columns = Object.keys(data[0] || {});
  columns.forEach((col) => {
    metrics.nullValues[col] = 0;
    metrics.uniqueValues[col] = new Set();
  });

```

```
  data.forEach((record) => {
    columns.forEach((col) => {
      const value = record[col];
```

```
      if (value === null || value === undefined || value === "") {
        metrics.nullValues[col]++;
      } else {
        metrics.uniqueValues[col].add(value);
```

```
        if (col === "NIP_Dosen" && !isNaN(value)) {
          metrics.dataTypes[col] = "numeric";
        } else {
```



```

        metrics.dataTypes[col] = "string";
    }
}
});
});

Object.keys(metrics.uniqueValues).forEach((col) => {
    metrics.uniqueValues[col] = metrics.uniqueValues[col].size;
});

console.log("Dim_Dosen_Wali Profiling Results:");
console.log("Total Records:", metrics.totalRecords);
console.log("Null Values:", metrics.nullValues);
console.log("Unique Values:", metrics.uniqueValues);
console.log("Data Types:", metrics.dataTypes);

return metrics;
}

function transformDimDosenWali(data) {
    logStep("TRANSFORMATION", "Starting transformation for Dim_Dosen_Wali");

    return data.map((record, index) => {
        const transformed = {};

        transformed.Dosen_SK = index + 1;

        transformed.NIP_Dosen = record.NIP_Dosen ? Number.parseInt(record.NIP_Dosen, 10) :
        null;

        transformed>Nama_Dosen = record>Nama_Dosen ? record>Nama_Dosen.trim() : "";

        transformed.Fakultas = record.Fakultas ? record.Fakultas.trim() : "";

        transformed.Departemen = record.Departemen ? record.Departemen.trim() : "";

        transformed.Program_Studi = record.Program_Studi ? record.Program_Studi.trim() : "";

        if (transformed>Nama_Dosen) {
            const titles = ["Dr.", "Prof.", "Ir.", "S.T.", "M.T.", "M.Sc.", "Ph.D."];
            let hasTitle = false;

            for (const title of titles) {
                if (transformed>Nama_Dosen.includes(title)) {

```

```

        hasTitle = true;
        break;
    }
}

transformed.Has_Academic_Title = hasTitle ? "Y" : "N";

let cleanName = transformed>Nama_Dosen;
titles.forEach((title) => {
    cleanName = cleanName.replace(new RegExp(title, "g"), "").trim();
});
transformed.Display_Name = cleanName.replace(/\s+/g, " ").trim();
} else {
    transformed.Has_Academic_Title = "N";
    transformed.Display_Name = "";
}

if (transformed.Fakultas && transformed.Departemen) {
    transformed.Faculty_Department = `${transformed.Fakultas} - ${transformed.Departemen}`;
} else {
    transformed.Faculty_Department = transformed.Fakultas || transformed.Departemen || "";
}

transformed.Effective_Date = new Date().toISOString().split("T")[0];
transformed.End_Date = "9999-12-31";
transformed.Current_Flag = "Y";
transformed.Version_Number = 1;

transformed.Created_Date = new Date().toISOString().split("T")[0];
transformed.Updated_Date = new Date().toISOString().split("T")[0];
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
"").split("T")[0]}_001`;

    return transformed;
});
}

function loadDimDosenWali(transformedData) {
    logStep("LOADING", "Starting loading for Dim_Dosen_Wali");

    const loadResults = {
        inserted: 0,
        errors: 0,
        warnings: 0,
    };

```

```

    errorRecords: [],
    warningRecords: [],
};

transformedData.forEach((record, index) => {
    const issues = [];

    if (!record.NIP_Dosen) issues.push("Missing NIP_Dosen");
    if (!record>Nama_Dosen) issues.push("Missing Nama_Dosen");

    if (record.NIP_Dosen && record.NIP_Dosen.toString().length < 8) {
        issues.push("NIP_Dosen too short (should be at least 8 digits)");
    }

    if (!record.Fakultas && !record.Departemen) {
        issues.push("Missing both Fakultas and Departemen");
    }

    if (issues.length > 0) {
        if (issues.some((issue) => issue.includes("Missing NIP_Dosen") || issue.includes("Missing
Nama_Dosen"))) {
            loadResults.errors++;
            loadResults.errorRecords.push({
                index: index,
                record: record,
                error: issues.join(", "),
            });
        } else {
            loadResults.warnings++;
            loadResults.warningRecords.push({
                index: index,
                record: record,
                warning: issues.join(", "),
            });
            loadResults.inserted++;
        }
    } else {
        loadResults.inserted++;
    }
});

logStep(
    "LOADING",

```

```

    `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
    ${loadResults.warnings} warnings`
  );

  if (loadResults.errors > 0) {
    console.log("Error Records:", loadResults.errorRecords);
  }

  if (loadResults.warnings > 0) {
    console.log("Warning Records (first 3):", loadResults.warningRecords.slice(0, 3));
  }

  return loadResults;
}

async function etlDimDosenWali() {
  try {
    logStep("START", "Starting ETL process for Dim_Dosen_Wali");

    logStep("EXTRACTION", "Fetching data from Dim_Dosen_Wali CSV");
    const rawData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Dosen_Wali%20%283%29-Kf57TxnnrnMot6hQbfzCPE2B1siPGB.csv"
    );
    logStep("EXTRACTION", `Extracted ${rawData.length} records`);

    const profilingResults = profileDimDosenWali(rawData);

    const transformedData = transformDimDosenWali(rawData);
    logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`);

    const loadResults = loadDimDosenWali(transformedData);

    logStep("EXPORT", "Exporting transformed data to CSV");
    await exportToCSV(transformedData, "Dim_Dosen_Wali_Output.csv");

    logStep("COMPLETE", "ETL process for Dim_Dosen_Wali completed successfully");
  } catch (error) {
    console.error("ETL process failed:", error.message);
  }
}

etlDimDosenWali();

```

Dimensi Kelas_MK

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[Running] node "c:\Users\micle\Desktop\DLH\Dim_Kelas_MK_historical.js"
[2025-05-27T20:39:17.964Z] [START] Starting ETL process for Dim_Kelas_MK
[2025-05-27T20:39:18.908Z] [EXTRACTION] Extracted 100 records
[2025-05-27T20:39:18.909Z] [PROFILING] Starting data profiling for Dim_Kelas_MK
Dim_Kelas_MK Profiling Results:
Total Records: 100
Null Values: {
  ID_kelas_MK: 0,
  Kode_mk: 0,
  Kelas: 0,
  'Dosen pengampu 1': 0,
  'Dosen pengampu 2': 0,
  'Dosen pengampu 3': 0,
  Nama_MK: 0,
  SKS: 0,
  Semester_mk: 0,
  Ruangan: 0,
  Hari: 0,
  Jam_mulai: 0,
  Jam_selesai: 0,
}

DLH > Dim_Kelas_MK_Output.csv
1  Kelas_MK_SK,ID_kelas_MK,Kode_mk,Kelas,Dosen_pengampu_1,Dosen_pengampu_2,Dosen_pengampu_3>Nama_MK,SKS,Semester_mk,Ruangan,Hari,Jam_mulai,Jam_selesai,Kapasitas,Effective
2  1,1,MK529,B,77827638,28728463,13561597,Organized zero administration ability,3,Genap,R.514,Senin,14:56,20:17,30,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_2
3  2,2,MK259,B,13999315,89254563,83197857,Multi-channelled incremental functionalities,3,Ganjil,R.147,Kamis,13:59,21:39,26,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,
4  3,3,MK926,A,66629388,13561597,83197857,Exclusive zero tolerance hub,4,Genap,R.374,Senin,12:56,11:50,44,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_0
5  4,4,MK949,C,13356886,77827638,21668732,Focused background infrastructure,4,Ganjil,R.460,Senin,05:59,21:17,22,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_2025
6  5,5,MK975,B,23756669,21668732,13356886,Programmable hybrid info-mediaries,2,Genap,R.242,Kamis,12:47,03:56,60,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_2025
7  6,6,MK463,B,66629388,39958838,13561597,Future-proofed interactive moderator,4,Genap,R.459,Senin,00:57,04:15,58,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20
8  7,7,MK267,D,39958838,77827638,23756669,Reduced tangible architecture,3,Genap,R.573,Jumat,01:50,17:40,34,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_
9  8,8,MK941,A,89254563,24942603,23756669,Ameliorated asynchronous projection,3,Genap,R.237,Senin,01:03,06:12,33,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_202
10 9,9,MK771,D,90801586,89254563,28728463,Balanced analyzing throughput,3,Genap,R.173,Rabu,15:49,08:04,28,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_0
11 10,10,MK369,D,23756669,77827638,90801586,Customer-focused demand-driven time-frame,4,Genap,R.285,Selasa,18:52,20:58,28,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,
12 11,11,MK873,A,41227216,39345092,13356886,Open-architected human-resource utilization,2,Ganjil,R.421,Selasa,04:34,15:57,47,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-
13 12,12,MK490,D,13561597,13356886,14265799,Intuitive directional infrastructure,4,Genap,R.597,Jumat,12:22,09:30,20,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_
14 13,13,MK887,C,46913810,77827638,83197857,Upgradable foreground model,2,Genap,R.322,Selasa,07:47,12:30,49,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527
15 14,14,MK880,B,95822412,83197857,41227216,Adaptive executive conglomeration,4,Ganjil,R.545,Rabu,03:57,17:01,60,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_202
16 15,15,MK482,B,41227216,28728463,42868828,Organized asymmetric complexity,4,Ganjil,R.406,Rabu,06:12,00:27,51,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250
17 16,16,MK999,C,95822412,46913810,66629388,Visionary responsive infrastructure,2,Ganjil,R.223,Jumat,11:51,12:34,25,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_
18 17,17,MK878,B,13356886,39345092,13561597,Horizontal maximized initiative,2,Genap,R.584,Jumat,18:14,14:14,30,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250
19 18,18,MK316,B,83197857,41227216,13999315,Progressive content-based methodology,4,Genap,R.304,Rabu,05:04,21:58,48,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_
20 19,19,MK353,B,41227216,22575562,46913810,Business-focused non-volatile strategy,2,Genap,R.110,Jumat,14:35,16:02,55,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_
21 20,20,MK107,A,23756669,90801586,13561597,Monitored actuating capability,4,Ganjil,R.217,Senin,05:01,22:53,22,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250
```

Source code :

```
const { parse } = require("csv-parse/sync");
const { createObjectCsvWriter } = require("csv-writer");
const fetch = require("node-fetch");
```

```
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`);
}
```

```
async function fetchCSVData(url) {
  try {
    const response = await fetch(url);
```

```

const csvText = await response.text();
return parse(csvText, {
  columns: true,
  skip_empty_lines: true,
  trim: true,
});
} catch (error) {
  throw new Error(`Failed to fetch CSV data: ${error.message}`);
}
}

async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }));

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  });

  await csvWriter.writeRecords(data);
  console.log(`Output written to ${outputPath}`);
}

function profileDimKelasMK(data) {
  logStep("PROFILING", "Starting data profiling for Dim_Kelas_MK");

  const metrics = {
    totalRecords: data.length,
    nullValues: {},
    uniqueValues: {},
    numericStats: {},
    dataTypes: {},
  };

  const columns = Object.keys(data[0] || {});
  columns.forEach((col) => {
    metrics.nullValues[col] = 0;
    metrics.uniqueValues[col] = new Set();
    if (col === "SKS" || col === "Kapasitas") {
      metrics.numericStats[col] = { min: null, max: null, sum: 0, count: 0 };
    }
  });

  data.forEach((record) => {

```

```

columns.forEach((col) => {
  const value = record[col];

  if (value === null || value === undefined || value === "") {
    metrics.nullValues[col]++;
  } else {
    metrics.uniqueValues[col].add(value);

    if (col === "SKS" || col === "Kapasitas") {
      const numValue = Number.parseFloat(value);
      if (!isNaN(numValue)) {
        metrics.dataTypes[col] = "numeric";
        const stats = metrics.numericStats[col];
        if (stats.min === null || numValue < stats.min) stats.min = numValue;
        if (stats.max === null || numValue > stats.max) stats.max = numValue;
        stats.sum += numValue;
        stats.count++;
      }
    } else if (col.includes("Jam") && value.match(/^\d{2}:\d{2}$/)) {
      metrics.dataTypes[col] = "time";
    } else {
      metrics.dataTypes[col] = "string";
    }
  }
});
});

```

```

Object.keys(metrics.numericStats).forEach((col) => {
  const stats = metrics.numericStats[col];
  if (stats.count > 0) {
    stats.average = stats.sum / stats.count;
  }
});

```

```

Object.keys(metrics.uniqueValues).forEach((col) => {
  metrics.uniqueValues[col] = metrics.uniqueValues[col].size;
});

```

```

console.log("Dim_Kelas_MK Profiling Results:");
console.log("Total Records:", metrics.totalRecords);
console.log("Null Values:", metrics.nullValues);
console.log("Unique Values:", metrics.uniqueValues);
console.log("Numeric Statistics:", metrics.numericStats);
console.log("Data Types:", metrics.dataTypes);

```

```

return metrics;
}

function transformDimKelasMK(data) {
  logStep("TRANSFORMATION", "Starting transformation for Dim_Kelas_MK");

  return data.map((record, index) => {
    const transformed = {};

    transformed.Kelas_MK_SK = index + 1;
    transformed.ID_kelas_MK = record.ID_kelas_MK;
    transformed.Kode_mk = record.Kode_mk ? record.Kode_mk.trim() : "";
    transformed.Kelas = record.Kelas ? record.Kelas.trim() : "";
    transformed.Dosen_pengampu_1 = record["Dosen pengampu 1"] ?
    Number.parseInt(record["Dosen pengampu 1"], 10) : null;
    transformed.Dosen_pengampu_2 = record["Dosen pengampu 2"] ?
    Number.parseInt(record["Dosen pengampu 2"], 10) : null;
    transformed.Dosen_pengampu_3 = record["Dosen pengampu 3"] ?
    Number.parseInt(record["Dosen pengampu 3"], 10) : null;
    transformed>Nama_MK = record>Nama_MK ? record>Nama_MK.trim() : "";
    transformed.SKS = record.SKS ? Number.parseInt(record.SKS, 10) : null;

    if (record.Semester_mk) {
      const semester = record.Semester_mk.toLowerCase().trim();
      if (semester.includes("ganjil")) {
        transformed.Semester_mk = "Ganjil";
      } else if (semester.includes("genap")) {
        transformed.Semester_mk = "Genap";
      } else {
        transformed.Semester_mk = record.Semester_mk.trim();
      }
    } else {
      transformed.Semester_mk = "";
    }

    transformed.Ruangan = record.Ruangan ? record.Ruangan.trim() : "";
    transformed.Hari = record.Hari ? record.Hari.charAt(0).toUpperCase() +
    record.Hari.slice(1).toLowerCase() : "";
    transformed.Jam_mulai = record.Jam_mulai && record.Jam_mulai.match(/^d{2}:d{2}$/) ?
    record.Jam_mulai : null;
    transformed.Jam_selesai = record.Jam_selesai && record.Jam_selesai.match(/^d{2}:d{2}$/)
    ? record.Jam_selesai : null;
    transformed.Kapasitas = record.Kapasitas ? Number.parseInt(record.Kapasitas, 10) : null;
  });
}

```



```

transformed.Effective_Date = new Date().toISOString().split("T")[0];
transformed.End_Date = "9999-12-31";
transformed.Current_Flag = "Y";
transformed.Version_Number = 1;
transformed.Created_Date = new Date().toISOString().split("T")[0];
transformed.Updated_Date = new Date().toISOString().split("T")[0];
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-/]/g,
"").split("T")[0]}_001`;

    return transformed;
  });
}

```

```

function loadDimKelasMK(transformedData) {
  logStep("LOADING", "Starting loading for Dim_Kelas_MK");

  const loadResults = {
    inserted: 0,
    errors: 0,
    warnings: 0,
    errorRecords: [],
    warningRecords: [],
  };

  transformedData.forEach((record, index) => {
    const issues = [];

    if (!record.ID_kelas_MK) issues.push("Missing ID_kelas_MK");
    if (!record.Kode_mk) issues.push("Missing Kode_mk");
    if (!record>Nama_MK) issues.push("Missing Nama_MK");

    if (record.SKS && (record.SKS < 1 || record.SKS > 6)) {
      issues.push("Invalid SKS range (should be 1-6)");
    }

    if (record.Kapasitas && (record.Kapasitas < 1 || record.Kapasitas > 200)) {
      issues.push("Invalid Kapasitas range");
    }

    if (record.Jam_mulai && record.Jam_selesai) {
      const [startHour, startMin] = record.Jam_mulai.split(":").map(Number);
      const [endHour, endMin] = record.Jam_selesai.split(":").map(Number);
      const startTime = startHour * 60 + startMin;

```

```

const endTime = endHour * 60 + endMin;

if (startTime >= endTime) {
  issues.push("Invalid time range (start time >= end time)");
}
}

if (issues.length > 0) {
  if (issues.some((issue) => issue.includes("Missing"))) {
    loadResults.errors++;
    loadResults.errorRecords.push({
      index: index,
      record: record,
      error: issues.join(", "),
    });
  } else {
    loadResults.warnings++;
    loadResults.warningRecords.push({
      index: index,
      record: record,
      warning: issues.join(", "),
    });
    loadResults.inserted++;
  }
} else {
  loadResults.inserted++;
}
});

logStep(
  "LOADING",
  `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
  ${loadResults.warnings} warnings`
);

if (loadResults.errors > 0) {
  console.log("Error Records (first 3):", loadResults.errorRecords.slice(0, 3));
}

if (loadResults.warnings > 0) {
  console.log("Warning Records (first 3):", loadResults.warningRecords.slice(0, 3));
}

return loadResults;

```

```

}

async function etlDimKelasMK() {
  try {
    logStep("START", "Starting ETL process for Dim_Kelas_MK");

    const rawData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Kelas_MK%20%283%29-arEvtzQeQANqCuLfqX4BjP1cmM5vMm.csv"
    );
    logStep("EXTRACTION", `Extracted ${rawData.length} records`);

    const profilingResults = profileDimKelasMK(rawData);

    const transformedData = transformDimKelasMK(rawData);
    logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`);

    const loadResults = loadDimKelasMK(transformedData);

    logStep("EXPORT", "Exporting transformed data to CSV");
    await exportToCSV(transformedData, "Dim_Kelas_MK_Output.csv");

    logStep("COMPLETE", "ETL process for Dim_Kelas_MK completed successfully");

    return {
      status: "success",
      extracted: rawData.length,
      transformed: transformedData.length,
      loaded: loadResults.inserted,
      errors: loadResults.errors,
      warnings: loadResults.warnings,
      transformedData: transformedData.slice(0, 3),
    };
  } catch (error) {
    logStep("ERROR", `ETL process failed: ${error.message}`);
    console.error(error);
    return {
      status: "error",
      message: error.message,
    };
  }
}

etlDimKelasMK().then((result) => {

```

```
console.log("\nETL Process Result:", result);
});
```

Dimensi Mahasiswa

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] node "c:\Users\micle\Desktop\DLH\Dim_mahasiswa_historical.js"
[2025-05-27T20:45:29.667Z] [START] Starting ETL process for Dim_Mahasiswa
[2025-05-27T20:45:30.674Z] [EXTRACTION] Extracted 100 records
[2025-05-27T20:45:30.674Z] [PROFILING] Starting data profiling for Dim_Mahasiswa
Dim_Mahasiswa Profiling Results:
Total Records: 100
Null Values: {
  NRP: 0,
  Nama: 0,
  IPK_terakhir: 0,
  Program_Studi: 0,
  Departemen: 0,
  'Jenis kelamin': 0,
  Fakultas: 0,
  Angkatan: 0,
  Semester_saat_ini: 0,
  Ips_terakhir: 0,
  Jenjang: 0,
  'Tanggal masuk': 0
}
Unique Values: {
  NRP: 100,

```

DLH > Dim_Mahasiswa_Output.csv

	Mahasiswa_SK	NRP	Nama	IPK_terakhir	Program_Studi	Departemen	Jenis_kelamin	Fakultas	Angkatan	Semester_saat_ini	Ips_terakhir	Jenjang	Tanggal_masuk	NIP_Dosen	Effective_Da
1	1000001	Dr.	Ratih Melani	2.75	S1 Sistem Informasi	Informatika	Perempuan	FTK	2022	8	2.75	S3	2023-01-08	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
2	1000002	Rizki Siregar	3.9	S1 Teknik Mesin	Informatika	Laki-laki	FTEIC	2018	2	3.5	S1	2024-04-02	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
3	1000003	Adiarja Kuswandari	3.46	S1 Teknik Elektro	Sipil	Laki-laki	FTI	2020	4	2.79	S3	2023-07-24	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
4	1000004	Hj. Kani Sihombing	3.2	S1 Teknik Mesin	Sipil	Laki-laki	FTSPK	2018	6	3.66	S3	2022-07-29	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
5	1000005	H. Rekza Pranowo	2.31	S1 Teknik Elektro	Mesin	Perempuan	FTSPK	2020	5	3.14	S3	2020-11-25	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
6	1000006	Oman Jailani	2.31	S1 Teknik Elektro	Elektro	Perempuan	FTI	2021	7	2.13	S2	2021-11-29	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
7	1000007	"H. Gaduh Puspita, S.H."	2.12	S1 Teknik Sipil	Sipil	Laki-laki	FTSPK	2019	2	2.07	S2	2024-01-31	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
8	1000008	"Wani Tamba, S.E."	3.73	S1 Teknik Elektro	Informatika	Laki-laki	FTI	2018	8	2.27	S2	2020-08-15	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
9	1000009	"Dr. Fitria Firmansyah, S.E."	3.2	S1 Teknik Mesin	Elektro	Perempuan	FTEIC	2022	5	2.03	S3	2020-09-27	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
10	1000010	"Kezia Putra, S.Psi"	3.42	S1 Teknik Elektro	Elektro	Laki-laki	FTEIC	2021	2	2.15	S2	2020-10-18	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
11	1000011	Diah Puspita	2.04	S1 Teknik Sipil	Informatika	Perempuan	FTI	2018	2	3.38	S2	2020-08-10	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
12	1000012	Bakiman Handayani	3.94	S1 Teknik Sipil	Sipil	Laki-laki	FTEIC	2022	2	3.07	S1	2022-12-11	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
13	1000013	drg. Salwa Pratama	3.66	S1 Teknik Mesin	Informatika	Perempuan	FTEIC	2018	7	3.5	S2	2023-06-03	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
14	1000014	Ami Maulana	2.42	S1 Informatika	Elektro	Laki-laki	FTEIC	2020	6	3.83	S2	2022-02-10	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
15	1000015	Amelia Rajata	2.36	S1 Informatika	Kelautan	Perempuan	FTM	2018	7	3.17	S3	2020-03-04	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
16	1000016	Raharja Budiyanto	2.37	S1 Teknik Elektro	Kelautan	Laki-laki	FTK	2018	3	3.45	S3	2023-06-08	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
17	1000017	Diana Firmansyah	2.61	S1 Sistem Informasi	Mesin	Perempuan	FTSPK	2018	5	3.51	S3	2021-10-21	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
18	1000018	R.A. Shania Widiastuti	3.05	S1 Teknik Elektro	Informatika	Perempuan	FTM	2021	6	2.76	S3	2019-11-29	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
19	1000019	Lianan Anggraini	2.86	S1 Teknik Mesin	Informatika	Laki-laki	FTEIC	2018	6	2.48	S3	2020-12-11	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	
20	1000020	"Umi Kuswoyo, S.T."	2.58	S1 Teknik Mesin	Mesin	Laki-laki	FTI	2022	6	2.41	S3	2021-04-14	,2025-05-27,9999-12-31	Y,1,2025-05-27,2025-05-27,BATCH_20250527_001	

Source Code :

```
const { parse } = require("csv-parse/sync");
const { createObjectCsvWriter } = require("csv-writer");
const { format } = require("date-fns");
const fetch = require("node-fetch");

function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`);
}
```

```
}
```

```
async function fetchCSVData(url) {  
  try {  
    const response = await fetch(url);  
    const csvText = await response.text();  
    return parse(csvText, {  
      columns: true,  
      skip_empty_lines: true,  
      trim: true,  
    });  
  } catch (error) {  
    throw new Error(`Failed to fetch CSV data: ${error.message}`);  
  }  
}
```

```
async function exportToCSV(data, outputPath) {  
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }));  
  
  const csvWriter = createObjectCsvWriter({  
    path: outputPath,  
    header: headers,  
  });  
  
  await csvWriter.writeRecords(data);  
  console.log(`Output written to ${outputPath}`);  
}
```

```
function profileDimMahasiswa(data) {  
  logStep("PROFILING", "Starting data profiling for Dim_Mahasiswa");  
  
  const metrics = {  
    totalRecords: data.length,  
    nullValues: {},  
    uniqueValues: {},  
    numericStats: {},  
    dataTypes: {},  
  };  
  
  const columns = Object.keys(data[0] || {});  
  columns.forEach((col) => {  
    metrics.nullValues[col] = 0;  
    metrics.uniqueValues[col] = new Set();  
    if (["IPK_terakhir", "Ips_terakhir", "Angkatan", "Semester_saat_ini"].includes(col)) {
```

```

    metrics.numericStats[col] = { min: null, max: null, sum: 0, count: 0 };
  }
});

```

```

data.forEach((record) => {
  columns.forEach((col) => {
    const value = record[col];
    if (value === null || value === undefined || value === "") {
      metrics.nullValues[col]++;
    } else {
      metrics.uniqueValues[col].add(value);

      if (["IPK_terakhir", "Ips_terakhir", "Angkatan", "Semester_saat_ini"].includes(col)) {
        const numValue = parseFloat(value);
        if (!isNaN(numValue)) {
          metrics.dataTypes[col] = "numeric";
          const stats = metrics.numericStats[col];
          if (stats.min === null || numValue < stats.min) stats.min = numValue;
          if (stats.max === null || numValue > stats.max) stats.max = numValue;
          stats.sum += numValue;
          stats.count++;
        }
      } else if (col === "Tanggal masuk" && value.match(/^\d{4}-\d{2}-\d{2}$/)) {
        metrics.dataTypes[col] = "date";
      } else {
        metrics.dataTypes[col] = "string";
      }
    }
  });
});

```

```

Object.keys(metrics.numericStats).forEach((col) => {
  const stats = metrics.numericStats[col];
  if (stats.count > 0) {
    stats.average = stats.sum / stats.count;
  }
});

```

```

Object.keys(metrics.uniqueValues).forEach((col) => {
  metrics.uniqueValues[col] = metrics.uniqueValues[col].size;
});

```

```

console.log("Dim_Mahasiswa Profiling Results:");
console.log("Total Records:", metrics.totalRecords);

```

```

console.log("Null Values:", metrics.nullValues);
console.log("Unique Values:", metrics.uniqueValues);
console.log("Numeric Statistics:", metrics.numericStats);
console.log("Data Types:", metrics.dataTypes);

return metrics;
}

function transformDimMahasiswa(data) {
  logStep("TRANSFORMATION", "Starting transformation for Dim_Mahasiswa");

  return data.map((record, index) => {
    const transformed = {};
    transformed.Mahasiswa_SK = index + 1;
    transformed.NRP = parseInt(record.NRP, 10);
    transformed>Nama = record>Nama ? record>Nama.trim() : "";
    transformed.IPK_terakhir = record.IPK_terakhir ?
parseFloat(parseFloat(record.IPK_terakhir).toFixed(2)) : null;
    transformed.Program_Studi = record.Program_Studi ? record.Program_Studi.trim() : "";
    transformed.Departemen = record.Departemen ? record.Departemen.trim() : "";

    if (record["Jenis kelamin"]) {
      const gender = record["Jenis kelamin"].toLowerCase().trim();
      transformed.Jenis_kelamin = gender.includes("laki") ? "Laki-laki" :
(gender.includes("perempuan") || gender.includes("wanita")) ?
"Perempuan" :
record["Jenis kelamin"];
    } else {
      transformed.Jenis_kelamin = "";
    }

    transformed.Fakultas = record.Fakultas ? record.Fakultas.trim() : "";
    transformed.Angkatan = record.Angkatan ? parseInt(record.Angkatan, 10) : null;

    const semester = parseInt(record.Semester_saat_ini, 10);
    transformed.Semester_saat_ini = semester >= 1 && semester <= 14 ? semester : null;

    transformed.lps_terakhir = record.lps_terakhir ?
parseFloat(parseFloat(record.lps_terakhir).toFixed(2)) : null;

    if (record.Jenjang) {
      const jenjang = record.Jenjang.toUpperCase().trim();
      transformed.Jenjang = ["S1", "S2", "S3"].includes(jenjang) ? jenjang : record.Jenjang;
    } else {

```

```

    transformed.Jenjang = "";
}

if (record["Tanggal masuk"]) {
    try {
        const [year, month, day] = record["Tanggal masuk"].split("-").map(Number);
        transformed.Tanggal_masuk = format(new Date(year, month - 1, day), "yyyy-MM-dd");
    } catch {
        transformed.Tanggal_masuk = null;
    }
} else {
    transformed.Tanggal_masuk = null;
}

transformed.NIP_Dosen = record.NIP_Dosen ? parseInt(record.NIP_Dosen, 10) : null;

transformed.Effective_Date = new Date().toISOString().split("T")[0];
transformed.End_Date = "9999-12-31";
transformed.Current_Flag = "Y";
transformed.Version_Number = 1;
transformed.Created_Date = new Date().toISOString().split("T")[0];
transformed.Updated_Date = new Date().toISOString().split("T")[0];
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
"").split("T")[0]}_001`;

    return transformed;
});
}

function loadDimMahasiswa(transformedData) {
    logStep("LOADING", "Starting loading for Dim_Mahasiswa");

    const loadResults = {
        inserted: 0,
        errors: 0,
        warnings: 0,
        errorRecords: [],
        warningRecords: [],
    };

    transformedData.forEach((record, index) => {
        const issues = [];

        if (!record.NRP) issues.push("Missing NRP");
    });
}

```



```

    if (!record>Nama) issues.push("Missing Nama");

    if (record.IPK_terakhir && (record.IPK_terakhir < 0 || record.IPK_terakhir > 4)) {
        issues.push("Invalid IPK range");
    }
    if (record.Ips_terakhir && (record.Ips_terakhir < 0 || record.Ips_terakhir > 4)) {
        issues.push("Invalid IPS range");
    }
    if (record.Angkatan && (record.Angkatan < 1900 || record.Angkatan > new
Date().getFullYear())) {
        issues.push("Invalid Angkatan year");
    }

    if (issues.length > 0) {
        if (issues.some((issue) => issue.includes("Missing"))) {
            loadResults.errors++;
            loadResults.errorRecords.push({ index, record, error: issues.join(", ") });
        } else {
            loadResults.warnings++;
            loadResults.warningRecords.push({ index, record, warning: issues.join(", ") });
            loadResults.inserted++;
        }
    } else {
        loadResults.inserted++;
    }
});

logStep(
    "LOADING",
    `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
    ${loadResults.warnings} warnings`
);

if (loadResults.errors > 0) {
    console.log("Error Records (first 3):", loadResults.errorRecords.slice(0, 3));
}

if (loadResults.warnings > 0) {
    console.log("Warning Records (first 3):", loadResults.warningRecords.slice(0, 3));
}

return loadResults;
}

```

```

async function etlDimMahasiswa() {
  try {
    logStep("START", "Starting ETL process for Dim_Mahasiswa");

    const rawData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Mahasiswa-ODzn65lS2Wxhzkfevt19w7rnpbq1Cy.csv"
    );
    logStep("EXTRACTION", `Extracted ${rawData.length} records`);

    const profilingResults = profileDimMahasiswa(rawData);

    const transformedData = transformDimMahasiswa(rawData);
    logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`);

    const loadResults = loadDimMahasiswa(transformedData);

    await exportToCSV(transformedData, "Dim_Mahasiswa_Output.csv");

    logStep("COMPLETE", "ETL process for Dim_Mahasiswa completed successfully");

    return {
      status: "success",
      extracted: rawData.length,
      transformed: transformedData.length,
      loaded: loadResults.inserted,
      errors: loadResults.errors,
      warnings: loadResults.warnings,
      transformedData: transformedData.slice(0, 3),
    };
  } catch (error) {
    logStep("ERROR", `ETL process failed: ${error.message}`);
    console.error(error);
    return {
      status: "error",
      message: error.message,
    };
  }
}

etlDimMahasiswa().then((result) => {
  console.log("\nETL Process Result:", result);
});

```

Dimensi Metode_pembayaran

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Filter
[Running] node "c:\Users\micle\Desktop\DLH\Dim_Metode_pembayaran.js"
[2025-05-27T20:57:45.118Z] [START] Starting ETL process for Dim_Metode_Pembayaran
[2025-05-27T20:57:46.271Z] [EXTRACTION] Extracted 100 records
[2025-05-27T20:57:46.271Z] [PROFILING] Starting data profiling for Dim_Metode_Pembayaran
Dim_Metode_Pembayaran Profiling Results:
Total Records: 100
Null Values: { ID_Metode: 0, Nama_Metode: 0, Bank: 0 }
Unique Values: { ID_Metode: 100, Nama_Metode: 5, Bank: 7 }
Data Types: { ID_Metode: 'numeric', Nama_Metode: 'string', Bank: 'string' }
[2025-05-27T20:57:46.274Z] [TRANSFORMATION] Starting transformation for Dim_Metode_Pembayaran
[2025-05-27T20:57:46.275Z] [TRANSFORMATION] Transformed 100 records
[2025-05-27T20:57:46.275Z] [LOADING] Starting loading for Dim_Metode_Pembayaran
[2025-05-27T20:57:46.275Z] [LOADING] Loaded 100 records successfully, 0 errors, 38 warnings
Warning Records: [
  {
    index: 1,
    record: {
      Metode_Pembayaran_SK: 2,
      ID_Metode: '2',
      Nama_Metode: 'QRIS'
    }
  }
]

DLH > Dim_Metode_Pembayaran_Output.csv
1  Metode_Pembayaran_SK,ID_Metode,Nama_Metode,Bank,Payment_Category,Is_Digital,Requires_Bank,Full_Description,Created_Date,ETL_Batch_ID
2  1,1,E-Wallet,GOPAY,Digital Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_001
3  2,2,QRIS,BCA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_001
4  3,3,Cash,MANDIRI,Cash Payment,N,N,Cash,2025-05-27,BATCH_20250527_001
5  4,4,QRIS,BCA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_001
6  5,5,Transfer,GOPAY,Bank Transfer,Y,Y,Transfer - GOPAY,2025-05-27,BATCH_20250527_001
7  6,6,QRIS,BNI,Other,N,N,QRIS,2025-05-27,BATCH_20250527_001
8  7,7,QRIS,DANA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_001
9  8,8,Cash,MANDIRI,Cash Payment,N,N,Cash,2025-05-27,BATCH_20250527_001
10 9,9,Cash,BCA,Cash Payment,N,N,Cash,2025-05-27,BATCH_20250527_001
11 10,10,E-Wallet,GOPAY,Digital Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_001
12 11,11,Virtual Account,OVO,Other,N,N,Virtual Account,2025-05-27,BATCH_20250527_001
13 12,12,E-Wallet,BRI,Digital Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_001
14 13,13,Cash,BNI,Cash Payment,N,N,Cash,2025-05-27,BATCH_20250527_001
15 14,14,E-Wallet,MANDIRI,Digital Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_001
```

Source code :

```
const fetch = require("node-fetch");
const { parse } = require("csv-parse/sync");
const { createObjectCsvWriter } = require("csv-writer");

function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`);
}

function fetchCSVData(url) {
  return fetch(url)
    .then((response) => {
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
      return response.text();
    })
}
```

```

.then((csvText) => {
  return parse(csvText, {
    columns: true,
    skip_empty_lines: true,
    trim: true,
  });
})
.catch((error) => {
  throw new Error(`Failed to fetch CSV data: ${error.message}`);
});
}

```

```

function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }));

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  });

  return csvWriter.writeRecords(data).then(() => {
    console.log(`Output written to ${outputPath}`);
  });
}

```

```

function profileDimMetodePembayaran(data) {
  logStep("PROFILING", "Starting data profiling for Dim_Metode_Pembayaran");

  const metrics = {
    totalRecords: data.length,
    nullValues: {},
    uniqueValues: {},
    dataTypes: {},
  };

  const columns = Object.keys(data[0] || {});
  columns.forEach((col) => {
    metrics.nullValues[col] = 0;
    metrics.uniqueValues[col] = new Set();
  });

  data.forEach((record) => {
    columns.forEach((col) => {
      const value = record[col];

```

```

    if (value === null || value === undefined || value === "") {
        metrics.nullValues[col]++;
    } else {
        metrics.uniqueValues[col].add(value);

        if (!isNaN(value)) {
            metrics.dataTypes[col] = "numeric";
        } else {
            metrics.dataTypes[col] = "string";
        }
    }
});
});

```

```

Object.keys(metrics.uniqueValues).forEach((col) => {
    metrics.uniqueValues[col] = metrics.uniqueValues[col].size;
});

```

```

console.log("Dim_Metode_Pembayaran Profiling Results:");
console.log("Total Records:", metrics.totalRecords);
console.log("Null Values:", metrics.nullValues);
console.log("Unique Values:", metrics.uniqueValues);
console.log("Data Types:", metrics.dataTypes);

```

```

return metrics;
}

```

```

function transformDimMetodePembayaran(data) {
    logStep("TRANSFORMATION", "Starting transformation for Dim_Metode_Pembayaran");

```

```

    return data.map((record, index) => {
        const transformed = {};

```

```

        transformed.Metode_Pembayaran_SK = index + 1;
        transformed.ID_Metode = record.ID_Metode;

```

```

        if (record>Nama_Metode) {
            const metode = record>Nama_Metode.trim();
            const metodeMap = {
                atm: "ATM",
                "m-banking": "M-Banking",
                "mobile banking": "M-Banking",
                "internet banking": "Internet Banking",

```

```
teller: "Teller",  
"e-wallet": "E-Wallet",  
transfer: "Transfer",  
cash: "Cash",  
tunai: "Cash",  
};
```

```
const normalizedMetode = metodeMap[metode.toLowerCase()] || metode;  
transformed>Nama_Metode = normalizedMetode;  
} else {  
    transformed>Nama_Metode = "";  
}
```

```
transformed.Bank = record.Bank ? record.Bank.trim().toUpperCase() : "";
```

```
if (transformed>Nama_Metode) {  
    if (["ATM", "M-Banking", "Internet Banking", "Teller"].includes(transformed>Nama_Metode)) {  
        transformed.Payment_Category = "Bank Services";  
    } else if (["E-Wallet"].includes(transformed>Nama_Metode)) {  
        transformed.Payment_Category = "Digital Wallet";  
    } else if (["Transfer"].includes(transformed>Nama_Metode)) {  
        transformed.Payment_Category = "Bank Transfer";  
    } else if (["Cash"].includes(transformed>Nama_Metode)) {  
        transformed.Payment_Category = "Cash Payment";  
    } else {  
        transformed.Payment_Category = "Other";  
    }  
}
```

```
if (["M-Banking", "Internet Banking", "E-Wallet",  
"Transfer"].includes(transformed>Nama_Metode)) {  
    transformed.Is_Digital = "Y";  
} else {  
    transformed.Is_Digital = "N";  
}
```

```
if (["ATM", "M-Banking", "Internet Banking", "Teller",  
"Transfer"].includes(transformed>Nama_Metode)) {  
    transformed.Requires_Bank = "Y";  
} else {  
    transformed.Requires_Bank = "N";  
}  
} else {  
    transformed.Payment_Category = "Unknown";  
    transformed.Is_Digital = "N";  
}
```

```

    transformed.Requires_Bank = "N";
  }

  if (transformed.Bank && transformed.Requires_Bank === "Y") {
    transformed.Full_Description = `${transformed>Nama_Metode} - ${transformed.Bank}`;
  } else {
    transformed.Full_Description = transformed>Nama_Metode;
  }

  transformed.Created_Date = new Date().toISOString().split("T")[0];
  transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-]/g,
    "").split("T")[0]}_001`;

  return transformed;
});
}

function loadDimMetodePembayaran(transformedData) {
  logStep("LOADING", "Starting loading for Dim_Metode_Pembayaran");

  const loadResults = {
    inserted: 0,
    errors: 0,
    warnings: 0,
    errorRecords: [],
    warningRecords: [],
  };

  transformedData.forEach((record, index) => {
    const issues = [];

    if (!record.ID_Metode) issues.push("Missing ID_Metode");
    if (!record>Nama_Metode) issues.push("Missing>Nama_Metode");

    if (record.Requires_Bank === "Y" && !record.Bank) {
      issues.push("Missing Bank for payment method that requires bank");
    }

    const validMethods = ["ATM", "M-Banking", "Internet Banking", "Teller", "E-Wallet",
      "Transfer", "Cash"];
    if (record>Nama_Metode && !validMethods.includes(record>Nama_Metode)) {
      issues.push(`Unknown payment method: ${record>Nama_Metode}`);
    }
  });
}

```

```

if (issues.length > 0) {
  if (issues.some((issue) => issue.includes("Missing"))) {
    loadResults.errors++;
    loadResults.errorRecords.push({
      index: index,
      record: record,
      error: issues.join(", "),
    });
  } else {
    loadResults.warnings++;
    loadResults.warningRecords.push({
      index: index,
      record: record,
      warning: issues.join(", "),
    });
    loadResults.inserted++;
  }
} else {
  loadResults.inserted++;
}
});

logStep(
  "LOADING",
  `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
  ${loadResults.warnings} warnings`
);

if (loadResults.errors > 0) {
  console.log("Error Records:", loadResults.errorRecords);
}

if (loadResults.warnings > 0) {
  console.log("Warning Records:", loadResults.warningRecords);
}

return loadResults;
}

function etlDimMetodePembayaran() {
  logStep("START", "Starting ETL process for Dim_Metode_Pembayaran");

  return fetchCSVData(

```



```

    "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Metode_Pembayaran%20%284%29-iH400cJAF6L8u66V6KzqiMq6xFPqY7.csv"
  )
  .then((rawData) => {
    logStep("EXTRACTION", `Extracted ${rawData.length} records`);

    const profilingResults = profileDimMetodePembayaran(rawData);

    const transformedData = transformDimMetodePembayaran(rawData);
    logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`);

    const loadResults = loadDimMetodePembayaran(transformedData);

    logStep("EXPORT", "Exporting transformed data to CSV");
    return exportToCSV(transformedData, "Dim_Metode_Pembayaran_Output.csv").then(() => {
      logStep("COMPLETE", "ETL process for Dim_Metode_Pembayaran completed successfully");

      return {
        status: "success",
        extracted: rawData.length,
        transformed: transformedData.length,
        loaded: loadResults.inserted,
        errors: loadResults.errors,
        warnings: loadResults.warnings,
        transformedData: transformedData.slice(0, 5),
      };
    });
  })
  .catch((error) => {
    logStep("ERROR", `ETL process failed: ${error.message}`);
    console.error(error);
    return {
      status: "error",
      message: error.message,
    };
  });
}

etlDimMetodePembayaran().then((result) => {
  console.log("\nETL Process Result:", result);
});

```

Dimensi Status_permohonan

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Filter

[Running] node "c:\Users\micle\Desktop\DLH\Dim_status_permohonan.js"
[2025-05-27T21:02:00.102Z] [START] Starting ETL process for Dim_Status_Permohonan
[2025-05-27T21:02:00.104Z] [EXTRACTION] Fetching data from Dim_Status_Permohonan CSV
[2025-05-27T21:02:01.166Z] [EXTRACTION] Extracted 100 records
[2025-05-27T21:02:01.166Z] [PROFILING] Starting data profiling for Dim_Status_Permohonan
Dim_Status_Permohonan Profiling Results:
Total Records: 100
Null Values: { ID_Status: 0, Status: 0 }
Unique Values: { ID_Status: 100, Status: 5 }
Data Types: { ID_Status: 'numeric', Status: 'string' }
[2025-05-27T21:02:01.168Z] [TRANSFORMATION] Starting transformation for Dim_Status_Permohonan
[2025-05-27T21:02:01.169Z] [TRANSFORMATION] Transformed 100 records
[2025-05-27T21:02:01.169Z] [LOADING] Starting loading for Dim_Status_Permohonan
[2025-05-27T21:02:01.170Z] [LOADING] Loaded 100 records successfully, 0 errors, 43 warnings
Warning Records: [
  {
    index: 2,
    ...
  }
]

DLH > Dim_Status_Permohonan-Output.csv
1  Status_SK,ID_Status,Status,Status_Category,Status_Priority,Effective_Date,End_Date,Current_Flag,Version_Number,Created_Date,Updated_Date,ETL_Batch_ID
2  1,1,Ditolak,Rejected,5,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
3  2,2,Diproses,In Progress,2,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
4  3,3,Menunggu Verifikasi,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
5  4,4,Diproses,In Progress,2,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
6  5,5,Diproses,In Progress,2,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
7  6,6,Diterima,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
8  7,7,Menunggu Verifikasi,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
9  8,8,Diterima,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
10 9,9,Ditolak,Rejected,5,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
11 10,10,Ditolak,Rejected,5,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
12 11,11,Menunggu Verifikasi,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
13 12,12,Selesai,Approved,4,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
14 13,13,Diproses,In Progress,2,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
15 14,14,Diterima,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
16 15,15,Menunggu Verifikasi,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
17 16,16,Diterima,Other,99,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_001
```

```
const fetch = require('node-fetch');
const { parse } = require("csv-parse/sync");
const { createObjectCsvWriter } = require("csv-writer");

function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`);
}

function fetchCSVData(url) {
  return fetch(url)
    .then((response) => response.text())
    .then((csvText) =>
      parse(csvText, {
        columns: true,
        skip_empty_lines: true,
        trim: true,
      })
    )
}
```

```

    .catch((error) => {
      throw new Error(`Failed to fetch CSV data: ${error.message}`);
    });
  }

function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }));

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  });

  return csvWriter.writeRecords(data).then(() => {
    console.log(`Output written to ${outputPath}`);
  });
}

function profileDimStatusPermohonan(data) {
  logStep("PROFILING", "Starting data profiling for Dim_Status_Permohonan");

  const metrics = {
    totalRecords: data.length,
    nullValues: {},
    uniqueValues: {},
    dataTypes: {},
  };

  const columns = Object.keys(data[0] || {});
  columns.forEach((col) => {
    metrics.nullValues[col] = 0;
    metrics.uniqueValues[col] = new Set();
  });

  data.forEach((record) => {
    columns.forEach((col) => {
      const value = record[col];

      if (value === null || value === undefined || value === "") {
        metrics.nullValues[col]++;
      } else {
        metrics.uniqueValues[col].add(value);

        if (!isNaN(value)) {

```

```

        metrics.dataTypes[col] = "numeric";
    } else {
        metrics.dataTypes[col] = "string";
    }
}
});
});

```

```

Object.keys(metrics.uniqueValues).forEach((col) => {
    metrics.uniqueValues[col] = metrics.uniqueValues[col].size;
});

```

```

console.log("Dim_Status_Permohonan Profiling Results:");
console.log("Total Records:", metrics.totalRecords);
console.log("Null Values:", metrics.nullValues);
console.log("Unique Values:", metrics.uniqueValues);
console.log("Data Types:", metrics.dataTypes);

```

```

return metrics;
}

```

```

function transformDimStatusPermohonan(data) {
    logStep("TRANSFORMATION", "Starting transformation for Dim_Status_Permohonan");

```

```

    return data.map((record, index) => {
        const transformed = {};

```

```

        transformed.Status_SK = index + 1;
        transformed.ID_Status = record.ID_Status;

```

```

        if (record.Status) {
            const status = record.Status.trim();
            const statusMap = {
                diproses: "Diproses",
                disetujui: "Disetujui",
                ditolak: "Ditolak",
                pending: "Pending",
                selesai: "Selesai",
                dibatalkan: "Dibatalkan",
            };

```

```

            const normalizedStatus = statusMap[status.toLowerCase()] || status;
            transformed.Status = normalizedStatus;
        } else {

```

```

    transformed.Status = "";
}

if (transformed.Status) {
    if ([ "Disetujui", "Selesai" ].includes(transformed.Status)) {
        transformed.Status_Category = "Approved";
    } else if ([ "Ditolak", "Dibatalkan" ].includes(transformed.Status)) {
        transformed.Status_Category = "Rejected";
    } else if ([ "Diproses", "Pending" ].includes(transformed.Status)) {
        transformed.Status_Category = "In Progress";
    } else {
        transformed.Status_Category = "Other";
    }
}

const priorityMap = {
    Pending: 1,
    Diproses: 2,
    Disetujui: 3,
    Selesai: 4,
    Ditolak: 5,
    Dibatalkan: 6,
};
transformed.Status_Priority = priorityMap[transformed.Status] || 99;
} else {
    transformed.Status_Category = "Unknown";
    transformed.Status_Priority = 99;
}

transformed.Effective_Date = new Date().toISOString().split("T")[0];
transformed.End_Date = "9999-12-31";
transformed.Current_Flag = "Y";
transformed.Version_Number = 1;

transformed.Created_Date = new Date().toISOString().split("T")[0];
transformed.Updated_Date = new Date().toISOString().split("T")[0];
transformed.ETL_Batch_ID = `BATCH_${new Date()
    .toISOString()
    .replace(/[:-/]/g, "")
    .split("T")[0]}_001`;

return transformed;
});
}

```

```

function loadDimStatusPermohonan(transformedData) {
  logStep("LOADING", "Starting loading for Dim_Status_Permohonan");

  const loadResults = {
    inserted: 0,
    errors: 0,
    warnings: 0,
    errorRecords: [],
    warningRecords: [],
  };

  transformedData.forEach((record, index) => {
    const issues = [];

    if (!record.ID_Status) issues.push("Missing ID_Status");
    if (!record.Status) issues.push("Missing Status");

    const validStatuses = ["Diproses", "Disetujui", "Ditolak", "Pending", "Selesai", "Dibatalkan"];
    if (record.Status && !validStatuses.includes(record.Status)) {
      issues.push(`Invalid Status value: ${record.Status}`);
    }

    if (issues.length > 0) {
      if (issues.some((issue) => issue.includes("Missing"))) {
        loadResults.errors++;
        loadResults.errorRecords.push({
          index: index,
          record: record,
          error: issues.join(", "),
        });
      } else {
        loadResults.warnings++;
        loadResults.warningRecords.push({
          index: index,
          record: record,
          warning: issues.join(", "),
        });
        loadResults.inserted++;
      }
    } else {
      loadResults.inserted++;
    }
  });
}

```

```

logStep(
  "LOADING",
  `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
  ${loadResults.warnings} warnings`
);

if (loadResults.errors > 0) {
  console.log("Error Records:", loadResults.errorRecords);
}

if (loadResults.warnings > 0) {
  console.log("Warning Records:", loadResults.warningRecords);
}

return loadResults;
}

function etlDimStatusPermohonan() {
  logStep("START", "Starting ETL process for Dim_Status_Permohonan");

  logStep("EXTRACTION", "Fetching data from Dim_Status_Permohonan CSV");
  return fetchCSVData(
    "https://hebbkx1anhila5yf.public.blob.vercel-
storage.com/Dim_Status_Permohonan%20%282%29-
0qt4qqLLrWGLvSlqj8TOegy2QdTK3z.csv"
  )
  .then((rawData) => {
    logStep("EXTRACTION", `Extracted ${rawData.length} records`);

    const profilingResults = profileDimStatusPermohonan(rawData);

    const transformedData = transformDimStatusPermohonan(rawData);
    logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`);

    const loadResults = loadDimStatusPermohonan(transformedData);

    logStep("EXPORT", "Exporting transformed data to CSV");
    return exportToCSV(transformedData, "Dim_Status_Permohonan_Output.csv").then(() => ({
      rawData,
      profilingResults,
      transformedData,
      loadResults,
    }));
  })
}

```

```
.then(({ rawData, profilingResults, transformedData, loadResults }) => {  
  logStep("COMPLETE", "ETL process for Dim_Status_Permohonan completed  
successfully");
```

```
  return {  
    status: "success",  
    extracted: rawData.length,  
    transformed: transformedData.length,  
    loaded: loadResults.inserted,  
    errors: loadResults.errors,  
    warnings: loadResults.warnings,  
    transformedData: transformedData.slice(0, 5),  
  };  
})  
.catch((error) => {  
  logStep("ERROR", `ETL process failed: ${error.message}`);  
  console.error(error);  
  return {  
    status: "error",  
    message: error.message,  
  };  
});  
}
```

```
etlDimStatusPermohonan().then((result) => {  
  console.log("\nETL Process Result:", result);  
});
```


Fakta ambilFRS

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter

[Running] node "c:\Users\micle\Desktop\DLH\F_ambil_FRS_historical.js"
[2025-05-27T23:04:43.337Z] [START] Starting ETL process for F_ambilFRS
[2025-05-27T23:04:43.339Z] [EXTRACTION] Fetching data from ambilFRS CSV
[2025-05-27T23:04:44.040Z] [EXTRACTION] Extracted 100 records from fact table
[2025-05-27T23:04:44.040Z] [TRANSFORMATION] Starting transformation for F_ambilFRS
[2025-05-27T23:04:44.048Z] [TRANSFORMATION] Transformed 100 records
[2025-05-27T23:04:44.048Z] [LOADING] Starting loading for F_ambilFRS
[2025-05-27T23:04:44.048Z] [LOADING] Loaded 100 records successfully, 0 errors, 0 warnings
[2025-05-27T23:04:44.048Z] [EXPORT] Exporting transformed data to CSV
Output written to F_ambilFRS_Output.csv
[2025-05-27T23:04:44.052Z] [COMPLETE] ETL process for F_ambilFRS completed successfully

ETL Process Result: {
  status: 'success',
  extracted: 100,
  transformed: 100,
  loaded: 100,
  errors: 0,
  warnings: 0,
  sampleTransformedData: [
    {
      ambilFRS_SK: 1,
      Mahasiswa_SK: 1000004,
      Kelas_MK_SK: 19,
      Waktu_SK: 78,
      Mahasiswa_NK: 1000004,
      Kelas_MK_NK: '19',
      Waktu_NK: '78',
      SKS_diambil: 4,
      Jumlah_matakuliah_diambil: 1,
      Urutan_pengambilan_kelas: 8,
      Jumlah_peserta_di_kelas_saat_ini: 12,
    }
  ]
}

> transformFa Aa  No results

ambilFRS_SK,Mahasiswa_SK,Kelas_MK_SK,Waktu_SK,Mahasiswa_NK,Kelas_MK_NK,Waktu_NK,SKS_diambil,Jumlah_matakuliah_diambil,Urutan_pengambilan_kelas,Jumlah_peserta_di_ke
1,1000004,19,78,1000004,19,78,4,1,8,12,4,20,2025-01-23,5,5,20.00,Low,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
2,1000009,31,100,1000009,31,100,4,1,6,33,10,22,2024-09-15,17,0,18.18,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
3,1000073,5,78,1000073,5,78,2,1,8,33,14,22,2025-04-15,20,2,9.09,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
4,1000057,98,10,1000057,98,10,4,1,3,43,16,22,2024-09-18,13,3,18.18,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
5,1000051,41,84,1000051,41,84,3,1,4,17,14,20,2024-08-09,21,5,15.00,Low,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
6,1000095,24,64,1000095,24,64,4,1,7,45,6,20,2024-07-27,2,0,20.00,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
7,1000034,100,34,1000034,100,34,4,1,8,23,8,24,2024-08-04,9,0,16.67,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
8,1000037,89,63,1000037,89,63,2,1,2,18,18,20,2025-05-17,9,6,10.00,Low,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
9,1000058,23,92,1000058,23,92,3,1,2,53,19,22,2025-01-24,16,5,13.64,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
10,1000086,45,91,1000086,45,91,2,1,9,44,13,22,2024-06-04,0,3,9.09,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
11,1000039,21,92,1000039,21,92,4,1,3,60,16,22,2025-03-31,14,1,18.18,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
12,1000066,29,16,1000066,29,16,2,1,3,25,19,22,2024-08-26,9,1,9.09,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
13,1000004,47,71,1000004,47,71,4,1,6,39,12,24,2024-09-15,0,1,16.67,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
14,1000017,79,12,1000017,79,12,2,1,5,35,19,24,2024-07-14,8,0,8.33,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
15,1000093,62,68,1000093,62,68,3,1,7,46,10,20,2025-02-25,6,3,15.00,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
16,1000017,41,83,1000017,41,83,2,1,8,39,16,24,2024-08-18,4,1,8.33,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
17,1000067,45,17,1000067,45,17,4,1,10,21,0,20,2025-04-01,13,2,20.00,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
18,1000056,65,8,1000056,65,8,2,1,9,19,13,22,2024-08-03,23,6,9.09,Low,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
19,1000022,21,42,1000022,21,42,4,1,4,32,16,24,2025-05-03,13,6,16.67,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
20,1000037,11,33,1000037,11,33,2,1,9,27,11,20,2024-08-16,4,6,10.00,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
21,1000081,39,79,1000081,39,79,4,1,2,42,20,24,2024-11-15,15,5,16.67,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
22,1000022,76,75,1000022,76,75,2,1,3,52,21,24,2025-01-18,2,0,8.33,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
23,1000093,78,44,1000093,78,44,4,1,1,11,18,20,2024-12-27,11,5,20.00,Low,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
24,1000006,83,99,1000006,83,99,4,1,5,51,10,20,2025-05-01,10,4,20.00,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
25,1000099,74,54,1000099,74,54,4,1,1,41,22,24,2024-05-26,15,0,16.67,High,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
26,1000070,38,83,1000070,38,83,3,1,8,25,12,24,2025-03-18,8,2,12.50,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
27,1000052,39,59,1000052,39,59,2,1,1,20,21,22,2025-01-28,4,3,9.09,Medium,VALID,2025-05-27,BATCH_20250527_001,ambilFRS.csv
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")
```

```
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}
```

```
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}
```

```
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))
  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })
  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}
```

```
function transformFactAmbilFRS(data) {
  logStep("TRANSFORMATION", "Starting transformation for F_ambilFRS")
  return data.map((record, index) => {
    const transformed = {}
    transformed.ambilFRS_SK = index + 1
    transformed.Mahasiswa_SK = Number.parseInt(record.Mahasiswa_SK) || null
    transformed.Kelas_MK_SK = Number.parseInt(record.Kelas_MK_SK) || null
    transformed.Waktu_SK = Number.parseInt(record.Waktu_SK) || null
    transformed.Mahasiswa_NK = Number.parseInt(record.Mahasiswa_SK)
    transformed.Kelas_MK_NK = record.Kelas_MK_SK
    transformed.Waktu_NK = record.Waktu_SK
    transformed.SKS_diambil = validatePositiveInteger(record.SKS_diambil)
    transformed.Jumlah_matakuliah_diambil =
      validatePositiveInteger(record.Jumlah_matakuliah_diambil)
    transformed.Urutan_pengambilan_kelas =
      validatePositiveInteger(record.Urutan_pengambilan_kelas)
  })
}
```

```

transformed.Jumlah_peserta_di_kelas_saat_ini =
validatePositiveInteger(record.Jumlah_peserta_di_kelas_saat_ini)
transformed.Sks_tersisa_saat_ini = validateNonNegativeInteger(record.Sks_tersisa_saat_ini)
transformed.Sks_maksimum_semester_ini =
validatePositiveInteger(record.Sks_maksimum_semester_ini)

if (record.Waktu_submit) {
  try {
    const submitDate = new Date(record.Waktu_submit)
    transformed.Waktu_submit = submitDate.toISOString().split("T")[0]
    transformed.Waktu_submit_hour = submitDate.getHours()
    transformed.Waktu_submit_day_of_week = submitDate.getDay()
  } catch (e) {
    transformed.Waktu_submit = null
    transformed.Waktu_submit_hour = null
    transformed.Waktu_submit_day_of_week = null
  }
} else {
  transformed.Waktu_submit = null
  transformed.Waktu_submit_hour = null
  transformed.Waktu_submit_day_of_week = null
}

if (transformed.Sks_maksimum_semester_ini && transformed.SKS_diambil) {
  transformed.SKS_Utilization_Percentage = (
    (transformed.SKS_diambil / transformed.Sks_maksimum_semester_ini) * 100
  ).toFixed(2)
} else {
  transformed.SKS_Utilization_Percentage = null
}

if (transformed.Jumlah_peserta_di_kelas_saat_ini) {
  if (transformed.Jumlah_peserta_di_kelas_saat_ini < 20) {
    transformed.Class_Popularity = "Low"
  } else if (transformed.Jumlah_peserta_di_kelas_saat_ini < 35) {
    transformed.Class_Popularity = "Medium"
  } else {
    transformed.Class_Popularity = "High"
  }
} else {
  transformed.Class_Popularity = "Unknown"
}

transformed.Data_Quality_Flag = "VALID"

```

```

const issues = []
if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Kelas_MK_SK) issues.push("MISSING_KELAS")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")

if (issues.length > 0) {
  transformed.Data_Quality_Flag = "WARNING"
  transformed.Data_Quality_Issues = issues.join(", ")
}

transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
'').split("T")[0]}_001`
transformed.Record_Source = "ambilFRS.csv"

return transformed
})
}

function validatePositiveInteger(value) {
  const num = Number.parseInt(value)
  return num > 0 ? num : null
}

function validateNonNegativeInteger(value) {
  const num = Number.parseInt(value)
  return num >= 0 ? num : null
}

function loadFactAmbilFRS(transformedData) {
  logStep("LOADING", "Starting loading for F_ambilFRS")
  const loadResults = {
    inserted: 0,
    errors: 0,
    warnings: 0,
    errorRecords: [],
    warningRecords: [],
  }

  transformedData.forEach((record, index) => {
    if (!record.Mahasiswa_SK || !record.Kelas_MK_SK || !record.Waktu_SK) {
      loadResults.errors++
      loadResults.errorRecords.push({
        index: index,

```

```

        record: record,
        error: "Missing required dimension keys",
    })
} else if (record.Data_Quality_Flag === "WARNING") {
    loadResults.warnings++
    loadResults.warningRecords.push({
        index: index,
        record: record,
        warning: record.Data_Quality_Issues,
    })
    loadResults.inserted++
} else {
    loadResults.inserted++
}
})

logStep(
    "LOADING",
    `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
    ${loadResults.warnings} warnings`
)

return loadResults
}

async function etlFactAmbilFRS() {
    try {
        logStep("START", "Starting ETL process for F_ambilFRS")
        logStep("EXTRACTION", "Fetching data from ambilFRS CSV")
        const rawData = await fetchCSVData(
            "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/ambilFRS-
            KrhT9EsKk7QMhojwuHTURvjI9ih3s0.csv"
        )
        logStep("EXTRACTION", `Extracted ${rawData.length} records from fact table`)
        const transformedData = transformFactAmbilFRS(rawData)
        logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`)
        const loadResults = loadFactAmbilFRS(transformedData)
        logStep("EXPORT", "Exporting transformed data to CSV")
        await exportToCSV(transformedData, "F_ambilFRS_Output.csv")
        logStep("COMPLETE", "ETL process for F_ambilFRS completed successfully")
        return {
            status: "success",
            extracted: rawData.length,
            transformed: transformedData.length,
        }
    }
}

```

```

        loaded: loadResults.inserted,
        errors: loadResults.errors,
        warnings: loadResults.warnings,
        sampleTransformedData: transformedData.slice(0, 3),
    }
} catch (error) {
    logStep("ERROR", `ETL process failed: ${error.message}`)
    console.error(error)
    return {
        status: "error",
        message: error.message,
    }
}
}
}

```

```

etlFactAmbilFRS()
    .then((result) => {
        console.log("\nETL Process Result:", result)
    })
    .catch((error) => {
        console.error("ETL Process Failed:", error)
    })

```

Fakta bayarFRS

The screenshot shows the Output window of a code editor. The 'OUTPUT' tab is selected, displaying the following log messages:

```

[Running] node "c:\Users\micle\Desktop\DLH\F_bayarFRS_historical.js"
[2025-05-27T23:07:12.315Z] [START] Starting ETL process for F_Bayar_FRS
[2025-05-27T23:07:12.317Z] [EXTRACTION] Fetching data from bayarFRS CSV
[2025-05-27T23:07:13.488Z] [EXTRACTION] Extracted 100 records from fact table
[2025-05-27T23:07:13.488Z] [TRANSFORMATION] Starting transformation for F_Bayar_FRS
[2025-05-27T23:07:13.490Z] [TRANSFORMATION] Transformed 100 records
[2025-05-27T23:07:13.490Z] [LOADING] Starting loading for F_Bayar_FRS
[2025-05-27T23:07:13.490Z] [LOADING] Loaded 100 records successfully, 0 errors, 0 warnings
[2025-05-27T23:07:13.490Z] [EXPORT] Exporting transformed data to CSV
Output written to F_Bayar_FRS_Output.csv
[2025-05-27T23:07:13.494Z] [COMPLETE] ETL process for F_Bayar_FRS completed successfully

```

Below the logs, the 'ETL Process Result' is displayed as a JSON object:

```

ETL Process Result: {
  status: 'success',
  extracted: 100,
  transformed: 100,
  loaded: 100,
  errors: 0,
  warnings: 0,
  sampleTransformedData: [
    {
      BayarFRS_SK: 1,
      Mahasiswa_SK: 1000036,
      Waktu_SK: 60,
      Metode_Pembayaran_SK: 32,
      Mahasiswa_NK: 1000036,
      Waktu_NK: '60',
      Metode_Pembayaran_NK: '32',
      Nominal_bayar: 500000,
      Status_pembayaran: 'Failed',
      Is_Payment_Success: 'N',
      Payment_Category: 'Low',
      Data_Quality_Flag: 'VALID',
    }
  ]
}

```

```
DLH > F_Bayar_FRS_Output.csv
> transformFa Aa ab No results ↑ ↓ ≡ ×

1 BayarFRS_SK,Mahasiswa_SK,Waktu_SK,Metode_Pembayaran_SK,Mahasiswa_NK,Waktu_NK,Metode_Pembayaran_NK,Nominal_bayar,Status_pembayaran,Is_Payment_Success,Payment_Category,0
2 1,1000036,60,32,1000036,60,32,500000,Failed,N,Low,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
3 2,1000071,13,7,1000071,13,7,2500000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
4 3,1000012,97,31,1000012,97,31,1000000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
5 4,1000063,62,28,1000063,62,28,2000000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
6 5,1000022,49,1,1000022,49,1,2000000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
7 6,1000059,37,55,1000059,37,55,2500000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
8 7,1000020,25,38,1000020,25,38,1000000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
9 8,1000075,95,70,1000075,95,70,500000,Failed,N,Low,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
10 9,1000008,7,75,1000008,7,75,2000000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
11 10,1000008,66,11,1000008,66,11,1000000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
12 11,1000077,9,87,1000077,9,87,1000000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
13 12,1000016,73,32,1000016,73,32,2500000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
14 13,1000080,11,54,1000080,11,54,2500000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
15 14,1000034,27,86,1000034,27,86,1500000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
16 15,1000034,51,17,1000034,51,17,1500000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
17 16,1000041,97,10,1000041,97,10,500000,Failed,N,Low,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
18 17,1000080,73,13,1000080,73,13,500000,Success,Y,Low,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
19 18,1000065,34,17,1000065,34,17,1500000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
20 19,1000032,48,37,1000032,48,37,1000000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
21 20,1000070,91,39,1000070,91,39,2500000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
22 21,1000086,71,39,1000086,71,39,500000,Success,Y,Low,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
23 22,1000034,15,14,1000034,15,14,2500000,Success,Y,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
24 23,1000035,37,78,1000035,37,78,1000000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
25 24,1000027,88,82,1000027,88,82,1500000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
26 25,1000033,7,12,1000033,7,12,2000000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
27 26,1000006,1,43,1000006,1,43,1000000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
28 27,1000021,95,57,1000021,95,57,2500000,Failed,N,Medium,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
29 28,1000072,2,15,1000072,2,15,500000,Success,Y,Low,VALID,2025-05-27,BATCH_20250527_001,bayarFRS.csv
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}

async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))
  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })
```

```

    await csvWriter.writeRecords(data)
    console.log(`Output written to ${outputPath}`)
  }

function transformFactBayarFRS(data) {
  logStep("TRANSFORMATION", "Starting transformation for F_Bayar_FRS")
  return data.map((record, index) => {
    const transformed = {}
    transformed.BayarFRS_SK = index + 1
    transformed.Mahasiswa_SK = Number.parseInt(record.Mahasiswa_SK) || null
    transformed.Waktu_SK = Number.parseInt(record.Waktu_SK) || null
    transformed.Metode_Pembayaran_SK = Number.parseInt(record.Metode_Pembayaran_SK)
    || null
    transformed.Mahasiswa_NK = Number.parseInt(record.Mahasiswa_SK)
    transformed.Waktu_NK = record.Waktu_SK
    transformed.Metode_Pembayaran_NK = record.Metode_Pembayaran_SK
    transformed.Nominal_bayar = validatePositiveDecimal(record.Nominal_bayar)

    if (record.Status_pembayaran) {
      const status = record.Status_pembayaran.toString().toLowerCase()
      if (status === "true" || status === "1" || status === "success" || status === "berhasil") {
        transformed.Status_pembayaran = "Success"
        transformed.Is_Payment_Success = "Y"
      } else {
        transformed.Status_pembayaran = "Failed"
        transformed.Is_Payment_Success = "N"
      }
    } else {
      transformed.Status_pembayaran = "Unknown"
      transformed.Is_Payment_Success = "N"
    }
  })

  if (transformed.Nominal_bayar) {
    if (transformed.Nominal_bayar < 1000000) {
      transformed.Payment_Category = "Low"
    } else if (transformed.Nominal_bayar < 5000000) {
      transformed.Payment_Category = "Medium"
    } else {
      transformed.Payment_Category = "High"
    }
  } else {
    transformed.Payment_Category = "Unknown"
  }
}

```



```

transformed.Data_Quality_Flag = "VALID"
const issues = []
if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")
if (!transformed.Metode_Pembayaran_SK)
issues.push("MISSING_METODE_PEMBAYARAN")
  if (transformed.Nominal_bayar && transformed.Nominal_bayar > 20000000) {
    issues.push("EXCESSIVE_PAYMENT_AMOUNT")
  }
  if (transformed.Nominal_bayar && transformed.Nominal_bayar < 100000) {
    issues.push("UNUSUALLY_LOW_PAYMENT")
  }

if (issues.length > 0) {
  transformed.Data_Quality_Flag = "WARNING"
  transformed.Data_Quality_Issues = issues.join(", ")
}

transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
""),split("T")[0]}_001`
transformed.Record_Source = "bayarFRS.csv"

return transformed
})
}

function validatePositiveDecimal(value) {
  const num = Number.parseFloat(value)
  return num > 0 ? Number.parseFloat(num.toFixed(2)) : null
}

function loadFactBayarFRS(transformedData) {
  logStep("LOADING", "Starting loading for F_Bayar_FRS")
  const loadResults = {
    inserted: 0,
    errors: 0,
    warnings: 0,
    errorRecords: [],
    warningRecords: [],
  }

  transformedData.forEach((record, index) => {
    if (!record.Mahasiswa_SK || !record.Waktu_SK || !record.Metode_Pembayaran_SK) {

```

```

        loadResults.errors++
        loadResults.errorRecords.push({
            index: index,
            record: record,
            error: "Missing required dimension keys",
        })
    } else if (record.Data_Quality_Flag === "WARNING") {
        loadResults.warnings++
        loadResults.warningRecords.push({
            index: index,
            record: record,
            warning: record.Data_Quality_Issues,
        })
        loadResults.inserted++
    } else {
        loadResults.inserted++
    }
})

logStep(
    "LOADING",
    `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
    ${loadResults.warnings} warnings`
)

return loadResults
}

async function etlFactBayarFRS() {
    try {
        logStep("START", "Starting ETL process for F_Bayar_FRS")
        logStep("EXTRACTION", "Fetching data from bayarFRS CSV")
        const rawData = await fetchCSVData(
            "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Bayar_FRS-
            lagNftDgjFRCHST5Ov2SWnbH2JnBJe.csv"
        )
        logStep("EXTRACTION", `Extracted ${rawData.length} records from fact table`)
        const transformedData = transformFactBayarFRS(rawData)
        logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`)
        const loadResults = loadFactBayarFRS(transformedData)
        logStep("EXPORT", "Exporting transformed data to CSV")
        await exportToCSV(transformedData, "F_Bayar_FRS_Output.csv")
        logStep("COMPLETE", "ETL process for F_Bayar_FRS completed successfully")
        return {

```

```

    status: "success",
    extracted: rawData.length,
    transformed: transformedData.length,
    loaded: loadResults.inserted,
    errors: loadResults.errors,
    warnings: loadResults.warnings,
    sampleTransformedData: transformedData.slice(0, 3),
  }
} catch (error) {
  logStep("ERROR", `ETL process failed: ${error.message}`)
  console.error(error)
  return {
    status: "error",
    message: error.message,
  }
}
}
}

```

```

etlFactBayarFRS()
  .then((result) => {
    console.log("\nETL Process Result:", result)
  })
  .catch((error) => {
    console.error("ETL Process Failed:", error)
  })

```

Fakta Drop FRS

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] node "c:\Users\micle\Desktop\DLH\F_dropFRS_Historical.js"
[2025-05-27T23:11:09.745Z] [START] Starting ETL process for F_dropFRS
[2025-05-27T23:11:09.747Z] [EXTRACTION] Fetching data from dropFRS CSV
[2025-05-27T23:11:09.917Z] [EXTRACTION] Extracted 100 records from fact table
[2025-05-27T23:11:09.917Z] [EXTRACTION] Loading dimension tables for lookups
[2025-05-27T23:11:10.562Z] [EXTRACTION] Loaded 100 mahasiswa, 100 kelas, 100 waktu, 100 status, 20 dosen records
[2025-05-27T23:11:10.562Z] [TRANSFORMATION] Starting transformation for F_dropFRS
[2025-05-27T23:11:10.564Z] [TRANSFORMATION] Transformed 100 records
[2025-05-27T23:11:10.565Z] [LOADING] Starting loading for F_dropFRS
[2025-05-27T23:11:10.565Z] [LOADING] Loaded 100 records successfully, 0 errors, 0 warnings
[2025-05-27T23:11:10.565Z] [EXPORT] Exporting transformed data to CSV
Output written to F_dropFRS_Output.csv
[2025-05-27T23:11:10.568Z] [COMPLETE] ETL process for F_dropFRS completed successfully

ETL Process Result: {
  status: 'success',
  extracted: 100,
  transformed: 100,
  loaded: 100,
  errors: 0,
  warnings: 0,
  sampleTransformedData: [
    {
      dropFRS_SK: 1,
      Mahasiswa_SK: 23,
      Kelas_MK_SK: 26,
      Waktu_SK: 17,
      Status_SK: 70,
    }
  ]
}

DLH > F_dropFRS_Output.csv
> transformFa Aa No results

1 dropFRS_SK,Mahasiswa_SK,Kelas_MK_SK,Waktu_SK,Status_SK,Dosen_SK,NRP_NK,ID_Kelas_MK_NK,ID_Waktu_NK,ID_Status_NK,NIP_Dosen_Wali_NK,Jumlah_SKS_tendrop,Jum1
2 1,23,26,17,70,12,1000023,26,17,70,66629388,2,22,20,7,6,9.09,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
3 2,62,38,96,44,4,1000062,38,96,44,46913810,2,24,22,6,5,8.33,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
4 3,97,29,87,93,13,1000097,29,87,93,14265799,2,22,20,7,6,9.09,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
5 4,2,34,69,16,15,1000002,34,69,16,22575562,4,22,18,8,7,18.18,12.50,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
6 5,87,34,75,49,12,1000087,34,75,49,66629388,4,18,14,6,5,22.22,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
7 6,61,4,80,72,11,1000061,4,80,72,89254563,4,20,16,6,5,20.00,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
8 7,82,60,90,39,14,1000082,60,90,39,13999315,2,18,16,6,5,11.11,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
9 8,5,39,64,15,4,1000005,39,64,15,46913810,4,20,16,6,5,20.00,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
10 9,50,59,48,86,18,1000050,59,48,86,77827638,4,24,20,8,7,16.67,12.50,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
11 10,94,20,54,84,4,1000094,20,54,84,46913810,4,24,20,7,6,16.67,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
12 11,36,5,89,48,7,1000036,5,89,48,28728463,3,24,21,6,5,12.50,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
13 12,47,13,88,48,18,1000047,13,88,48,77827638,2,22,20,7,6,9.09,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
14 13,36,25,16,59,3,1000036,25,16,59,13356886,4,20,16,8,7,20.00,12.50,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
15 14,77,3,7,43,8,1000077,3,7,43,23756669,4,20,16,6,5,20.00,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
16 15,9,98,71,27,19,1000009,98,71,27,90801586,2,20,18,7,6,10.00,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
17 16,100,19,77,1,9,1000100,19,77,1,83197857,2,20,18,8,7,10.00,12.50,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
18 17,33,23,15,85,1,1000033,23,15,85,95822412,2,20,18,7,6,10.00,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
19 18,31,76,42,3,6,1000031,76,42,3,39958838,2,22,20,6,5,9.09,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
20 19,95,54,68,15,3,1000095,54,68,15,13356886,3,24,21,7,6,12.50,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
21 20,66,76,14,58,17,1000066,76,14,58,41227216,4,20,16,6,5,20.00,16.67,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
22 21,94,85,67,39,15,1000094,85,67,39,22575562,2,18,16,7,6,11.11,14.29,VALID,2025-05-27,BATCH_20250527_001,dropFRS.csv
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")
```

```
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}
```

```
async function fetchCSVData(url) {
  const response = await fetch(url)
  const csvText = await response.text()
```

```

return parse(csvText, {
  columns: true,
  skip_empty_lines: true,
  trim: true,
})
}

```

```

async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))
  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })
  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}

```

```

function transformFactDropFRS(data, dimMahasiswa, dimKelas, dimWaktu, dimStatus,
dimDosen) {

```

```

  logStep("TRANSFORMATION", "Starting transformation for F_dropFRS")

```

```

  return data.map((record, index) => {
    const transformed = {}
    transformed.dropFRS_SK = index + 1

```

```

    const mahasiswa = dimMahasiswa.find((m) => m.NRP == record.NRP_SK)
    transformed.Mahasiswa_SK = mahasiswa ? mahasiswa.Mahasiswa_SK : null

```

```

    const kelas = dimKelas.find((k) => k.ID_kelas_MK == record.ID_Kelas_MK_SK)
    transformed.Kelas_MK_SK = kelas ? kelas.Kelas_MK_SK : null

```

```

    const waktu = dimWaktu.find((w) => w.ID_Waktu == record.ID_Waktu_SK)
    transformed.Waktu_SK = waktu ? waktu.Waktu_SK : null

```

```

    const status = dimStatus.find((s) => s.ID_Status == record.ID_Status_SK)
    transformed.Status_SK = status ? status.Status_SK : null

```

```

    const dosen = dimDosen.find((d) => d.NIP_Dosen == record.NIP_Dosen_Wali_SK)
    transformed.Dosen_SK = dosen ? dosen.Dosen_SK : null

```

```

    transformed.NRP_NK = Number.parseInt(record.NRP_SK) || null
    transformed.ID_Kelas_MK_NK = record.ID_Kelas_MK_SK
    transformed.ID_Waktu_NK = record.ID_Waktu_SK
    transformed.ID_Status_NK = record.ID_Status_SK

```

```

transformed.NIP_Dosen_Wali_NK = Number.parseInt(record.NIP_Dosen_Wali_SK) || null

transformed.Jumlah_SKS_terdrop = validatePositiveInteger(record.Jumlah_SKS_terdrop)
transformed.Jumlah_SKS_Before_drop =
validatePositiveInteger(record.Jumlah_SKS_Before_drop)
transformed.Jumlah_SKS_After_drop =
validateNonNegativeInteger(record.Jumlah_SKS_After_drop)
transformed.Jumlah_MK_Before_Drop =
validatePositiveInteger(record.Jumlah_Mata_Kuliah_Terdaftar_Before_Drop)
transformed.Jumlah_MK_After_Drop =
validateNonNegativeInteger(record.Jumlah_Mata_Kuliah_Terdaftar_After_Drop)

if (transformed.Jumlah_SKS_Before_drop && transformed.Jumlah_SKS_terdrop) {
  transformed.SKS_Drop_Percentage = Number(
    ((transformed.Jumlah_SKS_terdrop / transformed.Jumlah_SKS_Before_drop) *
100).toFixed(2),
  )
} else {
  transformed.SKS_Drop_Percentage = null
}

if (transformed.Jumlah_MK_Before_Drop && transformed.Jumlah_MK_After_Drop) {
  const mkDropped = transformed.Jumlah_MK_Before_Drop -
transformed.Jumlah_MK_After_Drop
  transformed.MK_Drop_Percentage = Number(((mkDropped /
transformed.Jumlah_MK_Before_Drop) * 100).toFixed(2))
} else {
  transformed.MK_Drop_Percentage = null
}

transformed.Data_Quality_Flag = "VALID"
const issues = []

if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Kelas_MK_SK) issues.push("MISSING_KELAS")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")
if (!transformed.Status_SK) issues.push("MISSING_STATUS")
if (!transformed.Dosen_SK) issues.push("MISSING_DOSEN")

if (issues.length > 0) {
  transformed.Data_Quality_Flag = "WARNING"
  transformed.Data_Quality_Issues = issues.join(", ")
}

```

```

    transformed.Created_Date = new Date().toISOString().split("T")[0]
    transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_001`
    transformed.Record_Source = "dropFRS.csv"

    return transformed
  })
}

```

```

function validatePositiveInteger(value) {
  const num = Number(value)
  return Number.isInteger(num) && num > 0 ? num : null
}

```

```

function validateNonNegativeInteger(value) {
  const num = Number(value)
  return Number.isInteger(num) && num >= 0 ? num : null
}

```

```

function loadFactDropFRS(transformedData) {
  logStep("LOADING", "Starting loading for F_dropFRS")

```

```

  const loadResults = {
    inserted: 0,
    errors: 0,
    warnings: 0,
    errorRecords: [],
    warningRecords: [],
  }

```

```

  transformedData.forEach((record, index) => {
    if (!record.Mahasiswa_SK || !record.Kelas_MK_SK || !record.Waktu_SK) {
      loadResults.errors++
      loadResults.errorRecords.push({
        index: index,
        record: record,
        error: "Missing required dimension keys",
      })
    } else if (record.Data_Quality_Flag === "WARNING") {
      loadResults.warnings++
      loadResults.warningRecords.push({
        index: index,
        record: record,
        warning: record.Data_Quality_Issues,

```

```

    })
    loadResults.inserted++
  } else {
    loadResults.inserted++
  }
})

logStep(
  "LOADING",
  `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
  ${loadResults.warnings} warnings`,
)

return loadResults
}

async function etlFactDropFRS() {
  try {
    logStep("START", "Starting ETL process for F_dropFRS")

    logStep("EXTRACTION", "Fetching data from dropFRS CSV")
    const rawData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/dropFRS-
      Uk32dMwpmKKU4TGfbqmoNic0sieJSB.csv",
    )
    logStep("EXTRACTION", `Extracted ${rawData.length} records from fact table`)

    logStep("EXTRACTION", "Loading dimension tables for lookups")
    const dimMahasiswa = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Mahasiswa-
      ODzn65IS2Wxhzkfevt19w7rnpbq1Cy.csv",
    )
    const dimKelas = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Kelas_MK%20%283%29-
      arEvtzQeQANqCuLfqX4BjP1cmM5vMm.csv",
    )
    const dimWaktu = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Waktu%20%282%29-
      bBN4wnuL2mryLiopDAgmhsKt8i90a2.csv",
    )
    const dimStatus = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-
      storage.com/Dim_Status_Permohonan%20%282%29-
      0qt4qqLLrWGLvSlqj8TOegy2QdTK3z.csv",
    )
  }
}

```



```

    )
    const dimDosen = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Dosen_Wali%20%283%29-Kf57TxnnrnMot6hQbfzCPE2B1siPGB.csv",
    )

    dimMahasiswa.forEach((record, index) => {
      record.Mahasiswa_SK = index + 1
    })
    dimKelas.forEach((record, index) => {
      record.Kelas_MK_SK = index + 1
    })
    dimWaktu.forEach((record, index) => {
      record.Waktu_SK = index + 1
    })
    dimStatus.forEach((record, index) => {
      record.Status_SK = index + 1
    })
    dimDosen.forEach((record, index) => {
      record.Dosen_SK = index + 1
    })

    logStep(
      "EXTRACTION",
      `Loaded ${dimMahasiswa.length} mahasiswa, ${dimKelas.length} kelas, ${dimWaktu.length} waktu, ${dimStatus.length} status, ${dimDosen.length} dosen records`,
    )

    const transformedData = transformFactDropFRS(rawData, dimMahasiswa, dimKelas, dimWaktu, dimStatus, dimDosen)
    logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`)

    const loadResults = loadFactDropFRS(transformedData)

    logStep("EXPORT", "Exporting transformed data to CSV")
    const timestamp = new Date().toISOString().replace(/[: -]/g, "").split("T")[0]
    await exportToCSV(transformedData, `F_dropFRS_Output_${timestamp}.csv`)

    logStep("COMPLETE", "ETL process for F_dropFRS completed successfully")

    return {
      status: "success",
      extracted: rawData.length,
      transformed: transformedData.length,
    }
  }
}

```

```

        loaded: loadResults.inserted,
        errors: loadResults.errors,
        warnings: loadResults.warnings,
        sampleTransformedData: transformedData.slice(0, 3),
    }
} catch (error) {
    logStep("ERROR", `ETL process failed: ${error.message}`)
    console.error(error)
    return {
        status: "error",
        message: error.message,
    }
}
}

etlFactDropFRS()
    .then((result) => {
        console.log("\nETL Process Result:", result)
    })
    .catch((error) => {
        console.error("ETL Process Failed:", error)
    })

```

Fakta Setujui FRS

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] node "c:\Users\micle\Desktop\DLH\F_setujui_FRS_Historical.js"
[2025-05-27T23:13:03.349Z] [START] Starting ETL process for F_setujui_FRS
[2025-05-27T23:13:03.351Z] [EXTRACTION] Fetching data from setujuiFRS CSV
[2025-05-27T23:13:03.994Z] [EXTRACTION] Extracted 100 records from fact table
[2025-05-27T23:13:03.994Z] [TRANSFORMATION] Starting transformation for F_setujui_FRS
[2025-05-27T23:13:03.995Z] [TRANSFORMATION] Transformed 100 records
[2025-05-27T23:13:03.996Z] [LOADING] Starting loading for F_setujui_FRS
[2025-05-27T23:13:03.996Z] [LOADING] Loaded 100 records successfully, 0 errors, 0 warnings
[2025-05-27T23:13:03.996Z] [EXPORT] Exporting transformed data to CSV
Output written to F_setujui_FRS_Output.csv
[2025-05-27T23:13:04.000Z] [COMPLETE] ETL process for F_setujui_FRS completed successfully

ETL Process Result: {
  status: 'success',
  extracted: 100,
  transformed: 100,
  loaded: 100,
  errors: 0,
  warnings: 0,
  sampleTransformedData: [
    {
      setujuiFRS_SK: 1,
      Mahasiswa_SK: 1000099,
      Waktu_SK: 100,
      Dosen_SK: 90801586,
      Status_SK: 1,
      Mahasiswa_NK: 1000099,
      Waktu_NK: '100',
      Dosen_Wali_NK: 90801586,
      Status_Permohonan_NK: '1',
      Jumlah_SKS_FRS: 3,
      Jumlah_MK_FRS: 3,
      Sks_tersisa_saa
    }
  ]
}

DLH > F_setujui_FRS_Output.csv
> transformFa Aa _ab_* No results ↑

1 setujuiFRS_SK,Mahasiswa_SK,Waktu_SK,Dosen_SK,Status_SK,Mahasiswa_NK,Waktu_NK,Dosen_Wali_NK,Status_Permohonan_NK,Jumlah_SKS_FRS,Jumlah_MK_FRS,Sks_tersisa_saa
2 1,1000099,100,90801586,1,1000099,100,90801586,1,7,2,15,22,3.50,31.82,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
3 2,1000052,35,13356886,2,1000052,35,13356886,2,24,8,0,24,3.00,100.00,Overload,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
4 3,1000041,28,23756669,8,1000041,28,23756669,8,20,6,0,20,3.33,100.00,Heavy,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
5 4,1000034,18,66629388,4,1000034,18,66629388,4,23,7,1,24,3.29,95.83,Overload,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
6 5,1000034,96,21668732,10,1000034,96,21668732,10,24,8,0,22,3.00,100.00,Overload,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
7 6,1000047,29,13999315,3,1000047,29,13999315,3,21,7,0,20,3.00,105.00,Heavy,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
8 7,1000097,7,41227216,2,1000097,7,41227216,2,11,3,13,24,3.67,45.83,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
9 8,1000055,77,77827638,2,1000055,77,77827638,2,18,6,6,24,3.00,75.00,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
10 9,1000060,68,22575562,5,1000060,68,22575562,5,6,2,18,24,3.00,25.00,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
11 10,1000093,15,24942603,9,1000093,15,24942603,9,14,4,10,24,3.50,58.33,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
12 11,1000044,15,46913810,5,1000044,15,46913810,5,11,3,11,22,3.67,50.00,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
13 12,1000001,93,23756669,5,1000001,93,23756669,5,11,3,13,24,3.67,45.83,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
14 13,1000014,81,89254563,5,1000014,81,89254563,5,22,7,2,24,3.14,91.67,Overload,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
15 14,1000026,20,95822412,6,1000026,20,95822412,6,11,3,13,24,3.67,45.83,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
16 15,1000100,68,28728463,1,1000100,68,28728463,1,16,5,6,22,3.20,72.73,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
17 16,1000003,15,13561597,6,1000003,15,13561597,6,13,4,7,20,3.25,65.00,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
18 17,1000031,73,23756669,2,1000031,73,23756669,2,21,7,0,20,3.00,105.00,Heavy,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
19 18,1000098,69,95822412,3,1000098,69,95822412,3,21,7,3,24,3.00,87.50,Heavy,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
20 19,1000022,34,83197857,9,1000022,34,83197857,9,19,6,1,20,3.17,95.00,Heavy,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
21 20,1000070,97,83197857,4,1000070,97,83197857,4,15,5,7,22,3.00,68.18,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
22 21,1000086,84,77827638,6,1000086,84,77827638,6,22,7,0,22,3.14,100.00,Overload,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
23 22,1000016,32,89254563,4,1000016,32,89254563,4,16,5,4,20,3.20,80.00,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
24 23,1000076,71,95822412,4,1000076,71,95822412,4,13,4,7,20,3.25,65.00,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
25 24,1000010,91,90801586,1,1000010,91,90801586,1,8,2,12,20,4.00,40.00,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
26 25,1000043,10,28728463,9,1000043,10,28728463,9,14,4,10,24,3.50,58.33,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
27 26,1000063,28,95822412,9,1000063,28,95822412,9,24,8,0,24,3.00,100.00,Overload,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
28 27,1000061,32,24942603,8,1000061,32,24942603,8,12,4,8,20,3.00,60.00,Normal,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
29 28,1000013,85,13356886,7,1000013,85,13356886,7,19,6,3,22,3.17,86.36,Heavy,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
30 29,1000050,04,13356886,4,1000050,04,13356886,4,9,3,11,20,3.00,45.00,Light,VALID,2025-05-27,BATCH_20250527_001,setujuiFRS.csv
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}
```

```

async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

```

```

async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))
  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })
  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}

```

```

function transformFactSetujuiFRS(data) {
  logStep("TRANSFORMATION", "Starting transformation for F_setujui_FRS")

  return data.map((record, index) => {
    const transformed = {}

    transformed.setujuiFRS_SK = index + 1
    transformed.Mahasiswa_SK = Number.parseInt(record.Mahasiswa_SK) || null
    transformed.Waktu_SK = Number.parseInt(record.Waktu_SK) || null
    transformed.Dosen_SK = Number.parseInt(record.Dosen_Wali_SK) || null
    transformed.Status_SK = Number.parseInt(record.Status_Permohonan_SK) || null

    transformed.Mahasiswa_NK = Number.parseInt(record.Mahasiswa_SK)
    transformed.Waktu_NK = record.Waktu_SK
    transformed.Dosen_Wali_NK = Number.parseInt(record.Dosen_Wali_SK)
    transformed.Status_Permohonan_NK = record.Status_Permohonan_SK

    transformed.Jumlah_SKS_FRS = validatePositiveInteger(record.Jumlah_SKS_FRS)
    transformed.Jumlah_MK_FRS = validatePositiveInteger(record.Jumlah_MK_FRS)
  })
}

```

```
transformed.Sks_tersisa_saat_ini = validateNonNegativeInteger(record.Sks_tersisa_saat_ini)
transformed.Sks_maksimum_semester_ini =
validatePositiveInteger(record.Sks_maksimum_semester_ini)
```

```
if (transformed.Jumlah_MK_FRS && transformed.Jumlah_SKS_FRS) {
    transformed.Avg_SKS_per_MK = (transformed.Jumlah_SKS_FRS /
transformed.Jumlah_MK_FRS).toFixed(2)
} else {
    transformed.Avg_SKS_per_MK = null
}
```

```
if (transformed.Sks_maksimum_semester_ini && transformed.Jumlah_SKS_FRS) {
    transformed.SK_S_Utilization_Percentage = (
        (transformed.Jumlah_SKS_FRS / transformed.Sks_maksimum_semester_ini) *
        100
    ).toFixed(2)
} else {
    transformed.SK_S_Utilization_Percentage = null
}
```

```
if (transformed.Jumlah_SKS_FRS) {
    if (transformed.Jumlah_SKS_FRS < 12) {
        transformed.Workload_Category = "Light"
    } else if (transformed.Jumlah_SKS_FRS <= 18) {
        transformed.Workload_Category = "Normal"
    } else if (transformed.Jumlah_SKS_FRS <= 21) {
        transformed.Workload_Category = "Heavy"
    } else {
        transformed.Workload_Category = "Overload"
    }
} else {
    transformed.Workload_Category = "Unknown"
}
```

```
transformed.Data_Quality_Flag = "VALID"
const issues = []
```

```
if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")
if (!transformed.Dosen_SK) issues.push("MISSING_DOSEN")
if (!transformed.Status_SK) issues.push("MISSING_STATUS")
```

```
if (issues.length > 0) {
    transformed.Data_Quality_Flag = "WARNING"
```

```

    transformed.Data_Quality_Issues = issues.join(", ")
  }

  transformed.Created_Date = new Date().toISOString().split("T")[0]
  transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_001`
  transformed.Record_Source = "setujuiFRS.csv"

  return transformed
})
}

function validatePositiveInteger(value) {
  const num = Number.parseInt(value)
  return num > 0 ? num : null
}

function validateNonNegativeInteger(value) {
  const num = Number.parseInt(value)
  return num >= 0 ? num : null
}

function loadFactSetujuiFRS(transformedData) {
  logStep("LOADING", "Starting loading for F_setujui_FRS")

  const loadResults = {
    inserted: 0,
    errors: 0,
    warnings: 0,
    errorRecords: [],
    warningRecords: [],
  }

  transformedData.forEach((record, index) => {
    if (!record.Mahasiswa_SK || !record.Waktu_SK || !record.Dosen_SK || !record.Status_SK) {
      loadResults.errors++
      loadResults.errorRecords.push({
        index: index,
        record: record,
        error: "Missing required dimension keys",
      })
    } else if (record.Data_Quality_Flag === "WARNING") {
      loadResults.warnings++
      loadResults.warningRecords.push({

```

```

        index: index,
        record: record,
        warning: record.Data_Quality_Issues,
    })
    loadResults.inserted++
} else {
    loadResults.inserted++
}
})

logStep(
    "LOADING",
    `Loaded ${loadResults.inserted} records successfully, ${loadResults.errors} errors,
    ${loadResults.warnings} warnings`,
)

return loadResults
}

async function etlFactSetujuiFRS() {
    try {
        logStep("START", "Starting ETL process for F_setujui_FRS")

        logStep("EXTRACTION", "Fetching data from setujuiFRS CSV")
        const rawData = await fetchCSVData(
            "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Setujui_FRS-9TTEcs2gfzXYkazPgWdaUj9RiBUMrW.csv",
        )
        logStep("EXTRACTION", `Extracted ${rawData.length} records from fact table`)

        const transformedData = transformFactSetujuiFRS(rawData)
        logStep("TRANSFORMATION", `Transformed ${transformedData.length} records`)

        const loadResults = loadFactSetujuiFRS(transformedData)

        logStep("EXPORT", "Exporting transformed data to CSV")
        await exportToCSV(transformedData, "F_setujui_FRS_Output.csv")

        logStep("COMPLETE", "ETL process for F_setujui_FRS completed successfully")

        return {
            status: "success",
            extracted: rawData.length,
            transformed: transformedData.length,
        }
    }
}

```

```

        loaded: loadResults.inserted,
        errors: loadResults.errors,
        warnings: loadResults.warnings,
        sampleTransformedData: transformedData.slice(0, 3),
    }
} catch (error) {
    logStep("ERROR", `ETL process failed: ${error.message}`)
    console.error(error)
    return {
        status: "error",
        message: error.message,
    }
}
}
}

```

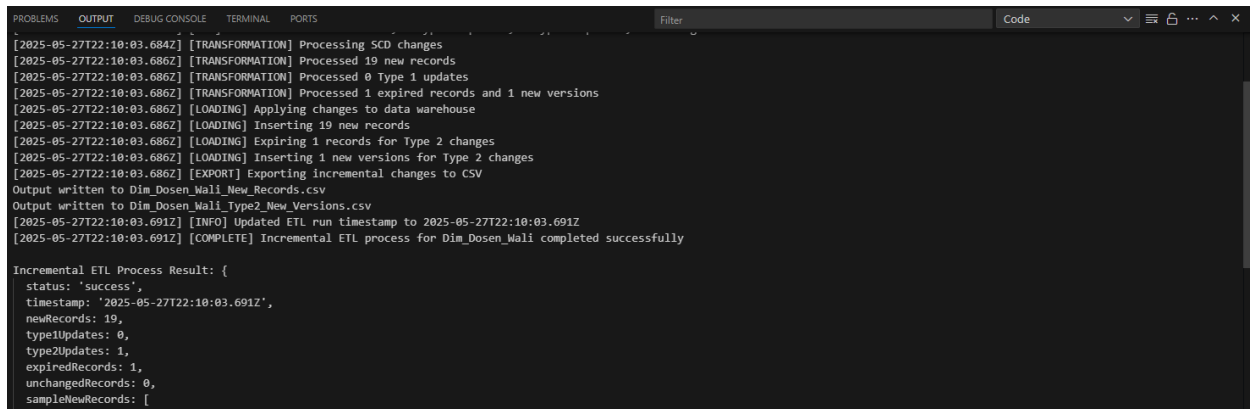
```

etlFactSetujuiFRS()
    .then((result) => {
        console.log("\nETL Process Result:", result)
    })
    .catch((error) => {
        console.error("ETL Process Failed:", error)
    })

```

Incremental ETL

Dimensi Dosen_wali



```

[2025-05-27T22:10:03.684Z] [TRANSFORMATION] Processing SCD changes
[2025-05-27T22:10:03.686Z] [TRANSFORMATION] Processed 19 new records
[2025-05-27T22:10:03.686Z] [TRANSFORMATION] Processed 0 Type 1 updates
[2025-05-27T22:10:03.686Z] [TRANSFORMATION] Processed 1 expired records and 1 new versions
[2025-05-27T22:10:03.686Z] [LOADING] Applying changes to data warehouse
[2025-05-27T22:10:03.686Z] [LOADING] Inserting 19 new records
[2025-05-27T22:10:03.686Z] [LOADING] Expiring 1 records for Type 2 changes
[2025-05-27T22:10:03.686Z] [LOADING] Inserting 1 new versions for Type 2 changes
[2025-05-27T22:10:03.686Z] [EXPORT] Exporting incremental changes to CSV
Output written to Dim_Dosen_Wali_New_Records.csv
Output written to Dim_Dosen_Wali_Type2_New_Versions.csv
[2025-05-27T22:10:03.691Z] [INFO] Updated ETL run timestamp to 2025-05-27T22:10:03.691Z
[2025-05-27T22:10:03.691Z] [COMPLETE] Incremental ETL process for Dim_Dosen_Wali completed successfully

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-27T22:10:03.691Z',
  newRecords: 19,
  type1Updates: 0,
  type2Updates: 1,
  expiredRecords: 1,
  unchangedRecords: 0,
  sampleNewRecords: [

```



```
DLH > Dim_Dosen_Wali_New_Records.csv
> transformFa Aa .ab .* No results ↑ ↓ ≡ ×

1 Dosen_SK,NIP_Dosen>Nama_Dosen,Fakultas,Departemen,Program_Studi,Has_Academic_Title,Display_Name,Faculty_Department,Effective_Date,End_Date,Current_Flag,Version_Number,
2 2,95822412,"Ozy Haryanto, S.I.Kom",FTSPK,Informatika,S1 Teknik Mesin,N,"Ozy Haryanto, S.I.Kom",FTSPK - Informatika,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATC
3 3,24942683,Digdaya Narpati,FTSPK,Informatika,S1 Sistem Informasi,N,Digdaya Narpati,FTSPK - Informatika,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_I
4 4,13356886,Tiara Anggriawan,FTEIC,Elektro,S1 Teknik Mesin,N,Tiara Anggriawan,FTEIC - Elektro,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
5 5,46913810,"Dr. Rafid Irawan, M.TI.",FTM,Elektro,S1 Teknik Mesin,Y,"Rafid wan, .",FTM - Elektro,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
6 6,42868828,Asmadi Mayasari,FTSPK,Mesin,S1 Sistem Informasi,N,Asmadi Mayasari,FTSPK - Mesin,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
7 7,39958838,Sutan Prima Habibi,FTI,Kelautan,S1 Sistem Informasi,N,Sutan Prima Habibi,FTI - Kelautan,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
8 8,28728463,H. Malik Pradana,FTI,Kelautan,S1 Teknik Sipil,N,H. Malik Pradana,FTI - Kelautan,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
9 9,23756669,Dwi Wahyuni,FTSPK,Informatika,S1 Teknik Elektro,N,Dwi Wahyuni,FTSPK - Informatika,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
10 10,83197857,"H. Daryani Mayasari, S.Kom",FTI,Elektro,S1 Informatika,N,"H. Daryani Mayasari, S.Kom",FTI - Elektro,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_
11 11,21668732,Tiara Yulianti,FTM,Kelautan,S1 Sistem Informasi,N,Tiara Yulianti,FTM - Kelautan,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}

// Function to fetch CSV data from URL
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })

  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}
```

```

// Function to get last ETL run timestamp
function getLastETLRunTimestamp() {
  return "2023-05-01T00:00:00Z"
}

// Function to update ETL run timestamp
function updateETLRunTimestamp() {
  const timestamp = new Date().toISOString()
  logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
  return timestamp
}

// Function to identify changed records for SCD processing
function identifyChangedDimDosenWaliRecords(sourceData, existingData) {
  logStep("CDC", "Identifying changed records for Dim_Dosen_Wali")

  const newRecords = []
  const scdType1Updates = []
  const scdType2Updates = []
  const unchangedRecords = []

  sourceData.forEach((sourceRecord) => {
    // Find existing record by NIP_Dosen (natural key)
    const existingRecord = existingData.find(
      (record) => record.NIP_Dosen === Number.parseInt(sourceRecord.NIP_Dosen) &&
      record.Current_Flag === "Y",
    )

    if (!existingRecord) {
      // New record
      newRecords.push(sourceRecord)
    } else {
      // Check for changes in SCD Type 1 fields (overwrite)
      const type1Changes = {}
      let hasType1Changes = false

      // Nama_Dosen - SCD Type 1
      if (sourceRecord.Nama_Dosen !== existingRecord.Nama_Dosen) {
        type1Changes.Nama_Dosen = sourceRecord.Nama_Dosen
        hasType1Changes = true
      }

      // Check for changes in SCD Type 2 fields (track history)

```

```

const type2Changes = {}
let hasType2Changes = false

// Fakultas - SCD Type 2
if (sourceRecord.Fakultas !== existingRecord.Fakultas) {
  type2Changes.Fakultas = sourceRecord.Fakultas
  hasType2Changes = true
}

// Departemen - SCD Type 2
if (sourceRecord.Departemen !== existingRecord.Departemen) {
  type2Changes.Departemen = sourceRecord.Departemen
  hasType2Changes = true
}

// Program_Studi - SCD Type 2
if (sourceRecord.Program_Studi !== existingRecord.Program_Studi) {
  type2Changes.Program_Studi = sourceRecord.Program_Studi
  hasType2Changes = true
}

if (hasType1Changes || hasType2Changes) {
  if (hasType1Changes && hasType2Changes) {
    // Both types of changes - handle as Type 2 (create new version) and update Type 1
fields
    scdType2Updates.push({
      sourceRecord,
      existingRecord,
      type1Changes,
      type2Changes,
      changeType: "MIXED",
    })
  } else if (hasType1Changes) {
    // Only Type 1 changes - update in place
    scdType1Updates.push({
      sourceRecord,
      existingRecord,
      type1Changes,
      changeType: "TYPE1_ONLY",
    })
  } else {
    // Only Type 2 changes - create new version
    scdType2Updates.push({
      sourceRecord,

```

```

        existingRecord,
        type2Changes,
        changeType: "TYPE2_ONLY",
    })
    }
    } else {
        unchangedRecords.push(sourceRecord)
    }
    }
    })

return {
    newRecords,
    scdType1Updates,
    scdType2Updates,
    unchangedRecords,
}
}

// Transform new records
function transformNewDimDosenWaliRecords(newRecords, maxSK = 0) {
    return newRecords.map((record, index) => {
        const transformed = {}

        // Generate surrogate key
        transformed.Dosen_SK = maxSK + index + 1

        // NIP_Dosen (Natural Key) - SCD Type 0
        transformed.NIP_Dosen = record.NIP_Dosen ? Number.parseInt(record.NIP_Dosen, 10) :
null

        // Nama_Dosen - SCD Type 1
        transformed>Nama_Dosen = record>Nama_Dosen ? record>Nama_Dosen.trim() : ""

        // Fakultas - SCD Type 2
        transformed.Fakultas = record.Fakultas ? record.Fakultas.trim() : ""

        // Departemen - SCD Type 2
        transformed.Departemen = record.Departemen ? record.Departemen.trim() : ""

        // Program_Studi - SCD Type 2
        transformed.Program_Studi = record.Program_Studi ? record.Program_Studi.trim() : ""

        // Add derived attributes for better analytics

```

```

if (transformed>Nama_Dosen) {
  // Extract title/degree from name
  const titles = ["Dr.", "Prof.", "Ir.", "S.T.", "M.T.", "M.Sc.", "Ph.D."]
  let hasTitle = false

  for (const title of titles) {
    if (transformed>Nama_Dosen.includes(title)) {
      hasTitle = true
      break
    }
  }
}

transformed.Has_Academic_Title = hasTitle ? "Y" : "N"

// Clean name (remove common titles for display)
let cleanName = transformed>Nama_Dosen
titles.forEach((title) => {
  cleanName = cleanName.replace(new RegExp(title, "g"), "").trim()
})
transformed.Display_Name = cleanName.replace(/\s+/g, " ").trim()
} else {
  transformed.Has_Academic_Title = "N"
  transformed.Display_Name = ""
}

// Add faculty-department hierarchy
if (transformed.Fakultas && transformed.Departemen) {
  transformed.Faculty_Department = `${transformed.Fakultas} - ${transformed.Departemen}`
} else {
  transformed.Faculty_Department = transformed.Fakultas || transformed.Departemen || ""
}

// Add SCD Type 2 fields
transformed.Effective_Date = new Date().toISOString().split("T")[0]
transformed.End_Date = "9999-12-31"
transformed.Current_Flag = "Y"
transformed.Version_Number = 1

// Add audit fields
transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.Updated_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
""),split("T")[0]}_INC`

```

```

    return transformed
  })
}

// Process SCD Type 1 updates
function processSCDType1Updates(scdType1Updates) {
  const updatedRecords = []

  scdType1Updates.forEach(({ existingRecord, type1Changes }) => {
    const updatedRecord = { ...existingRecord }

    // Apply Type 1 changes (overwrite)
    Object.keys(type1Changes).forEach((field) => {
      updatedRecord[field] = type1Changes[field]
    })

    // Update derived attributes if name changed
    if (type1Changes>Nama_Dosen) {
      const titles = ["Dr.", "Prof.", "Ir.", "S.T.", "M.T.", "M.Sc.", "Ph.D."]
      let hasTitle = false

      for (const title of titles) {
        if (updatedRecord>Nama_Dosen.includes(title)) {
          hasTitle = true
          break
        }
      }
    }

    updatedRecord.Has_Academic_Title = hasTitle ? "Y" : "N"

    // Clean name
    let cleanName = updatedRecord>Nama_Dosen
    titles.forEach((title) => {
      cleanName = cleanName.replace(new RegExp(title, "g"), "").trim()
    })
    updatedRecord.Display_Name = cleanName.replace(/\s+/g, " ").trim()
  })

  // Update audit fields
  updatedRecord.Updated_Date = new Date().toISOString().split("T")[0]
  updatedRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
  "").split("T")[0]}_INC`

  updatedRecords.push(updatedRecord)
}

```

```

    })

    return updatedRecords
}

// Process SCD Type 2 updates
function processSCDType2Updates(scdType2Updates, maxSK) {
    const expiredRecords = []
    const newVersionRecords = []
    let currentMaxSK = maxSK

    scdType2Updates.forEach(({ sourceRecord, existingRecord, type1Changes, type2Changes,
    changeType }) => {
        // Expire the current record
        const expiredRecord = { ...existingRecord }
        expiredRecord.End_Date = new Date().toISOString().split("T")[0]
        expiredRecord.Current_Flag = "N"
        expiredRecord.Updated_Date = new Date().toISOString().split("T")[0]
        expiredRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
        "").split("T")[0]}_INC`
        expiredRecords.push(expiredRecord)

        // Create new version
        const newVersion = { ...existingRecord }
        newVersion.Dosen_SK = ++currentMaxSK

        // Apply Type 1 changes (if any)
        if (type1Changes) {
            Object.keys(type1Changes).forEach((field) => {
                newVersion[field] = type1Changes[field]
            })
        }

        // Update derived attributes if name changed
        if (type1Changes>Nama_Dosen) {
            const titles = ["Dr.", "Prof.", "Ir.", "S.T.", "M.T.", "M.Sc.", "Ph.D."]
            let hasTitle = false

            for (const title of titles) {
                if (newVersion>Nama_Dosen.includes(title)) {
                    hasTitle = true
                    break
                }
            }
        }
    })
}

```

```

newVersion.Has_Academic_Title = hasTitle ? "Y" : "N"

// Clean name
let cleanName = newVersion>Nama_Dosen
titles.forEach((title) => {
  cleanName = cleanName.replace(new RegExp(title, "g"), "").trim()
})
newVersion.Display_Name = cleanName.replace(/\s+/g, " ").trim()
}
}

// Apply Type 2 changes
if (type2Changes) {
  Object.keys(type2Changes).forEach((field) => {
    newVersion[field] = type2Changes[field]
  })
}

// Update faculty-department hierarchy
if (newVersion.Fakultas && newVersion.Departemen) {
  newVersion.Faculty_Department = `${newVersion.Fakultas} - ${newVersion.Departemen}`
} else {
  newVersion.Faculty_Department = newVersion.Fakultas || newVersion.Departemen || ""
}

// Update SCD Type 2 fields
newVersion.Effective_Date = new Date().toISOString().split("T")[0]
newVersion.End_Date = "9999-12-31"
newVersion.Current_Flag = "Y"
newVersion.Version_Number = existingRecord.Version_Number + 1

// Update audit fields
newVersion.Created_Date = new Date().toISOString().split("T")[0]
newVersion.Updated_Date = new Date().toISOString().split("T")[0]
newVersion.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
"").split("T")[0]}_INC`

  newVersionRecords.push(newVersion)
})

return { expiredRecords, newVersionRecords }
}

// Main incremental ETL function for Dim_Dosen_Wali

```



```

async function incrementalETLDimDosenWali() {
  try {
    logStep("START", "Starting incremental ETL process for Dim_Dosen_Wali")

    // Get last ETL run timestamp
    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Fetching source data from Dim_Dosen_Wali CSV")
    const sourceData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Dosen_Wali%20%283%29-Kf57TxnnrnMot6hQbfzCPE2B1siPGB.csv",
    )
    logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

    // Simulate existing warehouse data
    const existingData = [
      {
        Dosen_SK: 1,
        NIP_Dosen: 14265799,
        Nama_Dosen: "Dr. John Smith",
        Fakultas: "FTIF",
        Departemen: "Informatika",
        Program_Studi: "Teknik Informatika",
        Has_Academic_Title: "Y",
        Display_Name: "John Smith",
        Faculty_Department: "FTIF - Informatika",
        Effective_Date: "2023-05-01",
        End_Date: "9999-12-31",
        Current_Flag: "Y",
        Version_Number: 1,
        Created_Date: "2023-05-01",
        Updated_Date: "2023-05-01",
        ETL_Batch_ID: "BATCH_20230501_001",
      },
    ]

    logStep("EXTRACTION", `Read ${existingData.length} existing records from data warehouse`)

    // 2. CHANGE DATA CAPTURE
    logStep("CDC", "Identifying changed records")
  }
}

```

```

    const { newRecords, scdType1Updates, scdType2Updates, unchangedRecords } =
identifyChangedDimDosenWaliRecords(
    sourceData,
    existingData,
)

logStep(
    "CDC",
    `Identified ${newRecords.length} new records, ${scdType1Updates.length} Type 1 updates,
${scdType2Updates.length} Type 2 updates, ${unchangedRecords.length} unchanged records`,
)

// 3. TRANSFORMATION
logStep("TRANSFORMATION", "Processing SCD changes")

// Get max SK from existing data
const maxSK = Math.max(...existingData.map((r) => r.Dosen_SK), 0)

// Transform new records
const transformedNewRecords = transformNewDimDosenWaliRecords(newRecords, maxSK)

// Process SCD Type 1 updates
const type1UpdatedRecords = processSCDType1Updates(scdType1Updates)

// Process SCD Type 2 updates
const { expiredRecords, newVersionRecords } =
processSCDType2Updates(scdType2Updates, maxSK + newRecords.length)

logStep("TRANSFORMATION", `Processed ${transformedNewRecords.length} new records`)
logStep("TRANSFORMATION", `Processed ${type1UpdatedRecords.length} Type 1
updates`)
logStep(
    "TRANSFORMATION",
    `Processed ${expiredRecords.length} expired records and ${newVersionRecords.length}
new versions`,
)

// 4. LOADING
logStep("LOADING", "Applying changes to data warehouse")

if (transformedNewRecords.length > 0) {
    logStep("LOADING", `Inserting ${transformedNewRecords.length} new records`)
}

```

```

    if (type1UpdatedRecords.length > 0) {
      logStep("LOADING", `Updating ${type1UpdatedRecords.length} records with Type 1
changes`)
    }

    if (expiredRecords.length > 0) {
      logStep("LOADING", `Expiring ${expiredRecords.length} records for Type 2 changes`)
    }

    if (newVersionRecords.length > 0) {
      logStep("LOADING", `Inserting ${newVersionRecords.length} new versions for Type 2
changes`)
    }

    // 5. EXPORT TO CSV
    logStep("EXPORT", "Exporting incremental changes to CSV")

    if (transformedNewRecords.length > 0) {
      await exportToCSV(transformedNewRecords, "Dim_Dosen_Wali_New_Records.csv")
    }

    if (type1UpdatedRecords.length > 0) {
      await exportToCSV(type1UpdatedRecords, "Dim_Dosen_Wali_Type1_Updates.csv")
    }

    if (newVersionRecords.length > 0) {
      await exportToCSV(newVersionRecords, "Dim_Dosen_Wali_Type2_New_Versions.csv")
    }

    // Update ETL run timestamp
    const newTimestamp = updateETLRunTimestamp()

    logStep("COMPLETE", "Incremental ETL process for Dim_Dosen_Wali completed
successfully")

    return {
      status: "success",
      timestamp: newTimestamp,
      newRecords: transformedNewRecords.length,
      type1Updates: type1UpdatedRecords.length,
      type2Updates: newVersionRecords.length,
      expiredRecords: expiredRecords.length,
      unchangedRecords: unchangedRecords.length,
      sampleNewRecords: transformedNewRecords.slice(0, 2),
    }
  }
}

```

```

        sampleType1Updates: type1UpdatedRecords.slice(0, 2),
        sampleType2Updates: newVersionRecords.slice(0, 2),
    }
} catch (error) {
    logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
    console.error(error)
    return {
        status: "error",
        message: error.message,
    }
}
}
}

```

// Run the incremental ETL process using promise chaining
incrementalETLDimDosenWali()

```

.then((result) => {
    console.log("\nIncremental ETL Process Result:", result)
})
.catch((error) => {
    console.error("Incremental ETL Process Failed:", error)
})

```

Dimensi Mahasiswa

```

[Running] node "c:\Users\micle\Desktop\DLH\Dim_mahasiswa_incremental.js"
[2025-05-27T22:42:43.366Z] [START] Starting incremental ETL process for Dim_Mahasiswa
[2025-05-27T22:42:43.367Z] [INFO] Last ETL run timestamp: 2023-05-01T00:00:00Z
[2025-05-27T22:42:43.367Z] [EXTRACTION] Fetching source data from Dim_Mahasiswa CSV
[2025-05-27T22:42:44.081Z] [EXTRACTION] Extracted 100 records from source
[2025-05-27T22:42:44.081Z] [EXTRACTION] Read 1 existing records from data warehouse
[2025-05-27T22:42:44.081Z] [CDC] Identifying changed records
[2025-05-27T22:42:44.081Z] [CDC] Identifying changed records for Dim_Mahasiswa
[2025-05-27T22:42:44.082Z] [CDC] Identified 99 new records, 0 Type 1 updates, 1 Type 2 updates, 0 unchanged records
[2025-05-27T22:42:44.082Z] [TRANSFORMATION] Processing SCD changes
[2025-05-27T22:42:44.084Z] [TRANSFORMATION] Processed 99 new records
[2025-05-27T22:42:44.084Z] [TRANSFORMATION] Processed 0 Type 1 updates
[2025-05-27T22:42:44.084Z] [TRANSFORMATION] Processed 1 expired records and 1 new versions
[2025-05-27T22:42:44.084Z] [LOADING] Applying changes to data warehouse
[2025-05-27T22:42:44.084Z] [LOADING] Inserting 99 new records
[2025-05-27T22:42:44.084Z] [LOADING] Expiring 1 records for Type 2 changes
[2025-05-27T22:42:44.084Z] [LOADING] Inserting 1 new versions for Type 2 changes
[2025-05-27T22:42:44.084Z] [EXPORT] Exporting incremental changes to CSV
Output written to Dim_Mahasiswa_New_Records.csv
Output written to Dim_Mahasiswa_Type2_New_Versions.csv
[2025-05-27T22:42:44.090Z] [INFO] Updated ETL run timestamp to 2025-05-27T22:42:44.090Z
[2025-05-27T22:42:44.090Z] [COMPLETE] Incremental ETL process for Dim_Mahasiswa completed successfully

```

```

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-27T22:42:44.090Z',
  newRecords: 99,
  type1Updates: 0,
  type2Updates: 1,
  expiredRecords: 1,
  unchangedRecords: 0,
  sampleNewRecords: [
    {
      Mahasiswa_SK: 2,
      Mahasiswa_NK: 1,
      Mahasiswa_NL: 1,
      Mahasiswa_NP: 1,
      Mahasiswa_NR: 1,
      Mahasiswa_NS: 1,
      Mahasiswa_NT: 1,
      Mahasiswa_NU: 1,
      Mahasiswa_NV: 1,
      Mahasiswa_NW: 1,
      Mahasiswa_NX: 1,
      Mahasiswa_NY: 1,
      Mahasiswa_NZ: 1,
      Mahasiswa_OK: 1,
      Mahasiswa_OL: 1,
      Mahasiswa_OT: 1,
      Mahasiswa_OW: 1,
      Mahasiswa_OX: 1,
      Mahasiswa_OY: 1,
      Mahasiswa_OZ: 1,
      Mahasiswa_PK: 1,
      Mahasiswa_PL: 1,
      Mahasiswa_PT: 1,
      Mahasiswa_PW: 1,
      Mahasiswa_PX: 1,
      Mahasiswa_PY: 1,
      Mahasiswa_PZ: 1,
      Mahasiswa_QK: 1,
      Mahasiswa_QL: 1,
      Mahasiswa_QT: 1,
      Mahasiswa_QW: 1,
      Mahasiswa_QX: 1,
      Mahasiswa_QY: 1,
      Mahasiswa_QZ: 1,
      Mahasiswa_RK: 1,
      Mahasiswa_RL: 1,
      Mahasiswa_RT: 1,
      Mahasiswa_RW: 1,
      Mahasiswa_RX: 1,
      Mahasiswa_RY: 1,
      Mahasiswa_RZ: 1,
      Mahasiswa_SK: 1,
      Mahasiswa_SL: 1,
      Mahasiswa_ST: 1,
      Mahasiswa_SW: 1,
      Mahasiswa_SX: 1,
      Mahasiswa_SY: 1,
      Mahasiswa_SZ: 1,
      Mahasiswa_TK: 1,
      Mahasiswa_TL: 1,
      Mahasiswa_TT: 1,
      Mahasiswa_TW: 1,
      Mahasiswa_TX: 1,
      Mahasiswa_TY: 1,
      Mahasiswa_TZ: 1,
      Mahasiswa_UK: 1,
      Mahasiswa_UL: 1,
      Mahasiswa_UT: 1,
      Mahasiswa_UW: 1,
      Mahasiswa_UX: 1,
      Mahasiswa_UY: 1,
      Mahasiswa_UZ: 1,
      Mahasiswa_VK: 1,
      Mahasiswa_VL: 1,
      Mahasiswa_VT: 1,
      Mahasiswa_VW: 1,
      Mahasiswa_VX: 1,
      Mahasiswa_VY: 1,
      Mahasiswa_VZ: 1,
      Mahasiswa_WK: 1,
      Mahasiswa_WL: 1,
      Mahasiswa_WT: 1,
      Mahasiswa_WW: 1,
      Mahasiswa_WX: 1,
      Mahasiswa_WY: 1,
      Mahasiswa_WZ: 1,
      Mahasiswa_XK: 1,
      Mahasiswa_XL: 1,
      Mahasiswa_XT: 1,
      Mahasiswa_XW: 1,
      Mahasiswa_XX: 1,
      Mahasiswa_XY: 1,
      Mahasiswa_XZ: 1,
      Mahasiswa_YK: 1,
      Mahasiswa_YL: 1,
      Mahasiswa_YT: 1,
      Mahasiswa_YW: 1,
      Mahasiswa_YX: 1,
      Mahasiswa_YY: 1,
      Mahasiswa_YZ: 1,
      Mahasiswa_ZK: 1,
      Mahasiswa_ZL: 1,
      Mahasiswa_ZT: 1,
      Mahasiswa_ZW: 1,
      Mahasiswa_ZX: 1,
      Mahasiswa_ZY: 1,
      Mahasiswa_ZZ: 1
    }
  ]
}

```

```

Mahasiswa_SK,NRP>Nama,IPK_terakhir,Program_Studi,Departemen,Jenis_kelamin,Fakultas,Angkatan,Semester_saat_ini,Ips_terakhir,Jenjang,Tanggal_masuk,NIP_Dosen,Effective_Da
2,1000001,Dr. Ratih Melani,2.75,S1 Sistem Informasi,Informatika,Perempuan,FTK,2022,8,2.75,S3,2023-01-07,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527
3,1000002,Rizki Siregar,3.9,S1 Teknik Mesin,Informatika,Laki-laki,FTEIC,2018,2,3.5,S1,2024-04-01,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
4,1000003,Adiarja Kuswandari,3.46,S1 Teknik Elektro,Sipil,Laki-laki,FTI,2020,4,2.79,S3,2023-07-23,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
5,1000004,Hj. Kani Sihombing,3.2,S1 Teknik Mesin,Sipil,Laki-laki,FTSPK,2018,6,3.66,S3,2022-07-28,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
6,1000005,H. Reksa Pranowo,2.31,S1 Teknik Elektro,Mesin,Perempuan,FTSPK,2020,5,3.14,S3,2020-11-24,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
7,1000006,Oman Jailani,2.31,S1 Teknik Elektro,Elektro,Perempuan,FTI,2021,7,2.13,S2,2021-11-28,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
8,1000007,H. Gaduh Puspita, S.H.,2.12,S1 Teknik Sipil,Sipil,Laki-laki,FTSPK,2019,2,2.07,S2,2024-01-30,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527
9,1000009,"Dr. Fitria Firmansyah, S.E.",3.2,S1 Teknik Mesin,Elektro,Perempuan,FTEIC,2022,5,2.03,S3,2020-09-26,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20
10,1000010,"Kezia Putra, S.Psi",3.42,S1 Teknik Elektro,Elektro,Laki-laki,FTEIC,2021,2,2.15,S2,2020-10-17,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_2025052
11,1000011,Diah Puspita,2.04,S1 Teknik Sipil,Informatika,Perempuan,FTI,2018,2,3.38,S2,2020-08-09,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
12,1000012,Bakiman Handayani,3.94,S1 Teknik Sipil,Sipil,Laki-laki,FTEIC,2022,2,3.07,S1,2022-12-10,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
13,1000013,drg. Salwa Pratama,3.66,S1 Teknik Mesin,Informatika,Perempuan,FTEIC,2018,7,3.5,S2,2023-06-02,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527
14,1000014,Ami Maulana,2.42,S1 Informatika,Elektro,Laki-laki,FTEIC,2020,6,3.83,S2,2022-02-09,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
15,1000015,Amelia Rajata,2.36,S1 Informatika,Kelautan,Perempuan,FTM,2018,7,3.17,S3,2020-03-03,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_INC
16,1000016,Raharja Budiyanto,2.37,S1 Teknik Elektro,Kelautan,Laki-laki,FTK,2018,3,3.45,S3,2023-06-07,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_IN
17,1000017,Diana Firmansyah,2.61,S1 Sistem Informasi,Mesin,Perempuan,FTSPK,2018,5,3.51,S3,2021-10-20,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_20250527_IN
18,1000018,R.A. Shania Widiastuti,3.05,S1 Teknik Elektro,Informatika,Perempuan,FTM,2021,6,2.76,S3,2019-11-28,,2025-05-27,9999-12-31,Y,1,2025-05-27,2025-05-27,BATCH_202

```

Source code :

```

const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}

// Function to fetch CSV data from URL
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })
}

```

```

    await csvWriter.writeRecords(data)
    console.log(`Output written to ${outputPath}`)
  }

// Function to get last ETL run timestamp
function getLastETLRunTimestamp() {
  return "2023-05-01T00:00:00Z"
}

// Function to update ETL run timestamp
function updateETLRunTimestamp() {
  const timestamp = new Date().toISOString()
  logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
  return timestamp
}

// Function to identify changed records for SCD processing
function identifyChangedDimMahasiswaRecords(sourceData, existingData) {
  logStep("CDC", "Identifying changed records for Dim_Mahasiswa")

  const newRecords = []
  const scdType1Updates = []
  const scdType2Updates = []
  const unchangedRecords = []

  sourceData.forEach((sourceRecord) => {
    // Find existing record by NRP (natural key)
    const existingRecord = existingData.find(
      (record) => record.NRP === Number.parseInt(sourceRecord.NRP) && record.Current_Flag
      === "Y",
    )

    if (!existingRecord) {
      // New record
      newRecords.push(sourceRecord)
    } else {
      // Check for changes in SCD Type 1 fields (overwrite)
      const type1Changes = {}
      let hasType1Changes = false

      // Nama - SCD Type 1
      if (sourceRecord>Nama !== existingRecord>Nama) {
        type1Changes>Nama = sourceRecord>Nama
        hasType1Changes = true
      }
    }
  })

```

```

}

// Fakultas - SCD Type 1
if (sourceRecord.Fakultas !== existingRecord.Fakultas) {
  type1Changes.Fakultas = sourceRecord.Fakultas
  hasType1Changes = true
}

// Angkatan - SCD Type 1
if (Number.parseInt(sourceRecord.Angkatan) !== existingRecord.Angkatan) {
  type1Changes.Angkatan = Number.parseInt(sourceRecord.Angkatan)
  hasType1Changes = true
}

// Semester_saat_ini - SCD Type 1
if (Number.parseInt(sourceRecord.Semester_saat_ini) !==
existingRecord.Semester_saat_ini) {
  type1Changes.Semester_saat_ini = Number.parseInt(sourceRecord.Semester_saat_ini)
  hasType1Changes = true
}

// Jenjang - SCD Type 1
if (sourceRecord.Jenjang !== existingRecord.Jenjang) {
  type1Changes.Jenjang = sourceRecord.Jenjang
  hasType1Changes = true
}

// Tanggal_masuk - SCD Type 1
const sourceTanggalMasuk = sourceRecord["Tanggal masuk"]
if (sourceTanggalMasuk !== existingRecord.Tanggal_masuk) {
  type1Changes.Tanggal_masuk = sourceTanggalMasuk
  hasType1Changes = true
}

// Check for changes in SCD Type 2 fields (track history)
const type2Changes = {}
let hasType2Changes = false

// IPK_terakhir - SCD Type 2
const sourceIPK = Number.parseFloat(sourceRecord.IPK_terakhir)
if (sourceIPK !== existingRecord.IPK_terakhir) {
  type2Changes.IPK_terakhir = sourceIPK
  hasType2Changes = true
}

```

```

// Program_Studi - SCD Type 2
if (sourceRecord.Program_Studi !== existingRecord.Program_Studi) {
  type2Changes.Program_Studi = sourceRecord.Program_Studi
  hasType2Changes = true
}

// Departemen - SCD Type 2
if (sourceRecord.Departemen !== existingRecord.Departemen) {
  type2Changes.Departemen = sourceRecord.Departemen
  hasType2Changes = true
}

// lps_terakhir - SCD Type 2
const sourceIPS = Number.parseFloat(sourceRecord.lps_terakhir)
if (sourceIPS !== existingRecord.lps_terakhir) {
  type2Changes.lps_terakhir = sourceIPS
  hasType2Changes = true
}

// NIP_Dosen - SCD Type 2
const sourceNIPDosen = Number.parseInt(sourceRecord.NIP_Dosen)
if (sourceNIPDosen !== existingRecord.NIP_Dosen) {
  type2Changes.NIP_Dosen = sourceNIPDosen
  hasType2Changes = true
}

if (hasType1Changes || hasType2Changes) {
  if (hasType1Changes && hasType2Changes) {
    // Both types of changes - handle as Type 2 (create new version) and update Type 1
    fields
    scdType2Updates.push({
      sourceRecord,
      existingRecord,
      type1Changes,
      type2Changes,
      changeType: "MIXED",
    })
  } else if (hasType1Changes) {
    // Only Type 1 changes - update in place
    scdType1Updates.push({
      sourceRecord,
      existingRecord,
      type1Changes,

```



```

        changeType: "TYPE1_ONLY",
    })
    } else {
        // Only Type 2 changes - create new version
        scdType2Updates.push({
            sourceRecord,
            existingRecord,
            type2Changes,
            changeType: "TYPE2_ONLY",
        })
    }
    } else {
        unchangedRecords.push(sourceRecord)
    }
}
})

return {
    newRecords,
    scdType1Updates,
    scdType2Updates,
    unchangedRecords,
}
}

// Transform new records
function transformNewDimMahasiswaRecords(newRecords, maxSK = 0) {
    return newRecords.map((record, index) => {
        const transformed = {}

        // Generate surrogate key
        transformed.Mahasiswa_SK = maxSK + index + 1

        // NRP (Natural Key) - SCD Type 0
        transformed.NRP = Number.parseInt(record.NRP, 10)

        // Nama - SCD Type 1
        transformed>Nama = record>Nama ? record>Nama.trim() : ""

        // IPK_terakhir - SCD Type 2
        transformed.IPK_terakhir = record.IPK_terakhir
            ? Number.parseFloat(Number.parseFloat(record.IPK_terakhir).toFixed(2))
            : null
    })
}

```

```

// Program_Studi - SCD Type 2
transformed.Program_Studi = record.Program_Studi ? record.Program_Studi.trim() : ""

// Departemen - SCD Type 2
transformed.Departemen = record.Departemen ? record.Departemen.trim() : ""

// Jenis_kelamin - SCD Type 0
if (record["Jenis kelamin"]) {
  const gender = record["Jenis kelamin"].toLowerCase().trim()
  if (gender.includes("laki")) {
    transformed.Jenis_kelamin = "Laki-laki"
  } else if (gender.includes("perempuan") || gender.includes("wanita")) {
    transformed.Jenis_kelamin = "Perempuan"
  } else {
    transformed.Jenis_kelamin = record["Jenis kelamin"]
  }
} else {
  transformed.Jenis_kelamin = ""
}

// Fakultas - SCD Type 1
transformed.Fakultas = record.Fakultas ? record.Fakultas.trim() : ""

// Angkatan - SCD Type 1
transformed.Angkatan = record.Angkatan ? Number.parseInt(record.Angkatan, 10) : null

// Semester_saat_ini - SCD Type 1
const semester = Number.parseInt(record.Semester_saat_ini, 10)
transformed.Semester_saat_ini = semester >= 1 && semester <= 14 ? semester : null

// Ips_terakhir - SCD Type 2
transformed.Ips_terakhir = record.Ips_terakhir
  ? Number.parseFloat(Number.parseFloat(record.Ips_terakhir).toFixed(2))
  : null

// Jenjang - SCD Type 1
if (record.Jenjang) {
  const jenjang = record.Jenjang.toUpperCase().trim()
  if (["S1", "S2", "S3"].includes(jenjang)) {
    transformed.Jenjang = jenjang
  } else {
    transformed.Jenjang = record.Jenjang
  }
} else {

```

```

    transformed.Jenjang = ""
}

// Tanggal_masuk - SCD Type 1
if (record["Tanggal masuk"]) {
    try {
        const dateParts = record["Tanggal masuk"].split("-")
        if (dateParts.length === 3) {
            const year = Number.parseInt(dateParts[0], 10)
            const month = Number.parseInt(dateParts[1], 10) - 1
            const day = Number.parseInt(dateParts[2], 10)
            const date = new Date(year, month, day)
            transformed.Tanggal_masuk = date.toISOString().split("T")[0]
        } else {
            transformed.Tanggal_masuk = null
        }
    } catch (e) {
        transformed.Tanggal_masuk = null
    }
} else {
    transformed.Tanggal_masuk = null
}

// NIP_Dosen - SCD Type 2
transformed.NIP_Dosen = record.NIP_Dosen ? Number.parseInt(record.NIP_Dosen, 10) :
null

// Add SCD Type 2 fields
transformed.Effective_Date = new Date().toISOString().split("T")[0]
transformed.End_Date = "9999-12-31"
transformed.Current_Flag = "Y"
transformed.Version_Number = 1

// Add audit fields
transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.Updated_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-]/g,
"").split("T")[0]}_INC`

    return transformed
})
}

// Process SCD Type 1 updates

```

```

function processSCDType1Updates(scdType1Updates) {
  const updatedRecords = []

  scdType1Updates.forEach(({ existingRecord, type1Changes }) => {
    const updatedRecord = { ...existingRecord }

    // Apply Type 1 changes (overwrite)
    Object.keys(type1Changes).forEach((field) => {
      updatedRecord[field] = type1Changes[field]
    })

    // Update audit fields
    updatedRecord.Updated_Date = new Date().toISOString().split("T")[0]
    updatedRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_INC`

    updatedRecords.push(updatedRecord)
  })

  return updatedRecords
}

// Process SCD Type 2 updates
function processSCDType2Updates(scdType2Updates, maxSK) {
  const expiredRecords = []
  const newVersionRecords = []
  let currentMaxSK = maxSK

  scdType2Updates.forEach(({ sourceRecord, existingRecord, type1Changes, type2Changes,
  changeType }) => {
    // Expire the current record
    const expiredRecord = { ...existingRecord }
    expiredRecord.End_Date = new Date().toISOString().split("T")[0]
    expiredRecord.Current_Flag = "N"
    expiredRecord.Updated_Date = new Date().toISOString().split("T")[0]
    expiredRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_INC`
    expiredRecords.push(expiredRecord)

    // Create new version
    const newVersion = { ...existingRecord }
    newVersion.Mahasiswa_SK = ++currentMaxSK

    // Apply Type 1 changes (if any)

```

```

    if (type1Changes) {
      Object.keys(type1Changes).forEach((field) => {
        newVersion[field] = type1Changes[field]
      })
    }

    // Apply Type 2 changes
    if (type2Changes) {
      Object.keys(type2Changes).forEach((field) => {
        newVersion[field] = type2Changes[field]
      })
    }

    // Update SCD Type 2 fields
    newVersion.Effective_Date = new Date().toISOString().split("T")[0]
    newVersion.End_Date = "9999-12-31"
    newVersion.Current_Flag = "Y"
    newVersion.Version_Number = existingRecord.Version_Number + 1

    // Update audit fields
    newVersion.Created_Date = new Date().toISOString().split("T")[0]
    newVersion.Updated_Date = new Date().toISOString().split("T")[0]
    newVersion.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_INC`

    newVersionRecords.push(newVersion)
  })

  return { expiredRecords, newVersionRecords }
}

// Main incremental ETL function for Dim_Mahasiswa
async function incrementalETLDimMahasiswa() {
  try {
    logStep("START", "Starting incremental ETL process for Dim_Mahasiswa")

    // Get last ETL run timestamp
    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Fetching source data from Dim_Mahasiswa CSV")
    const sourceData = await fetchCSVData(

```

```
    "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Mahasiswa-ODzn65IS2Wxhzkfevt19w7rnpbq1Cy.csv",
  )
  logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)
```

```
// Simulate existing warehouse data
```

```
const existingData = [
  {
    Mahasiswa_SK: 1,
    NRP: 1000008,
    Nama: "John Doe",
    IPK_terakhir: 3.5,
    Program_Studi: "Teknik Informatika",
    Departemen: "Informatika",
    Jenis_kelamin: "Laki-laki",
    Fakultas: "FTIF",
    Angkatan: 2020,
    Semester_saat_ini: 6,
    Ips_terakhir: 3.6,
    Jenjang: "S1",
    Tanggal_masuk: "2020-08-01",
    NIP_Dosen: 14265799,
    Effective_Date: "2023-05-01",
    End_Date: "9999-12-31",
    Current_Flag: "Y",
    Version_Number: 1,
    Created_Date: "2023-05-01",
    Updated_Date: "2023-05-01",
    ETL_Batch_ID: "BATCH_20230501_001",
  },
]
```

```
logStep("EXTRACTION", `Read ${existingData.length} existing records from data warehouse`)
```

```
// 2. CHANGE DATA CAPTURE
```

```
logStep("CDC", "Identifying changed records")
```

```
const { newRecords, scdType1Updates, scdType2Updates, unchangedRecords } =
identifyChangedDimMahasiswaRecords(
  sourceData,
  existingData,
)
```

```
logStep(
```

```
    "CDC",  
    `Identified ${newRecords.length} new records, ${scdType1Updates.length} Type 1 updates,  
    ${scdType2Updates.length} Type 2 updates, ${unchangedRecords.length} unchanged records`,  
    )
```

```
// 3. TRANSFORMATION
```

```
logStep("TRANSFORMATION", "Processing SCD changes")
```

```
// Get max SK from existing data
```

```
const maxSK = Math.max(...existingData.map((r) => r.Mahasiswa_SK), 0)
```

```
// Transform new records
```

```
const transformedNewRecords = transformNewDimMahasiswaRecords(newRecords,  
maxSK)
```

```
// Process SCD Type 1 updates
```

```
const type1UpdatedRecords = processSCDType1Updates(scdType1Updates)
```

```
// Process SCD Type 2 updates
```

```
const { expiredRecords, newVersionRecords } =  
processSCDType2Updates(scdType2Updates, maxSK + newRecords.length)
```

```
logStep("TRANSFORMATION", `Processed ${transformedNewRecords.length} new records`)  
logStep("TRANSFORMATION", `Processed ${type1UpdatedRecords.length} Type 1  
updates`)
```

```
logStep(  
    "TRANSFORMATION",  
    `Processed ${expiredRecords.length} expired records and ${newVersionRecords.length}  
new versions`,  
    )
```

```
// 4. LOADING
```

```
logStep("LOADING", "Applying changes to data warehouse")
```

```
if (transformedNewRecords.length > 0) {  
    logStep("LOADING", `Inserting ${transformedNewRecords.length} new records`)  
}
```

```
if (type1UpdatedRecords.length > 0) {  
    logStep("LOADING", `Updating ${type1UpdatedRecords.length} records with Type 1  
changes`)  
}
```

```
if (expiredRecords.length > 0) {
```

```

    logStep("LOADING", `Expiring ${expiredRecords.length} records for Type 2 changes`)
  }

  if (newVersionRecords.length > 0) {
    logStep("LOADING", `Inserting ${newVersionRecords.length} new versions for Type 2
changes`)
  }

  // 5. EXPORT TO CSV
  logStep("EXPORT", "Exporting incremental changes to CSV")

  if (transformedNewRecords.length > 0) {
    await exportToCSV(transformedNewRecords, "Dim_Mahasiswa_New_Records.csv")
  }

  if (type1UpdatedRecords.length > 0) {
    await exportToCSV(type1UpdatedRecords, "Dim_Mahasiswa_Type1_Updates.csv")
  }

  if (newVersionRecords.length > 0) {
    await exportToCSV(newVersionRecords, "Dim_Mahasiswa_Type2_New_Versions.csv")
  }

  // Update ETL run timestamp
  const newTimestamp = updateETLRunTimestamp()

  logStep("COMPLETE", "Incremental ETL process for Dim_Mahasiswa completed
successfully")

  return {
    status: "success",
    timestamp: newTimestamp,
    newRecords: transformedNewRecords.length,
    type1Updates: type1UpdatedRecords.length,
    type2Updates: newVersionRecords.length,
    expiredRecords: expiredRecords.length,
    unchangedRecords: unchangedRecords.length,
    sampleNewRecords: transformedNewRecords.slice(0, 2),
    sampleType1Updates: type1UpdatedRecords.slice(0, 2),
    sampleType2Updates: newVersionRecords.slice(0, 2),
  }
} catch (error) {
  logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
  console.error(error)
}

```



```

    return {
      status: "error",
      message: error.message,
    }
  }
}

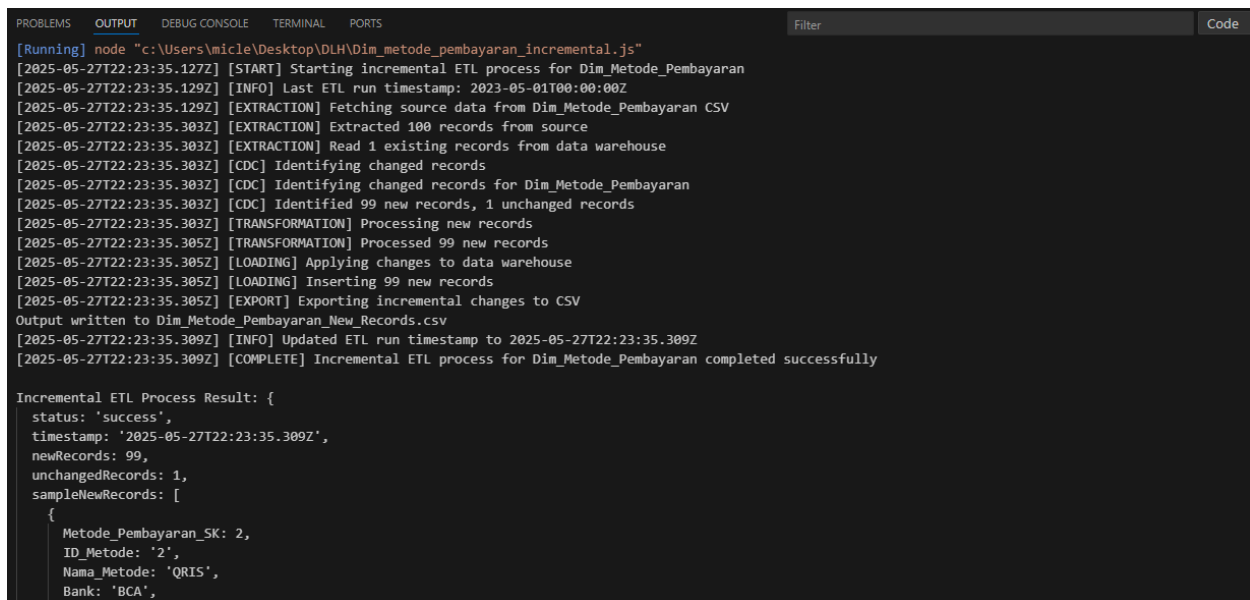
```

```

// Run the incremental ETL process using promise chaining
incrementalETLDimMahasiswa()
  .then((result) => {
    console.log("\nIncremental ETL Process Result:", result)
  })
  .catch((error) => {
    console.error("Incremental ETL Process Failed:", error)
  })

```

Dimensi Metode pembayaran



```

[Running] node "c:\Users\micle\Desktop\DLH\Dim_metode_pembayaran_incremental.js"
[2025-05-27T22:23:35.127Z] [START] Starting incremental ETL process for Dim_Metode_Pembayaran
[2025-05-27T22:23:35.129Z] [INFO] Last ETL run timestamp: 2023-05-01T00:00:00Z
[2025-05-27T22:23:35.129Z] [EXTRACTION] Fetching source data from Dim_Metode_Pembayaran CSV
[2025-05-27T22:23:35.303Z] [EXTRACTION] Extracted 100 records from source
[2025-05-27T22:23:35.303Z] [EXTRACTION] Read 1 existing records from data warehouse
[2025-05-27T22:23:35.303Z] [CDC] Identifying changed records
[2025-05-27T22:23:35.303Z] [CDC] Identifying changed records for Dim_Metode_Pembayaran
[2025-05-27T22:23:35.303Z] [CDC] Identified 99 new records, 1 unchanged records
[2025-05-27T22:23:35.303Z] [TRANSFORMATION] Processing new records
[2025-05-27T22:23:35.305Z] [TRANSFORMATION] Processed 99 new records
[2025-05-27T22:23:35.305Z] [LOADING] Applying changes to data warehouse
[2025-05-27T22:23:35.305Z] [LOADING] Inserting 99 new records
[2025-05-27T22:23:35.305Z] [EXPORT] Exporting incremental changes to CSV
Output written to Dim_Metode_Pembayaran_New_Records.csv
[2025-05-27T22:23:35.309Z] [INFO] Updated ETL run timestamp to 2025-05-27T22:23:35.309Z
[2025-05-27T22:23:35.309Z] [COMPLETE] Incremental ETL process for Dim_Metode_Pembayaran completed successfully

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-27T22:23:35.309Z',
  newRecords: 99,
  unchangedRecords: 1,
  sampleNewRecords: [
    {
      Metode_Pembayaran_SK: 2,
      ID_Metode: '2',
      Nama_Metode: 'QRIS',
      Bank: 'BCA',
    }
  ]
}

```

```
Metode_Pembayaran_SK_ID_Metode>Nama_Metode|Bank|Payment_Category|Is_Digital|Requires_Bank|Full_Description|Created_Date|ETL_Batch_ID
2,2,QRIS,BCA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_INC
3,3,Cash,MANDIRI,Cash_Payment,N,N,Cash,2025-05-27,BATCH_20250527_INC
4,4,QRIS,BCA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_INC
5,5,Transfer,GOPAY,Bank_Transfer,Y,Y,Transfer - GOPAY,2025-05-27,BATCH_20250527_INC
6,6,QRIS,BNI,Other,N,N,QRIS,2025-05-27,BATCH_20250527_INC
7,7,QRIS,DANA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_INC
8,8,Cash,MANDIRI,Cash_Payment,N,N,Cash,2025-05-27,BATCH_20250527_INC
9,9,Cash,BCA,Cash_Payment,N,N,Cash,2025-05-27,BATCH_20250527_INC
10,10,E-Wallet,GOPAY,Digital_Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_INC
11,11,Virtual_Account,OVO,Other,N,N,Virtual_Account,2025-05-27,BATCH_20250527_INC
12,12,E-Wallet,BRI,Digital_Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_INC
13,13,Cash,BNI,Cash_Payment,N,N,Cash,2025-05-27,BATCH_20250527_INC
14,14,E-Wallet,MANDIRI,Digital_Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_INC
15,15,QRIS,BCA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_INC
16,16,QRIS,OVO,Other,N,N,QRIS,2025-05-27,BATCH_20250527_INC
17,17,Cash,BCA,Cash_Payment,N,N,Cash,2025-05-27,BATCH_20250527_INC
18,18,Cash,GOPAY,Cash_Payment,N,N,Cash,2025-05-27,BATCH_20250527_INC
19,19,E-Wallet,MANDIRI,Digital_Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_INC
20,20,E-Wallet,GOPAY,Digital_Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_INC
21,21,QRIS,BCA,Other,N,N,QRIS,2025-05-27,BATCH_20250527_INC
22,22,Virtual_Account,MANDIRI,Other,N,N,Virtual_Account,2025-05-27,BATCH_20250527_INC
23,23,Virtual_Account,DANA,Other,N,N,Virtual_Account,2025-05-27,BATCH_20250527_INC
24,24,Cash,BCA,Cash_Payment,N,N,Cash,2025-05-27,BATCH_20250527_INC
25,25,E-Wallet,BRI,Digital_Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_INC
26,26,Virtual_Account,MANDIRI,Other,N,N,Virtual_Account,2025-05-27,BATCH_20250527_INC
27,27,E-Wallet,DANA,Digital_Wallet,Y,N,E-Wallet,2025-05-27,BATCH_20250527_INC
```

Source Code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}

// Function to fetch CSV data from URL
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
```

```

    path: outputPath,
    header: headers,
  })

  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}

// Function to get last ETL run timestamp
function getLastETLRunTimestamp() {
  return "2023-05-01T00:00:00Z"
}

// Function to update ETL run timestamp
function updateETLRunTimestamp() {
  const timestamp = new Date().toISOString()
  logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
  return timestamp
}

// Function to identify changed records for SCD processing
function identifyChangedDimMetodePembayaranRecords(sourceData, existingData) {
  logStep("CDC", "Identifying changed records for Dim_Metode_Pembayaran")

  const newRecords = []
  const unchangedRecords = []

  sourceData.forEach((sourceRecord) => {
    // Find existing record by ID_Metode (natural key)
    const existingRecord = existingData.find((record) => record.ID_Metode ===
sourceRecord.ID_Metode)

    if (!existingRecord) {
      // New record
      newRecords.push(sourceRecord)
    } else {
      // Since all fields are SCD Type 0 (never change), we only check for new records
      // In a real scenario, we might still want to update audit fields or derived attributes
      unchangedRecords.push(sourceRecord)
    }
  })

  return {
    newRecords,

```

```

    unchangedRecords,
  }
}

// Transform new records
function transformNewDimMetodePembayaranRecords(newRecords, maxSK = 0) {
  return newRecords.map((record, index) => {
    const transformed = {}

    // Generate surrogate key
    transformed.Metode_Pembayaran_SK = maxSK + index + 1

    // All fields are SCD Type 0 - they never change
    // ID_Metode (Natural Key) - SCD Type 0
    transformed.ID_Metode = record.ID_Metode

    // Nama_Metode - SCD Type 0
    if (record>Nama_Metode) {
      const metode = record>Nama_Metode.trim()
      // Standardize payment method names
      const metodeMap = {
        atm: "ATM",
        "m-banking": "M-Banking",
        "mobile banking": "M-Banking",
        "internet banking": "Internet Banking",
        teller: "Teller",
        "e-wallet": "E-Wallet",
        transfer: "Transfer",
        cash: "Cash",
        tunai: "Cash",
      }

      const normalizedMetode = metodeMap[metode.toLowerCase()] || metode
      transformed>Nama_Metode = normalizedMetode
    } else {
      transformed>Nama_Metode = ""
    }

    // Bank - SCD Type 0
    transformed.Bank = record.Bank ? record.Bank.trim().toUpperCase() : ""

    // Add derived attributes for better analytics
    if (transformed>Nama_Metode) {
      // Payment category

```

```

    if ([ "ATM", "M-Banking", "Internet Banking", "Teller" ].includes(transformed>Nama_Metode)) {
        transformed.Payment_Category = "Bank Services"
    } else if ([ "E-Wallet" ].includes(transformed>Nama_Metode)) {
        transformed.Payment_Category = "Digital Wallet"
    } else if ([ "Transfer" ].includes(transformed>Nama_Metode)) {
        transformed.Payment_Category = "Bank Transfer"
    } else if ([ "Cash" ].includes(transformed>Nama_Metode)) {
        transformed.Payment_Category = "Cash Payment"
    } else {
        transformed.Payment_Category = "Other"
    }

    // Digital vs Traditional
    if ([ "M-Banking", "Internet Banking", "E-Wallet",
"Transfer" ].includes(transformed>Nama_Metode)) {
        transformed.Is_Digital = "Y"
    } else {
        transformed.Is_Digital = "N"
    }

    // Requires bank
    if ([ "ATM", "M-Banking", "Internet Banking", "Teller",
"Transfer" ].includes(transformed>Nama_Metode)) {
        transformed.Requires_Bank = "Y"
    } else {
        transformed.Requires_Bank = "N"
    }
} else {
    transformed.Payment_Category = "Unknown"
    transformed.Is_Digital = "N"
    transformed.Requires_Bank = "N"
}

// Create full payment method description
if (transformed.Bank && transformed.Requires_Bank === "Y") {
    transformed.Full_Description = `${transformed>Nama_Metode} - ${transformed.Bank}`
} else {
    transformed.Full_Description = transformed>Nama_Metode
}

// Add audit fields (not SCD related, just for tracking)
transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-/]/g,
"").split("T")[0]}_INC`

```

```

    return transformed
  })
}

// Main incremental ETL function for Dim_Metode_Pembayaran
async function incrementalETLDimMetodePembayaran() {
  try {
    logStep("START", "Starting incremental ETL process for Dim_Metode_Pembayaran")

    // Get last ETL run timestamp
    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Fetching source data from Dim_Metode_Pembayaran CSV")
    const sourceData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Metode_Pembayaran%20%284%29-iH400cJAF6L8u66V6KzqiMq6xFPqY7.csv",
    )
    logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

    // Simulate existing warehouse data
    const existingData = [
      {
        Metode_Pembayaran_SK: 1,
        ID_Metode: "1",
        Nama_Metode: "ATM",
        Bank: "BCA",
        Payment_Category: "Bank Services",
        Is_Digital: "N",
        Requires_Bank: "Y",
        Full_Description: "ATM - BCA",
        Created_Date: "2023-05-01",
        ETL_Batch_ID: "BATCH_20230501_001",
      },
    ]

    logStep("EXTRACTION", `Read ${existingData.length} existing records from data warehouse`)

    // 2. CHANGE DATA CAPTURE
    logStep("CDC", "Identifying changed records")
  }
}

```

```

    const { newRecords, unchangedRecords } =
identifyChangedDimMetodePembayaranRecords(sourceData, existingData)

    logStep("CDC", `Identified ${newRecords.length} new records, ${unchangedRecords.length}
unchanged records`)

    // 3. TRANSFORMATION
    logStep("TRANSFORMATION", "Processing new records")

    // Get max SK from existing data
    const maxSK = Math.max(...existingData.map((r) => r.Metode_Pembayaran_SK), 0)

    // Transform new records
    const transformedNewRecords =
transformNewDimMetodePembayaranRecords(newRecords, maxSK)

    logStep("TRANSFORMATION", `Processed ${transformedNewRecords.length} new records`)

    // 4. LOADING
    logStep("LOADING", "Applying changes to data warehouse")

    if (transformedNewRecords.length > 0) {
        logStep("LOADING", `Inserting ${transformedNewRecords.length} new records`)
    }

    // 5. EXPORT TO CSV
    logStep("EXPORT", "Exporting incremental changes to CSV")

    if (transformedNewRecords.length > 0) {
        await exportToCSV(transformedNewRecords,
"Dim_Metode_Pembayaran_New_Records.csv")
    }

    // Update ETL run timestamp
    const newTimestamp = updateETLRunTimestamp()

    logStep("COMPLETE", "Incremental ETL process for Dim_Metode_Pembayaran completed
successfully")

    return {
        status: "success",
        timestamp: newTimestamp,
        newRecords: transformedNewRecords.length,
        unchangedRecords: unchangedRecords.length,
    }

```

```

        sampleNewRecords: transformedNewRecords.slice(0, 2),
    }
} catch (error) {
    logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
    console.error(error)
    return {
        status: "error",
        message: error.message,
    }
}
}
}

```

```

// Run the incremental ETL process using promise chaining
incrementalETLDimMetodePembayaran()
    .then((result) => {
        console.log("\nIncremental ETL Process Result:", result)
    })
    .catch((error) => {
        console.error("Incremental ETL Process Failed:", error)
    })

```


Dimensi status permohonan

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter
[Running] node "c:\Users\micle\Desktop\DLH\Dim_status_permohonan_incremental.js"
[2025-05-28T00:12:00.258Z] [START] Starting incremental ETL process for Dim_Status_Permohonan
[2025-05-28T00:12:00.260Z] [INFO] Last ETL run timestamp: 2023-05-01T00:00:00Z
[2025-05-28T00:12:00.260Z] [EXTRACTION] Fetching source data from Dim_Status_Permohonan CSV
[2025-05-28T00:12:01.055Z] [EXTRACTION] Extracted 100 records from source
[2025-05-28T00:12:01.055Z] [EXTRACTION] Read 1 existing records from data warehouse
[2025-05-28T00:12:01.055Z] [CDC] Identifying changed records
[2025-05-28T00:12:01.055Z] [CDC] Identifying changed records for Dim_Status_Permohonan
[2025-05-28T00:12:01.055Z] [CDC] Identified 99 new records, 1 Type 2 updates, 0 unchanged records
[2025-05-28T00:12:01.055Z] [TRANSFORMATION] Processing SCD changes
[2025-05-28T00:12:01.057Z] [TRANSFORMATION] Processed 99 new records
[2025-05-28T00:12:01.057Z] [TRANSFORMATION] Processed 1 expired records and 1 new versions
[2025-05-28T00:12:01.057Z] [LOADING] Applying changes to data warehouse
[2025-05-28T00:12:01.057Z] [LOADING] Inserting 99 new records
[2025-05-28T00:12:01.057Z] [LOADING] Expiring 1 records for Type 2 changes
[2025-05-28T00:12:01.057Z] [LOADING] Inserting 1 new versions for Type 2 changes
[2025-05-28T00:12:01.057Z] [EXPORT] Exporting incremental changes to CSV
Output written to Dim_Status_Permohonan_New_Records.csv
Output written to Dim_Status_Permohonan_Type2_New_Versions.csv
[2025-05-28T00:12:01.061Z] [INFO] Updated ETL run timestamp to 2025-05-28T00:12:01.061Z
[2025-05-28T00:12:01.061Z] [COMPLETE] Incremental ETL process for Dim_Status_Permohonan completed successfully

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-28T00:12:01.061Z',
  newRecords: 99,
  type2Updates: 1,
  expiredRecords: 1,
  unchangedRecords: 0,
  sampleNewRecords: [
    {
      Status_SK: 2,
      ID_Status: '1',
      Status: 'Ditolak',
      Status_Category: 'Rejected',
      Status_Priority: 5,
      Effective_Date: '2025-05-28',
      End_Date: '9999-12-31'
    }
  ]
}

DLH > Dim_Status_Permohonan_New_Records.csv
> transformFa Aa ab_* No results ↑ ↓ ≡

1 Status_SK,ID_Status,Status,Status_Category,Status_Priority,Effective_Date,End_Date,Current_Flag,Version_Number,Created_Date,Updated_Date,ETL_Batch_ID
2 2,1,Ditolak,Rejected,5,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
3 3,2,Diproses,In Progress,2,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
4 4,3,Menunggu Verifikasi,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
5 5,4,Diproses,In Progress,2,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
6 6,5,Diproses,In Progress,2,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
7 7,6,Diterima,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
8 8,7,Menunggu Verifikasi,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
9 9,8,Diterima,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
10 10,9,Ditolak,Rejected,5,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
11 11,10,Ditolak,Rejected,5,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
12 12,11,Menunggu Verifikasi,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
13 13,12,Selesai,Approved,4,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
14 14,13,Diproses,In Progress,2,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
15 15,14,Diterima,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
16 16,15,Menunggu Verifikasi,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
17 17,16,Diterima,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
18 18,17,Diproses,In Progress,2,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
19 19,18,Selesai,Approved,4,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
20 20,19,Diterima,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
21 21,20,Menunggu Verifikasi,Other,99,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
22 22,21,Diproses,In Progress,2,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
23 23,22,Diproses,In Progress,2,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528_INC
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}
```

```

// Function to fetch CSV data from URL
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })

  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}

// Function to get last ETL run timestamp
function getLastETLRunTimestamp() {
  return "2023-05-01T00:00:00Z"
}

// Function to update ETL run timestamp
function updateETLRunTimestamp() {
  const timestamp = new Date().toISOString()
  logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
  return timestamp
}

// Function to standardize status values
function standardizeStatus(status) {
  if (!status) return ""

```

```

const statusTrim = status.trim()
const statusMap = {
  diproses: "Diproses",
  disetujui: "Disetujui",
  ditolak: "Ditolak",
  pending: "Pending",
  selesai: "Selesai",
  dibatalkan: "Dibatalkan",
}

return statusMap[statusTrim.toLowerCase()] || statusTrim
}

// Function to identify changed records for SCD processing
function identifyChangedDimStatusPermohonanRecords(sourceData, existingData) {
  logStep("CDC", "Identifying changed records for Dim_Status_Permohonan")

  const newRecords = []
  const scdType2Updates = []
  const unchangedRecords = []

  sourceData.forEach((sourceRecord) => {
    // Find existing record by ID_Status (natural key)
    const existingRecord = existingData.find(
      (record) => record.ID_Status === sourceRecord.ID_Status && record.Current_Flag === "Y",
    )

    if (!existingRecord) {
      // New record
      newRecords.push(sourceRecord)
    } else {
      // Check for changes in SCD Type 2 fields (track history)
      const type2Changes = {}
      let hasType2Changes = false

      // Status - SCD Type 2
      // Standardize status values first
      const sourceStatus = standardizeStatus(sourceRecord.Status)
      if (sourceStatus !== existingRecord.Status) {
        type2Changes.Status = sourceStatus
        hasType2Changes = true
      }
    }
  })
}

```

```

    if (hasType2Changes) {
      scdType2Updates.push({
        sourceRecord,
        existingRecord,
        type2Changes,
        changeType: "TYPE2_ONLY",
      })
    } else {
      unchangedRecords.push(sourceRecord)
    }
  }
})

return {
  newRecords,
  scdType2Updates,
  unchangedRecords,
}
}

// Transform new records
function transformNewDimStatusPermohonanRecords(newRecords, maxSK = 0) {
  return newRecords.map((record, index) => {
    const transformed = {}

    // Generate surrogate key
    transformed.Status_SK = maxSK + index + 1

    // ID_Status (Natural Key) - SCD Type 0
    transformed.ID_Status = record.ID_Status

    // Status - SCD Type 2
    transformed.Status = standardizeStatus(record.Status)

    // Add derived attributes for better analytics
    if (transformed.Status) {
      // Status category
      if (["Disetujui", "Selesai"].includes(transformed.Status)) {
        transformed.Status_Category = "Approved"
      } else if (["Ditolak", "Dibatalakan"].includes(transformed.Status)) {
        transformed.Status_Category = "Rejected"
      } else if (["Diproses", "Pending"].includes(transformed.Status)) {
        transformed.Status_Category = "In Progress"
      } else {

```

```

    transformed.Status_Category = "Other"
  }

  // Status priority (for sorting/filtering)
  const priorityMap = {
    Pending: 1,
    Diproses: 2,
    Disetujui: 3,
    Selesai: 4,
    Ditolak: 5,
    Dibatalkan: 6,
  }
  transformed.Status_Priority = priorityMap[transformed.Status] || 99
} else {
  transformed.Status_Category = "Unknown"
  transformed.Status_Priority = 99
}

// Add SCD Type 2 fields
transformed.Effective_Date = new Date().toISOString().split("T")[0]
transformed.End_Date = "9999-12-31"
transformed.Current_Flag = "Y"
transformed.Version_Number = 1

// Add audit fields
transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.Updated_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
""),split("T")[0]}_INC`

  return transformed
})
}

// Process SCD Type 2 updates
function processSCDType2Updates(scdType2Updates, maxSK) {
  const expiredRecords = []
  const newVersionRecords = []
  let currentMaxSK = maxSK

  scdType2Updates.forEach(({ sourceRecord, existingRecord, type2Changes }) => {
    // Expire the current record
    const expiredRecord = { ...existingRecord }
    expiredRecord.End_Date = new Date().toISOString().split("T")[0]

```

```

expiredRecord.Current_Flag = "N"
expiredRecord.Updated_Date = new Date().toISOString().split("T")[0]
expiredRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
""),split("T")[0]}_INC`
expiredRecords.push(expiredRecord)

// Create new version
const newVersion = { ...existingRecord }
newVersion.Status_SK = ++currentMaxSK

// Apply Type 2 changes
Object.keys(type2Changes).forEach((field) => {
  newVersion[field] = type2Changes[field]
})

// Update derived attributes
if (newVersion.Status) {
  // Status category
  if (["Disetujui", "Selesai"].includes(newVersion.Status)) {
    newVersion.Status_Category = "Approved"
  } else if (["Ditolak", "Dibatalakan"].includes(newVersion.Status)) {
    newVersion.Status_Category = "Rejected"
  } else if (["Diproses", "Pending"].includes(newVersion.Status)) {
    newVersion.Status_Category = "In Progress"
  } else {
    newVersion.Status_Category = "Other"
  }
}

// Status priority
const priorityMap = {
  Pending: 1,
  Diproses: 2,
  Disetujui: 3,
  Selesai: 4,
  Ditolak: 5,
  Dibatalakan: 6,
}
newVersion.Status_Priority = priorityMap[newVersion.Status] || 99
}

// Update SCD Type 2 fields
newVersion.Effective_Date = new Date().toISOString().split("T")[0]
newVersion.End_Date = "9999-12-31"
newVersion.Current_Flag = "Y"

```

```

newVersion.Version_Number = existingRecord.Version_Number + 1

// Update audit fields
newVersion.Created_Date = new Date().toISOString().split("T")[0]
newVersion.Updated_Date = new Date().toISOString().split("T")[0]
newVersion.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
"").split("T")[0]}_INC`

    newVersionRecords.push(newVersion)
  })

  return { expiredRecords, newVersionRecords }
}

// Main incremental ETL function for Dim_Status_Permohonan
async function incrementalETLDimStatusPermohonan() {
  try {
    logStep("START", "Starting incremental ETL process for Dim_Status_Permohonan")

    // Get last ETL run timestamp
    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Fetching source data from Dim_Status_Permohonan CSV")
    const sourceData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-
storage.com/Dim_Status_Permohonan%20%282%29-
0qt4qqLLrWGLvSlqj8TOegy2QdTK3z.csv",
    )
    logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

    // Simulate existing warehouse data
    const existingData = [
      {
        Status_SK: 1,
        ID_Status: "82",
        Status: "Diproses",
        Status_Category: "In Progress",
        Status_Priority: 2,
        Effective_Date: "2023-05-01",
        End_Date: "9999-12-31",
        Current_Flag: "Y",
        Version_Number: 1,
      },
    ],

```

```

    Created_Date: "2023-05-01",
    Updated_Date: "2023-05-01",
    ETL_Batch_ID: "BATCH_20230501_001",
  },
]

```

```

logStep("EXTRACTION", `Read ${existingData.length} existing records from data
warehouse`)

```

```

// 2. CHANGE DATA CAPTURE
logStep("CDC", "Identifying changed records")
const { newRecords, scdType2Updates, unchangedRecords } =
identifyChangedDimStatusPermohonanRecords(
  sourceData,
  existingData,
)

```

```

logStep(
  "CDC",
  `Identified ${newRecords.length} new records, ${scdType2Updates.length} Type 2 updates,
  ${unchangedRecords.length} unchanged records`,
)

```

```

// 3. TRANSFORMATION
logStep("TRANSFORMATION", "Processing SCD changes")

```

```

// Get max SK from existing data
const maxSK = Math.max(...existingData.map((r) => r.Status_SK), 0)

// Transform new records
const transformedNewRecords = transformNewDimStatusPermohonanRecords(newRecords,
maxSK)

```

```

// Process SCD Type 2 updates
const { expiredRecords, newVersionRecords } =
processSCDType2Updates(scdType2Updates, maxSK + newRecords.length)

```

```

logStep("TRANSFORMATION", `Processed ${transformedNewRecords.length} new records`)
logStep(
  "TRANSFORMATION",
  `Processed ${expiredRecords.length} expired records and ${newVersionRecords.length}
new versions`,
)

```



```

// 4. LOADING
logStep("LOADING", "Applying changes to data warehouse")

if (transformedNewRecords.length > 0) {
  logStep("LOADING", `Inserting ${transformedNewRecords.length} new records`)
}

if (expiredRecords.length > 0) {
  logStep("LOADING", `Expiring ${expiredRecords.length} records for Type 2 changes`)
}

if (newVersionRecords.length > 0) {
  logStep("LOADING", `Inserting ${newVersionRecords.length} new versions for Type 2
changes`)
}

// 5. EXPORT TO CSV
logStep("EXPORT", "Exporting incremental changes to CSV")

if (transformedNewRecords.length > 0) {
  await exportToCSV(transformedNewRecords,
"Dim_Status_Permohonan_New_Records.csv")
}

if (newVersionRecords.length > 0) {
  await exportToCSV(newVersionRecords,
"Dim_Status_Permohonan_Type2_New_Versions.csv")
}

// Update ETL run timestamp
const newTimestamp = updateETLRunTimestamp()

logStep("COMPLETE", "Incremental ETL process for Dim_Status_Permohonan completed
successfully")

return {
  status: "success",
  timestamp: newTimestamp,
  newRecords: transformedNewRecords.length,
  type2Updates: newVersionRecords.length,
  expiredRecords: expiredRecords.length,
  unchangedRecords: unchangedRecords.length,
  sampleNewRecords: transformedNewRecords.slice(0, 2),
  sampleType2Updates: newVersionRecords.slice(0, 2),
}

```

```

    }
  } catch (error) {
    logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
    console.error(error)
    return {
      status: "error",
      message: error.message,
    }
  }
}

```

```

// Run the incremental ETL process using promise chaining
incrementalETLDimStatusPermohonan()
  .then((result) => {
    console.log("\nIncremental ETL Process Result:", result)
  })
  .catch((error) => {
    console.error("Incremental ETL Process Failed:", error)
  })

```

Dimensi kelas_MK

```

[2025-05-28T00:14:15.068Z] [EXTRACTION] Read 1 existing records from data warehouse
[2025-05-28T00:14:15.068Z] [CDC] Identifying changed records
[2025-05-28T00:14:15.068Z] [CDC] Identifying changed records for Dim_Kelas_MK
[2025-05-28T00:14:15.069Z] [CDC] Identified 99 new records, 0 Type 1 updates, 1 Type 2 updates, 0 unchanged records
[2025-05-28T00:14:15.069Z] [TRANSFORMATION] Processing SCD changes
[2025-05-28T00:14:15.070Z] [TRANSFORMATION] Processed 99 new records
[2025-05-28T00:14:15.070Z] [TRANSFORMATION] Processed 0 Type 1 updates
[2025-05-28T00:14:15.070Z] [TRANSFORMATION] Processed 1 expired records and 1 new versions
[2025-05-28T00:14:15.070Z] [LOADING] Applying changes to data warehouse
[2025-05-28T00:14:15.070Z] [LOADING] Inserting 99 new records
[2025-05-28T00:14:15.070Z] [LOADING] Expiring 1 records for Type 2 changes
[2025-05-28T00:14:15.070Z] [LOADING] Inserting 1 new versions for Type 2 changes
[2025-05-28T00:14:15.070Z] [EXPORT] Exporting incremental changes to CSV
Output written to Dim_Kelas_MK_New_Records.csv
Output written to Dim_Kelas_MK_Type2_New_Versions.csv
[2025-05-28T00:14:15.077Z] [INFO] Updated ETL run timestamp to 2025-05-28T00:14:15.077Z
[2025-05-28T00:14:15.077Z] [COMPLETE] Incremental ETL process for Dim_Kelas_MK completed successfully

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-28T00:14:15.077Z',
  newRecords: 99,
  type1Updates: 0,
  type2Updates: 1,
  expiredRecords: 1,
  unchangedRecords: 0,
  sampleNewRecords: [
    {
      Kelas_MK_SK: 2,
      ID_kelas_MK: '1',
      Kode_mk: 'MK529',
      Kelas: 'B',
      Dosen_pengampu_1: 77827638,
      Dosen_pengampu_2: 28728463,
      Dosen_pengampu_3: 13561597,
      Nama_MK: 'Organized zero administration ability',
      SKS: 3,
    }
  ]
}

```

DLH > Dim_Kelas_MK_New_Records.csv

1	Kelas_MK_SK_ID kelas_MK_Kode_mk,Kelas,Dosen_pengampu_1,Dosen_pengampu_2,Dosen_pengampu_3>Nama_MK,SKS,Semester_n	transformFa	Aa	ab	*	No results	↑	↓	≡	×
2	1,MK529,B,77827638,28728463,13561597,Organized zero administration ability,3,Genap,R.514,Senin,14:56,20:17,30									
3	2,MK259,B,13999315,89254563,83197857,Multi-channelled incremental functionalities,3,Ganjil,R.147,Kamis,13:59,21:39,26,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28									
4	4,3,MK926,A,66629388,13561597,83197857,Exclusive zero tolerance hub,4,Genap,R.374,Senin,12:56,11:50,44,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
5	5,4,MK949,C,13356886,77827638,21668732,Focused background Infrastructure,4,Ganjil,R.460,Senin,05:59,21:17,22,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
6	6,5,MK975,B,23756669,21668732,13356886,Programmable hybrid Info-mediaries,2,Genap,R.242,Kamis,12:47,03:56,60,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
7	7,6,MK463,B,66629388,39958838,13561597,Future-proofed interactive moderator,4,Genap,R.459,Senin,00:57,04:15,58,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
8	8,7,MK267,D,39958838,77827638,23756669,Reduced tangible architecture,3,Genap,R.573,Jumat,01:50,17:40,34,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
9	9,8,MK941,A,89254563,24942603,23756669,Ameliorated asynchronous projection,3,Genap,R.237,Senin,01:03,06:12,33,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
10	10,9,MK771,D,90801586,89254563,28728463,Balanced analyzing throughput,3,Genap,R.173,Rabu,15:49,08:04,28,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
11	11,11,MK873,A,41227216,39345092,13356886,Open-architected human-resource utilization,2,Ganjil,R.421,Selasa,04:34,15:57,47,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
12	12,12,MK490,D,13561597,13356886,14265799,Intuitive directional infrastructure,4,Genap,R.597,Jumat,12:22,09:30,20,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
13	13,13,MK887,C,46913810,77827638,83197857,Upgradable foreground model,2,Genap,R.322,Selasa,07:47,12:30,49,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
14	14,14,MK880,B,95822412,83197857,41227216,Adaptive executive conglomeration,4,Ganjil,R.545,Rabu,03:57,17:01,60,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
15	15,15,MK482,B,41227216,28728463,42868828,Organized asymmetric complexity,4,Ganjil,R.406,Rabu,06:12,00:27,51,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
16	16,16,MK999,C,95822412,46913810,66629388,Visionary responsive infrastructure,2,Ganjil,R.223,Jumat,11:51,12:34,25,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
17	17,17,MK878,B,13356886,39345092,13561597,Horizontal maximized initiative,2,Genap,R.584,Jumat,18:14,14:14,30,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
18	18,18,MK316,B,83197857,41227216,13999315,Progressive content-based methodology,4,Genap,R.304,Rabu,05:04,21:58,48,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
19	19,19,MK353,B,41227216,22575562,46913810,Business-focused non-volatile strategy,2,Genap,R.110,Jumat,14:35,16:02,55,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									
20	20,20,MK107,A,23756669,90801586,13561597,Monitored actuating capability,4,Ganjil,R.217,Senin,05:01,22:53,22,2025-05-28,9999-12-31,Y,1,2025-05-28,2025-05-28,BATCH_20250528									

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}

// Function to fetch CSV data from URL
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })
}
```

```

    await csvWriter.writeRecords(data)
    console.log(`Output written to ${outputPath}`)
  }

  // Function to get last ETL run timestamp
  function getLastETLRunTimestamp() {
    return "2023-05-01T00:00:00Z"
  }

  // Function to update ETL run timestamp
  function updateETLRunTimestamp() {
    const timestamp = new Date().toISOString()
    logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
    return timestamp
  }

  // Function to identify changed records for SCD processing
  function identifyChangedDimKelasMKRecords(sourceData, existingData) {
    logStep("CDC", "Identifying changed records for Dim_Kelas_MK")

    const newRecords = []
    const scdType1Updates = []
    const scdType2Updates = []
    const unchangedRecords = []

    sourceData.forEach((sourceRecord) => {
      // Find existing record by ID_kelas_MK (natural key)
      const existingRecord = existingData.find(
        (record) => record.ID_kelas_MK === sourceRecord.ID_kelas_MK && record.Current_Flag
        === "Y",
      )

      if (!existingRecord) {
        // New record
        newRecords.push(sourceRecord)
      } else {
        // Check for changes in SCD Type 1 fields (overwrite)
        const type1Changes = {}
        let hasType1Changes = false

        // Kelas - SCD Type 1
        if (sourceRecord.Kelas !== existingRecord.Kelas) {
          type1Changes.Kelas = sourceRecord.Kelas

```

```

        hasType1Changes = true
    }

    // Dosen pengampu 1, 2, 3 - SCD Type 1
    if (Number.parseInt(sourceRecord["Dosen pengampu 1"]) !==
existingRecord.Dosen_pengampu_1) {
        type1Changes.Dosen_pengampu_1 = Number.parseInt(sourceRecord["Dosen pengampu
1"])
        hasType1Changes = true
    }

    if (Number.parseInt(sourceRecord["Dosen pengampu 2"]) !==
existingRecord.Dosen_pengampu_2) {
        type1Changes.Dosen_pengampu_2 = Number.parseInt(sourceRecord["Dosen pengampu
2"])
        hasType1Changes = true
    }

    if (Number.parseInt(sourceRecord["Dosen pengampu 3"]) !==
existingRecord.Dosen_pengampu_3) {
        type1Changes.Dosen_pengampu_3 = Number.parseInt(sourceRecord["Dosen pengampu
3"])
        hasType1Changes = true
    }

    // Ruangan - SCD Type 1
    if (sourceRecord.Ruangan !== existingRecord.Ruangan) {
        type1Changes.Ruangan = sourceRecord.Ruangan
        hasType1Changes = true
    }

    // Hari - SCD Type 1
    if (sourceRecord.Hari !== existingRecord.Hari) {
        type1Changes.Hari = sourceRecord.Hari
        hasType1Changes = true
    }

    // Jam_mulai - SCD Type 1
    if (sourceRecord.Jam_mulai !== existingRecord.Jam_mulai) {
        type1Changes.Jam_mulai = sourceRecord.Jam_mulai
        hasType1Changes = true
    }

    // Jam_selesai - SCD Type 1

```

```

    if (sourceRecord.Jam_selesai !== existingRecord.Jam_selesai) {
      type1Changes.Jam_selesai = sourceRecord.Jam_selesai
      hasType1Changes = true
    }

    // Check for changes in SCD Type 2 fields (track history)
    const type2Changes = {}
    let hasType2Changes = false

    // Nama_MK - SCD Type 2
    if (sourceRecord>Nama_MK !== existingRecord>Nama_MK) {
      type2Changes>Nama_MK = sourceRecord>Nama_MK
      hasType2Changes = true
    }

    // SKS - SCD Type 2
    if (Number.parseInt(sourceRecord.SKS) !== existingRecord.SKS) {
      type2Changes.SKS = Number.parseInt(sourceRecord.SKS)
      hasType2Changes = true
    }

    // Semester_mk - SCD Type 2
    if (sourceRecord.Semester_mk !== existingRecord.Semester_mk) {
      type2Changes.Semester_mk = sourceRecord.Semester_mk
      hasType2Changes = true
    }

    // Kapasitas - SCD Type 2
    if (Number.parseInt(sourceRecord.Kapasitas) !== existingRecord.Kapasitas) {
      type2Changes.Kapasitas = Number.parseInt(sourceRecord.Kapasitas)
      hasType2Changes = true
    }

    if (hasType1Changes || hasType2Changes) {
      if (hasType1Changes && hasType2Changes) {
        // Both types of changes - handle as Type 2 (create new version) and update Type 1
        fields
        scdType2Updates.push({
          sourceRecord,
          existingRecord,
          type1Changes,
          type2Changes,
          changeType: "MIXED",
        })
      }
    }

```

```

    } else if (hasType1Changes) {
        // Only Type 1 changes - update in place
        scdType1Updates.push({
            sourceRecord,
            existingRecord,
            type1Changes,
            changeType: "TYPE1_ONLY",
        })
    } else {
        // Only Type 2 changes - create new version
        scdType2Updates.push({
            sourceRecord,
            existingRecord,
            type2Changes,
            changeType: "TYPE2_ONLY",
        })
    }
    } else {
        unchangedRecords.push(sourceRecord)
    }
}
})

return {
    newRecords,
    scdType1Updates,
    scdType2Updates,
    unchangedRecords,
}
}

// Transform new records
function transformNewDimKelasMKRecords(newRecords, maxSK = 0) {
    return newRecords.map((record, index) => {
        const transformed = {}

        // Generate surrogate key
        transformed.Kelas_MK_SK = maxSK + index + 1

        // ID_kelas_MK (Natural Key) - SCD Type 0
        transformed.ID_kelas_MK = record.ID_kelas_MK

        // Kode_mk - SCD Type 0
        transformed.Kode_mk = record.Kode_mk ? record.Kode_mk.trim() : ""
    })
}

```

```

// Kelas - SCD Type 1
transformed.Kelas = record.Kelas ? record.Kelas.trim() : ""

// Dosen pengampu 1, 2, 3 - SCD Type 1
transformed.Dosen_pengampu_1 = record["Dosen pengampu 1"] ?
Number.parseInt(record["Dosen pengampu 1"], 10) : null
transformed.Dosen_pengampu_2 = record["Dosen pengampu 2"] ?
Number.parseInt(record["Dosen pengampu 2"], 10) : null
transformed.Dosen_pengampu_3 = record["Dosen pengampu 3"] ?
Number.parseInt(record["Dosen pengampu 3"], 10) : null

// Nama_MK - SCD Type 2
transformed>Nama_MK = record>Nama_MK ? record>Nama_MK.trim() : ""

// SKS - SCD Type 2
transformed.SKS = record.SKS ? Number.parseInt(record.SKS, 10) : null

// Semester_mk - SCD Type 2
if (record.Semester_mk) {
  const semester = record.Semester_mk.toLowerCase().trim()
  if (semester.includes("ganjil")) {
    transformed.Semester_mk = "Ganjil"
  } else if (semester.includes("genap")) {
    transformed.Semester_mk = "Genap"
  } else {
    transformed.Semester_mk = record.Semester_mk.trim()
  }
} else {
  transformed.Semester_mk = ""
}

// Ruangan - SCD Type 1
transformed.Ruangan = record.Ruangan ? record.Ruangan.trim() : ""

// Hari - SCD Type 1
transformed.Hari = record.Hari ? record.Hari.charAt(0).toUpperCase() +
record.Hari.slice(1).toLowerCase() : ""

// Jam_mulai - SCD Type 1
transformed.Jam_mulai = record.Jam_mulai && record.Jam_mulai.match(/^\d{2}:\d{2}$/) ?
record.Jam_mulai : null

// Jam_selesai - SCD Type 1

```



```

    transformed.Jam_selesai =
      record.Jam_selesai && record.Jam_selesai.match(/^\d{2}:\d{2}$/) ? record.Jam_selesai :
    null

    // Kapasitas - SCD Type 2
    transformed.Kapasitas = record.Kapasitas ? Number.parseInt(record.Kapasitas, 10) : null

    // Add SCD Type 2 fields
    transformed.Effective_Date = new Date().toISOString().split("T")[0]
    transformed.End_Date = "9999-12-31"
    transformed.Current_Flag = "Y"
    transformed.Version_Number = 1

    // Add audit fields
    transformed.Created_Date = new Date().toISOString().split("T")[0]
    transformed.Updated_Date = new Date().toISOString().split("T")[0]
    transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_INC`

    return transformed
  })
}

// Process SCD Type 1 updates
function processSCDType1Updates(scdType1Updates) {
  const updatedRecords = []

  scdType1Updates.forEach(({ existingRecord, type1Changes }) => {
    const updatedRecord = { ...existingRecord }

    // Apply Type 1 changes (overwrite)
    Object.keys(type1Changes).forEach((field) => {
      updatedRecord[field] = type1Changes[field]
    })

    // Update audit fields
    updatedRecord.Updated_Date = new Date().toISOString().split("T")[0]
    updatedRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_INC`

    updatedRecords.push(updatedRecord)
  })

  return updatedRecords
}

```

```
}
```

```
// Process SCD Type 2 updates
```

```
function processSCDType2Updates(scdType2Updates, maxSK) {
```

```
  const expiredRecords = []
```

```
  const newVersionRecords = []
```

```
  let currentMaxSK = maxSK
```

```
  scdType2Updates.forEach(({ sourceRecord, existingRecord, type1Changes, type2Changes, changeType }) => {
```

```
    // Expire the current record
```

```
    const expiredRecord = { ...existingRecord }
```

```
    expiredRecord.End_Date = new Date().toISOString().split("T")[0]
```

```
    expiredRecord.Current_Flag = "N"
```

```
    expiredRecord.Updated_Date = new Date().toISOString().split("T")[0]
```

```
    expiredRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-]/g, '')}.split("T")[0]}_INC`
```

```
    expiredRecords.push(expiredRecord)
```

```
    // Create new version
```

```
    const newVersion = { ...existingRecord }
```

```
    newVersion.Kelas_MK_SK = ++currentMaxSK
```

```
    // Apply Type 1 changes (if any)
```

```
    if (type1Changes) {
```

```
      Object.keys(type1Changes).forEach((field) => {
```

```
        newVersion[field] = type1Changes[field]
```

```
      })
```

```
    }
```

```
    // Apply Type 2 changes
```

```
    if (type2Changes) {
```

```
      Object.keys(type2Changes).forEach((field) => {
```

```
        newVersion[field] = type2Changes[field]
```

```
      })
```

```
    }
```

```
    // Update SCD Type 2 fields
```

```
    newVersion.Effective_Date = new Date().toISOString().split("T")[0]
```

```
    newVersion.End_Date = "9999-12-31"
```

```
    newVersion.Current_Flag = "Y"
```

```
    newVersion.Version_Number = existingRecord.Version_Number + 1
```

```
    // Update audit fields
```

```

    newVersion.Created_Date = new Date().toISOString().split("T")[0]
    newVersion.Updated_Date = new Date().toISOString().split("T")[0]
    newVersion.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:]/g,
    "").split("T")[0]}_INC`

    newVersionRecords.push(newVersion)
  })

  return { expiredRecords, newVersionRecords }
}

// Main incremental ETL function for Dim_Kelas_MK
async function incrementalETLDimKelasMK() {
  try {
    logStep("START", "Starting incremental ETL process for Dim_Kelas_MK")

    // Get last ETL run timestamp
    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Fetching source data from Dim_Kelas_MK CSV")
    const sourceData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Kelas_MK%20%283%29-arEvtzQeQANqCuLfqX4BjP1cmM5vMm.csv",
    )
    logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

    // Simulate existing warehouse data
    const existingData = [
      {
        Kelas_MK_SK: 1,
        ID_kelas_MK: "10",
        Kode_mk: "IF2110",
        Kelas: "01",
        Dosen_pengampu_1: 14265799,
        Dosen_pengampu_2: null,
        Dosen_pengampu_3: null,
        Nama_MK: "Algoritma dan Struktur Data",
        SKS: 4,
        Semester_mk: "Ganjil",
        Ruangan: "7602",
        Hari: "Senin",
        Jam_mulai: "07:00",
      }
    ]
  } catch (error) {
    logStep("ERROR", `Error in incremental ETL process: ${error.message}`)
  }
}

```

```

    Jam_selesai: "09:30",
    Kapasitas: 40,
    Effective_Date: "2023-05-01",
    End_Date: "9999-12-31",
    Current_Flag: "Y",
    Version_Number: 1,
    Created_Date: "2023-05-01",
    Updated_Date: "2023-05-01",
    ETL_Batch_ID: "BATCH_20230501_001",
  },
]

```

```

logStep("EXTRACTION", `Read ${existingData.length} existing records from data
warehouse`)

```

```

// 2. CHANGE DATA CAPTURE

```

```

logStep("CDC", "Identifying changed records")
const { newRecords, scdType1Updates, scdType2Updates, unchangedRecords } =
identifyChangedDimKelasMKRecords(
  sourceData,
  existingData,
)

logStep(
  "CDC",
  `Identified ${newRecords.length} new records, ${scdType1Updates.length} Type 1 updates,
${scdType2Updates.length} Type 2 updates, ${unchangedRecords.length} unchanged records`,
)

```

```

// 3. TRANSFORMATION

```

```

logStep("TRANSFORMATION", "Processing SCD changes")

// Get max SK from existing data
const maxSK = Math.max(...existingData.map((r) => r.Kelas_MK_SK), 0)

// Transform new records
const transformedNewRecords = transformNewDimKelasMKRecords(newRecords, maxSK)

// Process SCD Type 1 updates
const type1UpdatedRecords = processSCDType1Updates(scdType1Updates)

// Process SCD Type 2 updates
const { expiredRecords, newVersionRecords } =
processSCDType2Updates(scdType2Updates, maxSK + newRecords.length)

```

```

logStep("TRANSFORMATION", `Processed ${transformedNewRecords.length} new records`)
logStep("TRANSFORMATION", `Processed ${type1UpdatedRecords.length} Type 1
updates`)
logStep(
  "TRANSFORMATION",
  `Processed ${expiredRecords.length} expired records and ${newVersionRecords.length}
new versions`,
)

```

// 4. LOADING

```

logStep("LOADING", "Applying changes to data warehouse")

if (transformedNewRecords.length > 0) {
  logStep("LOADING", `Inserting ${transformedNewRecords.length} new records`)
}

if (type1UpdatedRecords.length > 0) {
  logStep("LOADING", `Updating ${type1UpdatedRecords.length} records with Type 1
changes`)
}

if (expiredRecords.length > 0) {
  logStep("LOADING", `Expiring ${expiredRecords.length} records for Type 2 changes`)
}

if (newVersionRecords.length > 0) {
  logStep("LOADING", `Inserting ${newVersionRecords.length} new versions for Type 2
changes`)
}

```

// 5. EXPORT TO CSV

```

logStep("EXPORT", "Exporting incremental changes to CSV")

if (transformedNewRecords.length > 0) {
  await exportToCSV(transformedNewRecords, "Dim_Kelas_MK_New_Records.csv")
}

if (type1UpdatedRecords.length > 0) {
  await exportToCSV(type1UpdatedRecords, "Dim_Kelas_MK_Type1_Updates.csv")
}

if (newVersionRecords.length > 0) {
  await exportToCSV(newVersionRecords, "Dim_Kelas_MK_Type2_New_Versions.csv")
}

```

```

    }

    // Update ETL run timestamp
    const newTimestamp = updateETLRunTimestamp()

    logStep("COMPLETE", "Incremental ETL process for Dim_Kelas_MK completed successfully")

    return {
      status: "success",
      timestamp: newTimestamp,
      newRecords: transformedNewRecords.length,
      type1Updates: type1UpdatedRecords.length,
      type2Updates: newVersionRecords.length,
      expiredRecords: expiredRecords.length,
      unchangedRecords: unchangedRecords.length,
      sampleNewRecords: transformedNewRecords.slice(0, 2),
      sampleType1Updates: type1UpdatedRecords.slice(0, 2),
      sampleType2Updates: newVersionRecords.slice(0, 2),
    }
  } catch (error) {
    logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
    console.error(error)
    return {
      status: "error",
      message: error.message,
    }
  }
}

// Run the incremental ETL process using promise chaining
incrementalETLDimKelasMK()
  .then((result) => {
    console.log("\nIncremental ETL Process Result:", result)
  })
  .catch((error) => {
    console.error("Incremental ETL Process Failed:", error)
  })

```

Dimensi waktu

Fakta ambilFRS

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter

[2025-05-27T23:25:30.407Z] [CDC] Identified 100 new records, 0 updated records, and 0 unchanged records
[2025-05-27T23:25:30.407Z] [TRANSFORMATION] Starting data transformation for new and updated records
[2025-05-27T23:25:30.409Z] [TRANSFORMATION] Transformed 100 new records
[2025-05-27T23:25:30.409Z] [TRANSFORMATION] Transformed 0 updated records
[2025-05-27T23:25:30.409Z] [LOADING] Starting data loading
[2025-05-27T23:25:30.409Z] [LOADING] Inserting 100 new fact records
[2025-05-27T23:25:30.409Z] [EXPORT] Exporting incremental changes to CSV
Output written to F_ambilFRS_New_Records.csv
[2025-05-27T23:25:30.413Z] [INFO] Updated ETL run timestamp to 2025-05-27T23:25:30.413Z
[2025-05-27T23:25:30.413Z] [COMPLETE] Incremental ETL process for F_ambilFRS completed successfully

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-27T23:25:30.413Z',
  newRecords: 100,
  updatedRecords: 0,
  unchangedRecords: 0,
  sampleNewRecords: [
    {
      ambilFRS_SK: 2,
      Mahasiswa_SK: 1000004,
      Kelas_MK_SK: 19,
      Waktu_SK: 78,
      Mahasiswa_NK: 1000004,
      Kelas_MK_NK: '19',
      Waktu_NK: '78',
      SKS_diambil: 4,
      jumlah_matakuliah_diambil: 1
    }
  ]
}

> transformra Aa ab No results
ambilFRS_SK,Mahasiswa_SK,Kelas_MK_SK,Waktu_SK,Mahasiswa_NK,Kelas_MK_NK,Waktu_NK,SKS_diambil,Jumlah_matakuliah_diambil,Urutan_pengambilan_kelas,Jumlah_peserta_di_kelas
2,1000004,19,78,1000004,19,78,4,1,8,12,4,20,2025-01-23,5,5,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
3,1000009,31,100,1000009,31,100,4,1,6,33,10,22,2024-09-15,17,0,18.18,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
4,1000073,5,78,1000073,5,78,2,1,8,33,14,22,2025-04-15,20,2,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
5,1000057,98,10,1000057,98,10,4,1,3,43,16,22,2024-09-18,13,3,18.18,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
6,1000051,41,84,1000051,41,84,3,1,4,17,14,20,2024-08-09,21,5,15.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
7,1000095,24,64,1000095,24,64,4,1,7,45,6,20,2024-07-27,2,0,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
8,1000034,100,34,1000034,100,34,4,1,8,23,8,24,2024-08-04,9,0,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
9,1000037,89,63,1000037,89,63,2,1,2,18,18,20,2025-05-17,9,6,10.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
10,1000058,23,92,1000058,23,92,3,1,2,53,19,22,2025-01-24,16,5,13.64,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
11,1000086,45,91,1000086,45,91,2,1,9,44,13,22,2024-06-04,0,3,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
12,1000039,21,92,1000039,21,92,4,1,3,60,16,22,2025-03-31,14,1,18.18,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
13,1000066,29,16,1000066,29,16,2,1,3,25,19,22,2024-08-26,9,1,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
14,1000004,47,71,1000004,47,71,4,1,6,39,12,24,2024-09-15,0,1,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
15,1000017,79,12,1000017,79,12,2,1,5,35,19,24,2024-07-14,8,0,8.33,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
16,1000093,62,68,1000093,62,68,3,1,7,46,10,20,2025-02-25,6,3,15.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
17,1000017,41,83,1000017,41,83,2,1,8,39,16,24,2024-08-18,4,1,8.33,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
18,1000067,45,17,1000067,45,17,4,1,10,21,0,20,2025-04-01,13,2,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
19,1000056,65,8,1000056,65,8,2,1,9,19,13,22,2024-08-03,23,6,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
20,1000022,21,42,1000022,21,42,4,1,4,32,16,24,2025-05-03,13,6,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
21,1000037,11,33,1000037,11,33,2,1,9,27,11,20,2024-08-16,4,6,10.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
22,1000081,39,79,1000081,39,79,4,1,2,42,20,24,2024-11-15,15,5,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
23,1000022,76,75,1000022,76,75,2,1,3,52,21,24,2025-01-18,2,0,8.33,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
24,1000093,78,44,1000093,78,44,4,1,1,11,18,20,2024-12-27,11,5,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
25,1000006,83,99,1000006,83,99,4,1,5,51,10,20,2025-05-01,10,4,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
26,1000009,74,54,1000009,74,54,4,1,1,41,22,24,2024-05-26,15,0,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")
```

```
// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}
```

```
// Function to fetch CSV data from URL
```

```

async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })

  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}

// Function to get last ETL run timestamp
function getLastETLRunTimestamp() {
  return "2023-05-01T00:00:00Z"
}

// Function to update ETL run timestamp
function updateETLRunTimestamp() {
  const timestamp = new Date().toISOString()
  logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
  return timestamp
}

// Function to identify changed records for fact table
function identifyChangedFactAmbilFRSRecords(sourceData, existingData) {
  logStep("CDC", "Identifying changed records for F_ambilFRS")

  const newRecords = []

```



```

const updatedRecords = []
const unchangedRecords = []

sourceData.forEach((sourceRecord) => {
  // Business keys: Mahasiswa_SK + Kelas_MK_SK + Waktu_SK
  const existingRecord = existingData.find(
    (record) =>
      record.Mahasiswa_NK == sourceRecord.Mahasiswa_SK &&
      record.Kelas_MK_NK == sourceRecord.Kelas_MK_SK &&
      record.Waktu_NK == sourceRecord.Waktu_SK,
  )

  if (!existingRecord) {
    // New record
    newRecords.push(sourceRecord)
  } else {
    // Check if any measures have changed
    let hasChanged = false

    // Compare measures
    if (
      Number.parseInt(sourceRecord.SKS_diambil) !== existingRecord.SKS_diambil ||
      Number.parseInt(sourceRecord.Jumlah_matakuliah_diambil) !==
existingRecord.Jumlah_matakuliah_diambil ||
      Number.parseInt(sourceRecord.Urutan_pengambilan_kelas) !==
existingRecord.Urutan_pengambilan_kelas ||
      Number.parseInt(sourceRecord.Jumlah_peserta_di_kelas_saat_ini) !==
existingRecord.Jumlah_peserta_di_kelas_saat_ini ||
      Number.parseInt(sourceRecord.Sks_tersisa_saat_ini) !==
existingRecord.Sks_tersisa_saat_ini ||
      Number.parseInt(sourceRecord.Sks_maksimum_semester_ini) !==
existingRecord.Sks_maksimum_semester_ini
    ) {
      hasChanged = true
    }

    if (hasChanged) {
      updatedRecords.push({
        sourceRecord,
        existingRecord,
      })
    } else {
      unchangedRecords.push(sourceRecord)
    }
  }
})

```

```

    }
  })

  return {
    newRecords,
    updatedRecords,
    unchangedRecords,
  }
}

// Transform new fact records
function transformNewFactAmbilFRSRecords(data, startIndex = 0) {
  return data.map((record, index) => {
    const transformed = {}

    // Generate fact surrogate key
    transformed.ambilFRS_SK = startIndex + index + 1

    // Use surrogate keys directly (already in source)
    transformed.Mahasiswa_SK = Number.parseInt(record.Mahasiswa_SK) || null
    transformed.Kelas_MK_SK = Number.parseInt(record.Kelas_MK_SK) || null
    transformed.Waktu_SK = Number.parseInt(record.Waktu_SK) || null

    // Store original keys for CDC
    transformed.Mahasiswa_NK = Number.parseInt(record.Mahasiswa_SK)
    transformed.Kelas_MK_NK = record.Kelas_MK_SK
    transformed.Waktu_NK = record.Waktu_SK

    // Transform and validate measures
    transformed.SKS_diambil = validatePositiveInteger(record.SKS_diambil)
    transformed.Jumlah_matakuliah_diambil =
      validatePositiveInteger(record.Jumlah_matakuliah_diambil)
    transformed.Urutan_pengambilan_kelas =
      validatePositiveInteger(record.Urutan_pengambilan_kelas)
    transformed.Jumlah_peserta_di_kelas_saat_ini =
      validatePositiveInteger(record.Jumlah_peserta_di_kelas_saat_ini)
    transformed.Sks_tersisa_saat_ini = validateNonNegativeInteger(record.Sks_tersisa_saat_ini)
    transformed.Sks_maksimum_semester_ini =
      validatePositiveInteger(record.Sks_maksimum_semester_ini)

    // Transform Waktu_submit
    if (record.Waktu_submit) {
      try {
        const submitDate = new Date(record.Waktu_submit)

```

```

    transformed.Waktu_submit = submitDate.toISOString().split("T")[0]
    transformed.Waktu_submit_hour = submitDate.getHours()
    transformed.Waktu_submit_day_of_week = submitDate.getDay()
  } catch (e) {
    transformed.Waktu_submit = null
    transformed.Waktu_submit_hour = null
    transformed.Waktu_submit_day_of_week = null
  }
} else {
  transformed.Waktu_submit = null
  transformed.Waktu_submit_hour = null
  transformed.Waktu_submit_day_of_week = null
}

// Calculate derived measures
if (transformed.Sks_maksimum_semester_ini && transformed.SKS_diambil) {
  transformed.SKS_Utilization_Percentage = (
    (transformed.SKS_diambil / transformed.Sks_maksimum_semester_ini) *
    100
  ).toFixed(2)
} else {
  transformed.SKS_Utilization_Percentage = null
}

// Data quality checks
transformed.Data_Quality_Flag = "VALID"
const issues = []

if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Kelas_MK_SK) issues.push("MISSING_KELAS")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")

if (issues.length > 0) {
  transformed.Data_Quality_Flag = "WARNING"
  transformed.Data_Quality_Issues = issues.join(", ")
}

// Add audit fields
transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.Updated_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
"").split("T")[0]}_INC`

return transformed

```

```
    })  
  }
```

```
// Transform updated fact records
```

```
function transformUpdatedFactAmbilFRSRecords(updatedRecords) {
```

```
  const transformedRecords = []
```

```
  updatedRecords.forEach(({ sourceRecord, existingRecord }) => {
```

```
    // Update the existing record with new values
```

```
    const updatedFactRecord = { ...existingRecord }
```

```
    // Update measures
```

```
    updatedFactRecord.SKS_diambil = validatePositiveInteger(sourceRecord.SKS_diambil)
```

```
    updatedFactRecord.Jumlah_matakuliah_diambil =
```

```
    validatePositiveInteger(sourceRecord.Jumlah_matakuliah_diambil)
```

```
    updatedFactRecord.Urutan_pengambilan_kelas =
```

```
    validatePositiveInteger(sourceRecord.Urutan_pengambilan_kelas)
```

```
    updatedFactRecord.Jumlah_peserta_di_kelas_saat_ini = validatePositiveInteger(
```

```
      sourceRecord.Jumlah_peserta_di_kelas_saat_ini,
```

```
    )
```

```
    updatedFactRecord.Sks_tersisa_saat_ini =
```

```
    validateNonNegativeInteger(sourceRecord.Sks_tersisa_saat_ini)
```

```
    updatedFactRecord.Sks_maksimum_semester_ini =
```

```
    validatePositiveInteger(sourceRecord.Sks_maksimum_semester_ini)
```

```
    // Recalculate derived measures
```

```
    if (updatedFactRecord.Sks_maksimum_semester_ini && updatedFactRecord.SKS_diambil) {
```

```
      updatedFactRecord.SKS_Utilization_Percentage = (
```

```
        (updatedFactRecord.SKS_diambil / updatedFactRecord.Sks_maksimum_semester_ini) *  
        100
```

```
      ).toFixed(2)
```

```
    } else {
```

```
      updatedFactRecord.SKS_Utilization_Percentage = null
```

```
    }
```

```
    // Update audit fields
```

```
    updatedFactRecord.Updated_Date = new Date().toISOString().split("T")[0]
```

```
    updatedFactRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-]/g,  
    "").split("T")[0]}_INC`
```

```
    transformedRecords.push(updatedFactRecord)
```

```
  }
```

```
  return transformedRecords
```

```

}

// Validation helper functions
function validatePositiveInteger(value) {
  const num = Number.parseInt(value)
  return num > 0 ? num : null
}

function validateNonNegativeInteger(value) {
  const num = Number.parseInt(value)
  return num >= 0 ? num : null
}

// Main incremental ETL function for F_ambilFRS
async function incrementalETLFactAmbilFRS() {
  try {
    logStep("START", "Starting incremental ETL process for F_ambilFRS")

    // Get last ETL run timestamp
    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Starting extraction from ambilFRS CSV")
    const sourceData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/ambilFRS-KrhT9EsKk7QMhojwuHTURvjI9ih3s0.csv",
    )
    logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

    // Simulate existing fact data
    const existingFactData = [
      {
        ambilFRS_SK: 1,
        Mahasiswa_SK: 1,
        Kelas_MK_SK: 1,
        Waktu_SK: 1,
        Mahasiswa_NK: 1,
        Kelas_MK_NK: "1",
        Waktu_NK: "1",
        SKS_diambil: 4,
        Jumlah_matakuliah_diambil: 1,
        Urutan_pengambilan_kelas: 1,
        Jumlah_peserta_di_kelas_saat_ini: 35,
      },
    ]
  } catch (error) {
    logStep("ERROR", `Error in incrementalETLFactAmbilFRS: ${error}`)
  }
}

```

```

    Sks_tersisa_saat_ini: 18,
    Sks_maksimum_semester_ini: 22,
    Data_Quality_Flag: "VALID",
    Created_Date: "2023-05-01",
    Updated_Date: "2023-05-01",
    ETL_Batch_ID: "BATCH_20230501_001",
  },
]

```

```

logStep("EXTRACTION", `Read ${existingFactData.length} existing fact records from data
warehouse`)

```

```

// 2. CHANGE DATA CAPTURE
logStep("CDC", "Identifying changed records")
const { newRecords, updatedRecords, unchangedRecords } =
identifyChangedFactAmbilFRSRecords(
  sourceData,
  existingFactData,
)
logStep(
  "CDC",
  `Identified ${newRecords.length} new records, ${updatedRecords.length} updated records,
and ${unchangedRecords.length} unchanged records`,
)

```

```

// 3. TRANSFORMATION
logStep("TRANSFORMATION", "Starting data transformation for new and updated records")

// Transform new records
const transformedNewRecords = transformNewFactAmbilFRSRecords(newRecords,
existingFactData.length)
logStep("TRANSFORMATION", `Transformed ${transformedNewRecords.length} new
records`)

```

```

// Transform updated records
const transformedUpdatedRecords =
transformUpdatedFactAmbilFRSRecords(updatedRecords)
logStep("TRANSFORMATION", `Transformed ${transformedUpdatedRecords.length}
updated records`)

```

```

// 4. LOADING
logStep("LOADING", "Starting data loading")

if (transformedNewRecords.length > 0) {

```

```

    logStep("LOADING", `Inserting ${transformedNewRecords.length} new fact records`)
  }

  if (transformedUpdatedRecords.length > 0) {
    logStep("LOADING", `Updating ${transformedUpdatedRecords.length} existing fact
records`)
  }

  // 5. EXPORT TO CSV
  logStep("EXPORT", "Exporting incremental changes to CSV")

  if (transformedNewRecords.length > 0) {
    await exportToCSV(transformedNewRecords, "F_ambilFRS_New_Records.csv")
  }

  if (transformedUpdatedRecords.length > 0) {
    await exportToCSV(transformedUpdatedRecords, "F_ambilFRS_Updated_Records.csv")
  }

  // Update ETL run timestamp
  const newTimestamp = updateETLRunTimestamp()

  logStep("COMPLETE", "Incremental ETL process for F_ambilFRS completed successfully")

  return {
    status: "success",
    timestamp: newTimestamp,
    newRecords: transformedNewRecords.length,
    updatedRecords: transformedUpdatedRecords.length,
    unchangedRecords: unchangedRecords.length,
    sampleNewRecords: transformedNewRecords.slice(0, 2),
    sampleUpdatedRecords: transformedUpdatedRecords.slice(0, 2),
  }
} catch (error) {
  logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
  console.error(error)
  return {
    status: "error",
    message: error.message,
  }
}
}

// Run the incremental ETL process using promise chaining

```

incrementalETLFactAmbilFRS()

```
.then((result) => {
  console.log("\nIncremental ETL Process Result:", result)
})
.catch((error) => {
  console.error("Incremental ETL Process Failed:", error)
})
```

Fakta dropFRS

```
node "c:\Users\micle\Desktop\DLH\F_dropFRS_incremental.js"
[2025-05-27T23:27:56.024Z] [START] Starting Incremental ETL process for F_dropFRS
[2025-05-27T23:27:56.026Z] [INFO] Last ETL run timestamp: 2023-05-01T00:00:00Z
[2025-05-27T23:27:56.026Z] [EXTRACTION] Starting extraction from dropFRS CSV
[2025-05-27T23:27:56.256Z] [EXTRACTION] Extracted 100 records from source
[2025-05-27T23:27:56.256Z] [EXTRACTION] Loading dimension tables for lookups
[2025-05-27T23:27:56.967Z] [EXTRACTION] Read 1 existing fact records from data warehouse
[2025-05-27T23:27:56.967Z] [CDC] Identifying changed records
[2025-05-27T23:27:56.967Z] [CDC] Identifying changed records for F_dropFRS
[2025-05-27T23:27:56.968Z] [CDC] Identified 99 new records, 0 updated records, and 1 unchanged records
[2025-05-27T23:27:56.968Z] [TRANSFORMATION] Starting data transformation for new and updated records
[2025-05-27T23:27:56.969Z] [TRANSFORMATION] Transformed 99 new records
[2025-05-27T23:27:56.970Z] [TRANSFORMATION] Transformed 0 updated records
[2025-05-27T23:27:56.970Z] [LOADING] Starting data loading
[2025-05-27T23:27:56.970Z] [LOADING] Inserting 99 new fact records
[2025-05-27T23:27:56.970Z] [EXPORT] Exporting incremental changes to CSV
Output written to F_dropFRS_New_Records.csv
[2025-05-27T23:27:56.973Z] [INFO] Updated ETL run timestamp to 2025-05-27T23:27:56.973Z
[2025-05-27T23:27:56.973Z] [COMPLETE] Incremental ETL process for F_dropFRS completed successfully

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-27T23:27:56.973Z',
  newRecords: 99,
  updatedRecords: 0,
  unchangedRecords: 1,
  sampleNewRecords: [
    {
      dropFRS_SK: 2,
      Mahasiswa_SK: 23,
      Kelas_MK_SK: 26,
      Waktu_SK: 17,
      Status_SK: 70,
      Dosen_SK: 1,
      NRP_SK: 1,
      ID_Kelas_MK_SK: 26,
      ID_Waktu_SK: 17,
      ID_Status_SK: 70,
      NIP_Dosen_Wali_SK: 1,
      Jumlah_SKS: 1
    }
  ]
}
```

```
DLH > F_dropFRS_New_Records.csv
transformFa
Aa ab
1 dropFRS_SK,Mahasiswa_SK,Kelas_MK_SK,Waktu_SK,Status_SK,Dosen_SK,NRP_SK,ID_Kelas_MK_SK,ID_Waktu_SK,ID_Status_SK,NIP_Dosen_Wali_SK,Jumlah_SKS
2 2,23,26,17,70,1,1000023,26,17,70,66629388,2,22,20,7,6,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
3 3,62,38,96,44,4,1000062,38,96,44,46913810,2,24,22,6,5,8.33,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
4 4,97,29,87,93,13,1000097,29,87,93,14265799,2,22,20,7,6,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
5 5,2,34,69,16,15,1000002,34,69,16,22575562,4,22,18,8,7,18.18,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
6 6,87,34,75,49,12,1000087,34,75,49,66629388,4,18,14,6,5,22.22,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
7 7,61,4,80,72,11,1000061,4,80,72,89254563,4,20,16,6,5,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
8 8,82,60,90,39,14,1000082,60,90,39,13999315,2,18,16,6,5,11.11,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
9 9,5,39,64,15,4,1000005,39,64,15,46913810,4,20,16,6,5,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
10 10,50,59,48,86,18,1000050,59,48,86,77827638,4,24,20,8,7,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
11 11,94,20,54,84,4,1000094,20,54,84,46913810,4,24,20,7,6,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
12 12,36,5,89,48,7,1000036,5,89,48,28728463,3,24,21,6,5,12.50,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
13 13,47,13,88,48,18,1000047,13,88,48,77827638,2,22,20,7,6,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
14 14,36,25,16,59,3,1000036,25,16,59,13356886,4,20,16,8,7,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
15 15,77,3,7,43,8,1000077,3,7,43,23756669,4,20,16,6,5,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
16 16,9,98,71,27,19,1000009,98,71,27,90801586,2,20,18,7,6,10.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
17 17,100,19,77,1,9,1000100,19,77,1,83197857,2,20,18,8,7,10.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
18 18,33,23,15,85,1,1000033,23,15,85,95822412,2,20,18,7,6,10.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
19 19,31,76,42,3,6,1000031,76,42,3,39958838,2,22,20,6,5,9.09,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
20 20,95,54,68,15,3,1000095,54,68,15,13356886,3,24,21,7,6,12.50,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
21 21,66,76,14,58,17,1000066,76,14,58,41227216,4,20,16,6,5,20.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
22 22,94,85,67,39,15,1000094,85,67,39,22575562,2,18,16,7,6,11.11,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
23 23,52,55,88,14,16,1000052,55,88,14,39345092,2,24,22,6,5,8.33,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
24 24,42,78,19,9,5,1000042,78,19,9,42868828,4,22,18,8,7,18.18,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
25 25,75,71,92,42,13,1000075,71,92,42,14265799,3,22,19,8,7,13.64,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
26 26,78,56,13,90,4,1000078,56,13,90,46913810,3,20,17,7,6,15.00,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
27 27,30,53,44,59,13,1000030,53,44,59,14265799,4,24,20,6,5,16.67,VALID,2025-05-27,2025-05-27,BATCH_20250527_INC
```


Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}

// Function to fetch CSV data from URL
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
  })

  await csvWriter.writeRecords(data)
  console.log(`Output written to ${outputPath}`)
}

// Function to get last ETL run timestamp
function getLastETLRunTimestamp() {
  return "2023-05-01T00:00:00Z"
}

// Function to update ETL run timestamp
```

```

function updateETLRunTimestamp() {
  const timestamp = new Date().toISOString()
  logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
  return timestamp
}

// Function to identify changed records for fact table
function identifyChangedFactDropFRSRecords(sourceData, existingData) {
  logStep("CDC", "Identifying changed records for F_dropFRS")

  const newRecords = []
  const updatedRecords = []
  const unchangedRecords = []

  sourceData.forEach((sourceRecord) => {
    // For fact tables, we use business keys to identify records
    // Business keys: NRP_SK + ID_Kelas_MK_SK + ID_Waktu_SK
    const existingRecord = existingData.find(
      (record) =>
        record.NRP_NK == sourceRecord.NRP_SK &&
        record.ID_Kelas_MK_NK == sourceRecord.ID_Kelas_MK_SK &&
        record.ID_Waktu_NK == sourceRecord.ID_Waktu_SK,
    )

    if (!existingRecord) {
      // New record
      newRecords.push(sourceRecord)
    } else {
      // Check if any measures have changed
      let hasChanged = false

      // Compare measures
      if (
        Number.parseInt(sourceRecord.Jumlah_SKS_terdrop) !==
        existingRecord.Jumlah_SKS_terdrop ||
        Number.parseInt(sourceRecord.Jumlah_SKS_Before_drop) !==
        existingRecord.Jumlah_SKS_Before_drop ||
        Number.parseInt(sourceRecord.Jumlah_SKS_After_drop) !==
        existingRecord.Jumlah_SKS_After_drop ||
        Number.parseInt(sourceRecord.Jumlah_Mata_Kuliah_Terdaftar_Before_Drop) !==
        existingRecord.Jumlah_MK_Before_Drop ||
        Number.parseInt(sourceRecord.Jumlah_Mata_Kuliah_Terdaftar_After_Drop) !==
        existingRecord.Jumlah_MK_After_Drop ||
        sourceRecord.ID_Status_SK !== existingRecord.ID_Status_NK ||

```

```

        Number.parseInt(sourceRecord.NIP_Dosen_Wali_SK) !==
existingRecord.NIP_Dosen_Wali_NK
    ) {
        hasChanged = true
    }

    if (hasChanged) {
        updatedRecords.push({
            sourceRecord,
            existingRecord,
        })
    } else {
        unchangedRecords.push(sourceRecord)
    }
}
})

return {
    newRecords,
    updatedRecords,
    unchangedRecords,
}
}

```

// Transform new fact records

```

function transformNewFactDropFRSRecords(data, dimMahasiswa, dimKelas, dimWaktu,
dimStatus, dimDosen, startIndex = 0) {
    return data.map((record, index) => {
        const transformed = {}

```

// Generate fact surrogate key

```
transformed.dropFRS_SK = startIndex + index + 1
```

// Dimension lookups - using natural keys

```

const mahasiswa = dimMahasiswa.find((m) => m.NRP == record.NRP_SK)
transformed.Mahasiswa_SK = mahasiswa ? mahasiswa.Mahasiswa_SK || index + 1 : null

```

```

const kelas = dimKelas.find((k) => k.ID_kelas_MK == record.ID_Kelas_MK_SK)
transformed.Kelas_MK_SK = kelas ? kelas.Kelas_MK_SK || index + 1 : null

```

```

const waktu = dimWaktu.find((w) => w.ID_Waktu == record.ID_Waktu_SK)
transformed.Waktu_SK = waktu ? waktu.Waktu_SK || index + 1 : null

```

```
const status = dimStatus.find((s) => s.ID_Status == record.ID_Status_SK)
```

```

transformed.Status_SK = status ? status.Status_SK || index + 1 : null

const dosen = dimDosen.find((d) => d.NIP_Dosen == record.NIP_Dosen_Wali_SK)
transformed.Dosen_SK = dosen ? dosen.Dosen_SK || index + 1 : null

// Store original keys for CDC
transformed.NRP_NK = Number.parseInt(record.NRP_SK) || null
transformed.ID_Kelas_MK_NK = record.ID_Kelas_MK_SK
transformed.ID_Waktu_NK = record.ID_Waktu_SK
transformed.ID_Status_NK = record.ID_Status_SK
transformed.NIP_Dosen_Wali_NK = Number.parseInt(record.NIP_Dosen_Wali_SK) || null

// Validate and transform measures
transformed.Jumlah_SKS_terdrop = validatePositiveInteger(record.Jumlah_SKS_terdrop)
transformed.Jumlah_SKS_Before_drop =
validatePositiveInteger(record.Jumlah_SKS_Before_drop)
transformed.Jumlah_SKS_After_drop =
validateNonNegativeInteger(record.Jumlah_SKS_After_drop)
transformed.Jumlah_MK_Before_Drop =
validatePositiveInteger(record.Jumlah_Mata_Kuliah_Terdaftar_Before_Drop)
transformed.Jumlah_MK_After_Drop =
validateNonNegativeInteger(record.Jumlah_Mata_Kuliah_Terdaftar_After_Drop)

// Calculate derived measures
if (transformed.Jumlah_SKS_Before_drop && transformed.Jumlah_SKS_terdrop) {
  transformed.SKS_Drop_Percentage = (
    (transformed.Jumlah_SKS_terdrop / transformed.Jumlah_SKS_Before_drop) *
    100
  ).toFixed(2)
} else {
  transformed.SKS_Drop_Percentage = null
}

// Data quality checks
transformed.Data_Quality_Flag = "VALID"
const issues = []

if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Kelas_MK_SK) issues.push("MISSING_KELAS")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")

if (issues.length > 0) {
  transformed.Data_Quality_Flag = "WARNING"
  transformed.Data_Quality_Issues = issues.join(", ")
}

```

```

    }

    // Add audit fields
    transformed.Created_Date = new Date().toISOString().split("T")[0]
    transformed.Updated_Date = new Date().toISOString().split("T")[0]
    transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
    "").split("T")[0]}_INC`

    return transformed
  })
}

// Transform updated fact records
function transformUpdatedFactDropFRSRecords(updatedRecords, dimMahasiswa, dimKelas,
dimWaktu, dimStatus, dimDosen) {
  const transformedRecords = []

  updatedRecords.forEach(({ sourceRecord, existingRecord }) => {
    // Update the existing record with new values
    const updatedFactRecord = { ...existingRecord }

    // Update measures
    updatedFactRecord.Jumlah_SKS_terdrop =
    validatePositiveInteger(sourceRecord.Jumlah_SKS_terdrop)
    updatedFactRecord.Jumlah_SKS_Before_drop =
    validatePositiveInteger(sourceRecord.Jumlah_SKS_Before_drop)
    updatedFactRecord.Jumlah_SKS_After_drop =
    validateNonNegativeInteger(sourceRecord.Jumlah_SKS_After_drop)
    updatedFactRecord.Jumlah_MK_Before_Drop = validatePositiveInteger(
      sourceRecord.Jumlah_Mata_Kuliah_Terdaftar_Before_Drop,
    )
    updatedFactRecord.Jumlah_MK_After_Drop = validateNonNegativeInteger(
      sourceRecord.Jumlah_Mata_Kuliah_Terdaftar_After_Drop,
    )

    // Recalculate derived measures
    if (updatedFactRecord.Jumlah_SKS_Before_drop &&
    updatedFactRecord.Jumlah_SKS_terdrop) {
      updatedFactRecord.SKs_Drop_Percentage = (
        (updatedFactRecord.Jumlah_SKS_terdrop /
        updatedFactRecord.Jumlah_SKS_Before_drop) *
        100
      ).toFixed(2)
    } else {

```

```

    updatedFactRecord.SKS_Drop_Percentage = null
  }

  // Update audit fields
  updatedFactRecord.Updated_Date = new Date().toISOString().split("T")[0]
  updatedFactRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[:-]/g,
  "").split("T")[0]}_INC`

  transformedRecords.push(updatedFactRecord)
})

return transformedRecords
}

// Validation helper functions
function validatePositiveInteger(value) {
  const num = Number.parseInt(value)
  return num > 0 ? num : null
}

function validateNonNegativeInteger(value) {
  const num = Number.parseInt(value)
  return num >= 0 ? num : null
}

// Main incremental ETL function for F_dropFRS
async function incrementalETLFactDropFRS() {
  try {
    logStep("START", "Starting incremental ETL process for F_dropFRS")

    // Get last ETL run timestamp
    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Starting extraction from dropFRS CSV")
    const sourceData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/dropFRS-
      Uk32dMwpmKKU4TGfbqmoNic0sieJSB.csv",
    )
    logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

    // Load dimension data for lookups
    logStep("EXTRACTION", "Loading dimension tables for lookups")
  }
}

```

```

const dimMahasiswa = await fetchCSVData(
  "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Mahasiswa-
ODzn65IS2Wxhzkfevt19w7rnpbq1Cy.csv",
)
const dimKelas = await fetchCSVData(
  "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Kelas_MK%20%283%29-
arEvtzQeQANqCuLfqX4BjP1cmM5vMm.csv",
)
const dimWaktu = await fetchCSVData(
  "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Waktu%20%282%29-
bBN4wnuL2mryLiopDAgmhsKt8i90a2.csv",
)
const dimStatus = await fetchCSVData(
  "https://hebbkx1anhila5yf.public.blob.vercel-
storage.com/Dim_Status_Permohonan%20%282%29-
0qt4qqLLrWGLvSlqj8TOegy2QdTK3z.csv",
)
const dimDosen = await fetchCSVData(
  "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Dim_Dosen_Wali%20%283%29-
Kf57TxnnrnMot6hQbfzCPE2B1siPGB.csv",
)

```

```

// Add surrogate keys to dimension data
dimMahasiswa.forEach((record, index) => {
  record.Mahasiswa_SK = index + 1
})
dimKelas.forEach((record, index) => {
  record.Kelas_MK_SK = index + 1
})
dimWaktu.forEach((record, index) => {
  record.Waktu_SK = index + 1
})
dimStatus.forEach((record, index) => {
  record.Status_SK = index + 1
})
dimDosen.forEach((record, index) => {
  record.Dosen_SK = index + 1
})

```

```

// Simulate existing fact data
const existingFactData = [
  {
    dropFRS_SK: 1,
    Mahasiswa_SK: 1,

```

```

    Kelas_MK_SK: 1,
    Waktu_SK: 1,
    Status_SK: 1,
    Dosen_SK: 1,
    NRP_NK: 1000008,
    ID_Kelas_MK_NK: "10",
    ID_Waktu_NK: "86",
    ID_Status_NK: "82",
    NIP_Dosen_Wali_NK: 14265799,
    Jumlah_SKS_terdrop: 4,
    Jumlah_SKS_Before_drop: 22,
    Jumlah_SKS_After_drop: 18,
    Jumlah_MK_Before_Drop: 8,
    Jumlah_MK_After_Drop: 7,
    Data_Quality_Flag: "VALID",
    Created_Date: "2023-05-01",
    Updated_Date: "2023-05-01",
    ETL_Batch_ID: "BATCH_20230501_001",
  },
]

```

logStep("EXTRACTION", `Read \${existingFactData.length} existing fact records from data warehouse`)

```

// 2. CHANGE DATA CAPTURE
logStep("CDC", "Identifying changed records")
const { newRecords, updatedRecords, unchangedRecords } =
identifyChangedFactDropFRSRecords(
  sourceData,
  existingFactData,
)
logStep(
  "CDC",
  `Identified ${newRecords.length} new records, ${updatedRecords.length} updated records,
and ${unchangedRecords.length} unchanged records`,
)

```

```

// 3. TRANSFORMATION
logStep("TRANSFORMATION", "Starting data transformation for new and updated records")

```

```

// Transform new records
const transformedNewRecords = transformNewFactDropFRSRecords(
  newRecords,
  dimMahasiswa,
)

```



```

    dimKelas,
    dimWaktu,
    dimStatus,
    dimDosen,
    existingFactData.length,
  )
  logStep("TRANSFORMATION", `Transformed ${transformedNewRecords.length} new
records`)

  // Transform updated records
  const transformedUpdatedRecords = transformUpdatedFactDropFRSRecords(
    updatedRecords,
    dimMahasiswa,
    dimKelas,
    dimWaktu,
    dimStatus,
    dimDosen,
  )
  logStep("TRANSFORMATION", `Transformed ${transformedUpdatedRecords.length}
updated records`)

  // 4. LOADING
  logStep("LOADING", "Starting data loading")

  if (transformedNewRecords.length > 0) {
    logStep("LOADING", `Inserting ${transformedNewRecords.length} new fact records`)
  }

  if (transformedUpdatedRecords.length > 0) {
    logStep("LOADING", `Updating ${transformedUpdatedRecords.length} existing fact
records`)
  }

  // 5. EXPORT TO CSV
  logStep("EXPORT", "Exporting incremental changes to CSV")

  if (transformedNewRecords.length > 0) {
    await exportToCSV(transformedNewRecords, "F_dropFRS_New_Records.csv")
  }

  if (transformedUpdatedRecords.length > 0) {
    await exportToCSV(transformedUpdatedRecords, "F_dropFRS_Updated_Records.csv")
  }

```

```

// Update ETL run timestamp
const newTimestamp = updateETLRunTimestamp()

logStep("COMPLETE", "Incremental ETL process for F_dropFRS completed successfully")

return {
  status: "success",
  timestamp: newTimestamp,
  newRecords: transformedNewRecords.length,
  updatedRecords: transformedUpdatedRecords.length,
  unchangedRecords: unchangedRecords.length,
  sampleNewRecords: transformedNewRecords.slice(0, 2),
  sampleUpdatedRecords: transformedUpdatedRecords.slice(0, 2),
}
} catch (error) {
  logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
  console.error(error)
  return {
    status: "error",
    message: error.message,
  }
}
}

// Run the incremental ETL process using promise chaining
incrementalETLFactDropFRS()
  .then((result) => {
    console.log("\nIncremental ETL Process Result:", result)
  })
  .catch((error) => {
    console.error("Incremental ETL Process Failed:", error)
  })

```

Fakta setujuifRS

```
[Running] node "c:\Users\micle\Desktop\DLH\F_setujui_FRS_incremental.js"
[2025-05-28T00:19:02.762Z] [START] Starting incremental ETL process for F_setujui_FRS
[2025-05-28T00:19:02.764Z] [INFO] Last ETL run timestamp: 2023-05-01T00:00:00Z
[2025-05-28T00:19:02.764Z] [EXTRACTION] Starting extraction from setujuifRS CSV
[2025-05-28T00:19:03.720Z] [EXTRACTION] Extracted 100 records from source
[2025-05-28T00:19:03.721Z] [EXTRACTION] Read 1 existing fact records from data warehouse
[2025-05-28T00:19:03.721Z] [CDC] Identifying changed records
[2025-05-28T00:19:03.721Z] [CDC] Identifying changed records for F_setujui_FRS
[2025-05-28T00:19:03.721Z] [CDC] Identified 100 new records, 0 updated records, and 0 unchanged records
[2025-05-28T00:19:03.721Z] [TRANSFORMATION] Starting data transformation for new and updated records
[2025-05-28T00:19:03.722Z] [TRANSFORMATION] Transformed 100 new records
[2025-05-28T00:19:03.722Z] [TRANSFORMATION] Transformed 0 updated records
[2025-05-28T00:19:03.722Z] [LOADING] Starting data loading
[2025-05-28T00:19:03.722Z] [LOADING] Inserting 100 new fact records
[2025-05-28T00:19:03.722Z] [EXPORT] Exporting transformed data to CSV
Output written to F_setujui_FRS_New_Records.csv
[2025-05-28T00:19:03.726Z] [INFO] Updated ETL run timestamp to 2025-05-28T00:19:03.726Z
[2025-05-28T00:19:03.726Z] [COMPLETE] Incremental ETL process for F_setujui_FRS completed successfully
```

```
Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-28T00:19:03.726Z',
  newRecords: 100,
  updatedRecords: 0,
  unchangedRecords: 0,
  sampleNewRecords: [
    {
      setujuifRS_SK: 2,
      Mahasiswa_SK: 1000099,
      Waktu_SK: 100,
      Dosen_SK: 90801586,
      Status_SK: 1,
      Mahasiswa_NK: 1000099,
      Waktu_NK: '100',
      Dosen_Wali_NK: 90801586,
```

```
DLH > F_setujui_FRS_New_Records.csv
> transformFa  No results  ↑ ↓ ≡ ×
1 setujuifRS_SK,Mahasiswa_SK,Waktu_SK,Dosen_SK,Status_SK,Mahasiswa_NK,Waktu_NK,Dosen_Wali_NK,Status_Permohonan_NK,Jumlah_SKS_FRS,Jumlah_MK_FRS,Sks_tersisa_saat_ini,Sks_m
2 2,1000099,100,90801586,1,1000099,100,90801586,1,7,2,15,22,3.50,31.82,Light,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
3 3,1000052,35,13356886,2,1000052,35,13356886,2,24,8,,24,3.00,100.00,Overload,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
4 4,1000041,28,23756669,8,1000041,28,23756669,8,20,6,,20,3.33,100.00,Heavy,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
5 5,1000034,18,66629388,4,1000034,18,66629388,4,23,7,1,24,3.29,95.83,Overload,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
6 6,1000034,96,21668732,10,1000034,96,21668732,10,24,8,,22,3.00,109.09,Overload,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
7 7,1000047,29,13999315,3,1000047,29,13999315,3,21,7,,20,3.00,105.00,Heavy,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
8 8,1000097,7,41227216,2,1000097,7,41227216,2,11,3,13,24,3.67,45.83,Light,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
9 9,1000055,77,77827638,2,1000055,77,77827638,2,18,6,6,24,3.00,75.00,Normal,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
10 10,1000060,68,22575562,5,1000060,68,22575562,5,6,2,18,24,3.00,25.00,Light,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
11 11,1000093,15,24942603,9,1000093,15,24942603,9,14,4,10,24,3.50,58.33,Normal,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
12 12,1000044,15,46913810,5,1000044,15,46913810,5,11,3,11,22,3.67,50.00,Light,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
13 13,1000001,93,23756669,5,1000001,93,23756669,5,11,3,13,24,3.67,45.83,Light,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
14 14,1000014,81,89254563,6,1000014,81,89254563,6,22,7,2,24,3.14,91.67,Overload,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
15 15,1000026,20,95822412,6,1000026,20,95822412,6,11,3,13,24,3.67,45.83,Light,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
16 16,1000100,68,28728463,1,1000100,68,28728463,1,16,5,6,22,3.20,72.73,Normal,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")
```

// Function to log ETL steps

```
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}
```

// Function to fetch CSV data from URL

```
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
```

```

        skip_empty_lines: true,
        trim: true,
    })
} catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
}
}

// CSV Export function
async function exportToCSV(data, outputPath) {
    const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

    const csvWriter = createObjectCsvWriter({
        path: outputPath,
        header: headers,
    })

    await csvWriter.writeRecords(data)
    console.log(`Output written to ${outputPath}`)
}

// Function to get last ETL run timestamp
function getLastETLRunTimestamp() {
    return "2023-05-01T00:00:00Z"
}

// Function to update ETL run timestamp
function updateETLRunTimestamp() {
    const timestamp = new Date().toISOString()
    logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
    return timestamp
}

// Function to identify changed records for fact table
function identifyChangedFactSetujuiFRSRecords(sourceData, existingData) {
    logStep("CDC", "Identifying changed records for F_setujui_FRS")

    const newRecords = []
    const updatedRecords = []
    const unchangedRecords = []

    sourceData.forEach((sourceRecord) => {
        const existingRecord = existingData.find(
            (record) =>

```

```

        record.Mahasiswa_NK == sourceRecord.Mahasiswa_SK &&
        record.Waktu_NK == sourceRecord.Waktu_SK &&
        record.Dosen_Wali_NK == sourceRecord.Dosen_Wali_SK &&
        record.Status_Permohonan_NK == sourceRecord.Status_Permohonan_SK,
    )

    if (!existingRecord) {
        newRecords.push(sourceRecord)
    } else {
        let hasChanged = false

        if (
            Number.parseInt(sourceRecord.Jumlah_SKS_FRS) !== existingRecord.Jumlah_SKS_FRS ||
            Number.parseInt(sourceRecord.Jumlah_MK_FRS) !== existingRecord.Jumlah_MK_FRS ||
            Number.parseInt(sourceRecord.Sks_tersisa_saat_ini) !==
existingRecord.Sks_tersisa_saat_ini ||
            Number.parseInt(sourceRecord.Sks_maksimum_semester_ini) !==
existingRecord.Sks_maksimum_semester_ini
        ) {
            hasChanged = true
        }

        if (hasChanged) {
            updatedRecords.push(sourceRecord)
        } else {
            unchangedRecords.push(sourceRecord)
        }
    }
})

return {
    newRecords,
    updatedRecords,
    unchangedRecords,
}
}

// Transform new records
function transformNewFactSetujuiFRSRecords(data, startIndex = 0) {
    return data.map((record, index) => {
        const transformed = {}

        // Generate fact surrogate key

```

```

transformed.setujuiFRS_SK = startIndex + index + 1

// Use surrogate keys directly from fact table
transformed.Mahasiswa_SK = Number.parseInt(record.Mahasiswa_SK) || null
transformed.Waktu_SK = Number.parseInt(record.Waktu_SK) || null
transformed.Dosen_SK = Number.parseInt(record.Dosen_Wali_SK) || null
transformed.Status_SK = Number.parseInt(record.Status_Permohonan_SK) || null

// Store original business keys
transformed.Mahasiswa_NK = Number.parseInt(record.Mahasiswa_SK)
transformed.Waktu_NK = record.Waktu_SK
transformed.Dosen_Wali_NK = Number.parseInt(record.Dosen_Wali_SK)
transformed.Status_Permohonan_NK = record.Status_Permohonan_SK

// Transform measures
transformed.Jumlah_SKS_FRS = Number.parseInt(record.Jumlah_SKS_FRS) || null
transformed.Jumlah_MK_FRS = Number.parseInt(record.Jumlah_MK_FRS) || null
transformed.Sks_tersisa_saat_ini = Number.parseInt(record.Sks_tersisa_saat_ini) || null
transformed.Sks_maksimum_semester_ini =
Number.parseInt(record.Sks_maksimum_semester_ini) || null

// Calculate derived measures
if (transformed.Jumlah_MK_FRS && transformed.Jumlah_SKS_FRS) {
  transformed.Avg_SKS_per_MK = (transformed.Jumlah_SKS_FRS /
transformed.Jumlah_MK_FRS).toFixed(2)
} else {
  transformed.Avg_SKS_per_MK = null
}

// Calculate SKS utilization
if (transformed.Sks_maksimum_semester_ini && transformed.Jumlah_SKS_FRS) {
  transformed.SKS_Utilization_Percentage = (
    (transformed.Jumlah_SKS_FRS / transformed.Sks_maksimum_semester_ini) *
    100
  ).toFixed(2)
} else {
  transformed.SKS_Utilization_Percentage = null
}

// Add workload categorization
if (transformed.Jumlah_SKS_FRS) {
  if (transformed.Jumlah_SKS_FRS < 12) {
    transformed.Workload_Category = "Light"
  } else if (transformed.Jumlah_SKS_FRS <= 18) {

```

```

    transformed.Workload_Category = "Normal"
  } else if (transformed.Jumlah_SKS_FRS <= 21) {
    transformed.Workload_Category = "Heavy"
  } else {
    transformed.Workload_Category = "Overload"
  }
} else {
  transformed.Workload_Category = "Unknown"
}

// Data quality checks
transformed.Data_Quality_Flag = "VALID"
const issues = []

if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")
if (!transformed.Dosen_SK) issues.push("MISSING_DOSEN")
if (!transformed.Status_SK) issues.push("MISSING_STATUS")

if (transformed.Jumlah_SKS_FRS && transformed.Jumlah_SKS_FRS > 24) {
  issues.push("EXCESSIVE_SKS_APPROVED")
}

if (issues.length > 0) {
  transformed.Data_Quality_Flag = "WARNING"
  transformed.Data_Quality_Issues = issues.join(", ")
}

// Add audit fields
transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.Updated_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
"").split("T")[0]}_INC`

return transformed
})
}

// Transform updated records
function transformUpdatedFactSetujuiFRSRecords(updatedRecords, existingData) {
  const transformedRecords = []

  updatedRecords.forEach((updatedRecord) => {
    const existingRecord = existingData.find(

```

```

(record) =>
  record.Mahasiswa_NK == updatedRecord.Mahasiswa_SK &&
  record.Waktu_NK == updatedRecord.Waktu_SK &&
  record.Dosen_Wali_NK == updatedRecord.Dosen_Wali_SK &&
  record.Status_Permohonan_NK == updatedRecord.Status_Permohonan_SK,
)

if (existingRecord) {
  const updatedFactRecord = { ...existingRecord }

  // Update measures
  updatedFactRecord.Jumlah_SKS_FRS =
Number.parseInt(updatedRecord.Jumlah_SKS_FRS) || null
  updatedFactRecord.Jumlah_MK_FRS =
Number.parseInt(updatedRecord.Jumlah_MK_FRS) || null
  updatedFactRecord.Sks_tersisa_saat_ini =
Number.parseInt(updatedRecord.Sks_tersisa_saat_ini) || null
  updatedFactRecord.Sks_maksimum_semester_ini =
Number.parseInt(updatedRecord.Sks_maksimum_semester_ini) || null

  // Recalculate derived measures
  if (updatedFactRecord.Jumlah_MK_FRS && updatedFactRecord.Jumlah_SKS_FRS) {
    updatedFactRecord.Avg_SKS_per_MK = (updatedFactRecord.Jumlah_SKS_FRS /
updatedFactRecord.Jumlah_MK_FRS).toFixed(
      2,
    )
  } else {
    updatedFactRecord.Avg_SKS_per_MK = null
  }

  if (updatedFactRecord.Sks_maksimum_semester_ini &&
updatedFactRecord.Jumlah_SKS_FRS) {
    updatedFactRecord.SKS_Utilization_Percentage = (
      (updatedFactRecord.Jumlah_SKS_FRS /
updatedFactRecord.Sks_maksimum_semester_ini) *
      100
    ).toFixed(2)
  } else {
    updatedFactRecord.SKS_Utilization_Percentage = null
  }

  // Update workload categorization
  if (updatedFactRecord.Jumlah_SKS_FRS) {
    if (updatedFactRecord.Jumlah_SKS_FRS < 12) {

```



```

        updatedFactRecord.Workload_Category = "Light"
    } else if (updatedFactRecord.Jumlah_SKS_FRS <= 18) {
        updatedFactRecord.Workload_Category = "Normal"
    } else if (updatedFactRecord.Jumlah_SKS_FRS <= 21) {
        updatedFactRecord.Workload_Category = "Heavy"
    } else {
        updatedFactRecord.Workload_Category = "Overload"
    }
} else {
    updatedFactRecord.Workload_Category = "Unknown"
}

// Update audit fields
updatedFactRecord.Updated_Date = new Date().toISOString().split("T")[0]
updatedFactRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
""),split("T")[0]}_INC`

    transformedRecords.push(updatedFactRecord)
}
})

return transformedRecords
}

// Main incremental ETL function for F_setujui_FRS
async function incrementalETLFactSetujuiFRS() {
    try {
        logStep("START", "Starting incremental ETL process for F_setujui_FRS")

        const lastETLRunTimestamp = getLastETLRunTimestamp()
        logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

        // 1. EXTRACTION
        logStep("EXTRACTION", "Starting extraction from setujuiFRS CSV")
        const sourceData = await fetchCSVData(
            "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Setujui_FRS-9TTecs2gFzXYkazPgWdaUj9RiBUMrW.csv",
        )
        logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

        // Simulate existing fact data
        const existingFactData = [
            {
                setujuiFRS_SK: 1,

```

```

    Mahasiswa_SK: 1,
    Waktu_SK: 1,
    Dosen_SK: 1,
    Status_SK: 1,
    Mahasiswa_NK: 1,
    Waktu_NK: "1",
    Dosen_Wali_NK: 1,
    Status_Permohonan_NK: "1",
    Jumlah_SKS_FRS: 18,
    Jumlah_MK_FRS: 6,
    Sks_tersisa_saat_ini: 4,
    Sks_maksimum_semester_ini: 22,
    Avg_SKS_per_MK: "3.00",
    SKS_Utilization_Percentage: "81.82",
    Workload_Category: "Normal",
    Data_Quality_Flag: "VALID",
    Created_Date: "2023-05-01",
    Updated_Date: "2023-05-01",
    ETL_Batch_ID: "BATCH_20230501_001",
  },
]

```

```

logStep("EXTRACTION", `Read ${existingFactData.length} existing fact records from data
warehouse`)

```

```

// 2. CHANGE DATA CAPTURE

```

```

logStep("CDC", "Identifying changed records")
const { newRecords, updatedRecords, unchangedRecords } =
identifyChangedFactSetujuiFRSRecords(
  sourceData,
  existingFactData,
)
logStep(
  "CDC",
  `Identified ${newRecords.length} new records, ${updatedRecords.length} updated records,
and ${unchangedRecords.length} unchanged records`,
)

```

```

// 3. TRANSFORMATION

```

```

logStep("TRANSFORMATION", "Starting data transformation for new and updated records")

const transformedNewRecords = transformNewFactSetujuiFRSRecords(newRecords,
existingFactData.length)

```

```
logStep("TRANSFORMATION", `Transformed ${transformedNewRecords.length} new records`)
```

```
const transformedUpdatedRecords =  
transformUpdatedFactSetujuiFRSRecords(updatedRecords, existingFactData)  
logStep("TRANSFORMATION", `Transformed ${transformedUpdatedRecords.length} updated records`)
```

```
// 4. LOADING
```

```
logStep("LOADING", "Starting data loading")
```

```
if (transformedNewRecords.length > 0) {  
  logStep("LOADING", `Inserting ${transformedNewRecords.length} new fact records`)  
}
```

```
if (transformedUpdatedRecords.length > 0) {  
  logStep("LOADING", `Updating ${transformedUpdatedRecords.length} existing fact records`)  
}
```

```
// 5. EXPORT TO CSV
```

```
logStep("EXPORT", "Exporting transformed data to CSV")
```

```
if (transformedNewRecords.length > 0) {  
  await exportToCSV(transformedNewRecords, "F_setujui_FRS_New_Records.csv")  
}
```

```
if (transformedUpdatedRecords.length > 0) {  
  await exportToCSV(transformedUpdatedRecords, "F_setujui_FRS_Updated_Records.csv")  
}
```

```
const newTimestamp = updateETLRunTimestamp()
```

```
logStep("COMPLETE", "Incremental ETL process for F_setujui_FRS completed successfully")
```

```
return {  
  status: "success",  
  timestamp: newTimestamp,  
  newRecords: transformedNewRecords.length,  
  updatedRecords: transformedUpdatedRecords.length,  
  unchangedRecords: unchangedRecords.length,  
  sampleNewRecords: transformedNewRecords.slice(0, 2),  
  sampleUpdatedRecords: transformedUpdatedRecords.slice(0, 2),  
}
```

```

    }
  } catch (error) {
    logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
    console.error(error)
    return {
      status: "error",
      message: error.message,
    }
  }
}
}

```

```

// Run the incremental ETL process using promise chaining
incrementalETLFactSetujuiFRS()
  .then((result) => {
    console.log("\nIncremental ETL Process Result:", result)
  })
  .catch((error) => {
    console.error("Incremental ETL Process Failed:", error)
  })

```

Fakta bayarFRS

```

[Running] node "c:\Users\micle\Desktop\DLH\F_bayarFRS_incremental.js"
[2025-05-28T00:21:07.755Z] [START] Starting incremental ETL process for F_Bayar_FRS
[2025-05-28T00:21:07.757Z] [INFO] Last ETL run timestamp: 2023-05-01T00:00:00Z
[2025-05-28T00:21:07.757Z] [EXTRACTION] Starting extraction from bayarFRS CSV
[2025-05-28T00:21:08.641Z] [EXTRACTION] Extracted 100 records from source
[2025-05-28T00:21:08.641Z] [EXTRACTION] Read 1 existing fact records from data warehouse
[2025-05-28T00:21:08.641Z] [CDC] Identifying changed records
[2025-05-28T00:21:08.641Z] [CDC] Identifying changed records for F_Bayar_FRS
[2025-05-28T00:21:08.641Z] [CDC] Identified 100 new records, 0 updated records, and 0 unchanged records
[2025-05-28T00:21:08.641Z] [TRANSFORMATION] Starting data transformation for new and updated records
[2025-05-28T00:21:08.643Z] [TRANSFORMATION] Transformed 100 new records
[2025-05-28T00:21:08.643Z] [TRANSFORMATION] Transformed 0 updated records
[2025-05-28T00:21:08.643Z] [LOADING] Starting data loading
[2025-05-28T00:21:08.643Z] [LOADING] Inserting 100 new fact records
[2025-05-28T00:21:08.643Z] [EXPORT] Exporting transformed data to CSV
Output written to F_Bayar_FRS_New_Records.csv
[2025-05-28T00:21:08.646Z] [INFO] Updated ETL run timestamp to 2025-05-28T00:21:08.646Z
[2025-05-28T00:21:08.646Z] [COMPLETE] Incremental ETL process for F_Bayar_FRS completed successfully

Incremental ETL Process Result: {
  status: 'success',
  timestamp: '2025-05-28T00:21:08.646Z',
  newRecords: 100,
  updatedRecords: 0,
  unchangedRecords: 0,
  sampleNewRecords: [
    {
      BayarFRS_SK: 2,
      Mahasiswa_SK: 1000036,
      Waktu_SK: 60,
      Metode_Pembayaran_SK: 32,
      Mahasiswa_NK: 1000036,
      Waktu_NK: '60',
    }
  ]
}

```

```
DLH > F_Bayar_FRS_New_Records.csv
1 BayarFRS_SK,Mahasiswa_SK,Waktu_SK,Metode_Pembayaran_SK,Mahasiswa_NK,Waktu_NK,Metode_Pembayaran_NK,Nominal_bayar,Status_pembayaran,Is_Payment_Success,Payment_Category,D
2 2,1000036,60,32,1000036,60,32,500000,Failed,N,Low,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
3 3,1000071,13,7,1000071,13,7,2500000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
4 4,1000012,97,31,1000012,97,31,1000000,Failed,N,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
5 5,1000063,62,28,1000063,62,28,2000000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
6 6,1000022,49,1,1000022,49,1,2000000,Failed,N,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
7 7,1000059,37,55,1000059,37,55,2500000,Failed,N,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
8 8,1000020,25,38,1000020,25,38,1000000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
9 9,1000075,95,70,1000075,95,70,500000,Failed,N,Low,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
10 10,1000008,7,75,1000008,7,75,2000000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
11 11,1000008,66,11,1000008,66,11,1000000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
12 12,1000077,9,87,1000077,9,87,1000000,Failed,N,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
13 13,1000016,73,32,1000016,73,32,2500000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
14 14,1000000,11,54,1000000,11,54,2500000,Failed,N,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
15 15,1000034,27,86,1000034,27,86,1500000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
16 16,1000034,51,17,1000034,51,17,1500000,Failed,N,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
17 17,1000041,97,10,1000041,97,10,500000,Failed,N,Low,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
18 18,1000000,73,13,1000000,73,13,500000,Success,Y,Low,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
19 19,1000065,34,17,1000065,34,17,1500000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
20 20,1000032,48,37,1000032,48,37,1000000,Failed,N,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
21 21,1000070,91,39,1000070,91,39,2500000,Success,Y,Medium,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
22 22,1000000,71,20,1000000,71,20,500000,Success,Y,Low,VALID,2025-05-28,2025-05-28,BATCH_20250528_INC
```

Source code :

```
const { parse } = require("csv-parse/sync")
const { createObjectCsvWriter } = require("csv-writer")
const fetch = require("node-fetch")

// Function to log ETL steps
function logStep(step, message) {
  console.log(`[${new Date().toISOString()}] [${step}] ${message}`)
}

// Function to fetch CSV data from URL
async function fetchCSVData(url) {
  try {
    const response = await fetch(url)
    const csvText = await response.text()
    return parse(csvText, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    })
  } catch (error) {
    throw new Error(`Failed to fetch CSV data: ${error.message}`)
  }
}

// CSV Export function
async function exportToCSV(data, outputPath) {
  const headers = Object.keys(data[0] || {}).map((key) => ({ id: key, title: key }))

  const csvWriter = createObjectCsvWriter({
    path: outputPath,
    header: headers,
```

```

    })

    await csvWriter.writeRecords(data)
    console.log(`Output written to ${outputPath}`)
  }

  // Function to get last ETL run timestamp
  function getLastETLRunTimestamp() {
    return "2023-05-01T00:00:00Z"
  }

  // Function to update ETL run timestamp
  function updateETLRunTimestamp() {
    const timestamp = new Date().toISOString()
    logStep("INFO", `Updated ETL run timestamp to ${timestamp}`)
    return timestamp
  }

  // Function to identify changed records for fact table
  function identifyChangedFactBayarFRSRecords(sourceData, existingData) {
    logStep("CDC", "Identifying changed records for F_Bayar_FRS")

    const newRecords = []
    const updatedRecords = []
    const unchangedRecords = []

    sourceData.forEach((sourceRecord) => {
      const existingRecord = existingData.find(
        (record) =>
          record.Mahasiswa_NK == sourceRecord.Mahasiswa_SK &&
          record.Waktu_NK == sourceRecord.Waktu_SK &&
          record.Metode_Pembayaran_NK == sourceRecord.Metode_Pembayaran_SK,
      )

      if (!existingRecord) {
        newRecords.push(sourceRecord)
      } else {
        let hasChanged = false

        if (
          Number.parseFloat(sourceRecord.Nominal_bayar) !== existingRecord.Nominal_bayar ||
          sourceRecord.Status_pembayaran !== existingRecord.Status_pembayaran
        ) {
          hasChanged = true
        }
      }
    })
  }

```

```

    }

    if (hasChanged) {
        updatedRecords.push(sourceRecord)
    } else {
        unchangedRecords.push(sourceRecord)
    }
}
})

return {
    newRecords,
    updatedRecords,
    unchangedRecords,
}
}

// Transform new records
function transformNewFactBayarFRSRecords(data, startIndex = 0) {
    return data.map((record, index) => {
        const transformed = {}

        // Generate fact surrogate key
        transformed.BayarFRS_SK = startIndex + index + 1

        // Use surrogate keys directly from fact table
        transformed.Mahasiswa_SK = Number.parseInt(record.Mahasiswa_SK) || null
        transformed.Waktu_SK = Number.parseInt(record.Waktu_SK) || null
        transformed.Metode_Pembayaran_SK = Number.parseInt(record.Metode_Pembayaran_SK)
        || null

        // Store original business keys
        transformed.Mahasiswa_NK = Number.parseInt(record.Mahasiswa_SK)
        transformed.Waktu_NK = record.Waktu_SK
        transformed.Metode_Pembayaran_NK = record.Metode_Pembayaran_SK

        // Transform measures
        transformed.Nominal_bayar = Number.parseFloat(record.Nominal_bayar) || null

        // Transform Status_pembayaran
        if (record.Status_pembayaran) {
            const status = record.Status_pembayaran.toString().toLowerCase()
            if (status === "true" || status === "1" || status === "success" || status === "berhasil") {
                transformed.Status_pembayaran = "Success"
            }
        }
    })
}

```

```

    transformed.Is_Payment_Success = "Y"
  } else {
    transformed.Status_pembayaran = "Failed"
    transformed.Is_Payment_Success = "N"
  }
} else {
  transformed.Status_pembayaran = "Unknown"
  transformed.Is_Payment_Success = "N"
}

// Add payment categorization
if (transformed.Nominal_bayar) {
  if (transformed.Nominal_bayar < 1000000) {
    transformed.Payment_Category = "Low"
  } else if (transformed.Nominal_bayar < 5000000) {
    transformed.Payment_Category = "Medium"
  } else {
    transformed.Payment_Category = "High"
  }
} else {
  transformed.Payment_Category = "Unknown"
}

// Data quality checks
transformed.Data_Quality_Flag = "VALID"
const issues = []

if (!transformed.Mahasiswa_SK) issues.push("MISSING_MAHASISWA")
if (!transformed.Waktu_SK) issues.push("MISSING_WAKTU")
if (!transformed.Metode_Pembayaran_SK)
issues.push("MISSING_METODE_PEMBAYARAN")

if (transformed.Nominal_bayar && transformed.Nominal_bayar > 20000000) {
  issues.push("EXCESSIVE_PAYMENT_AMOUNT")
}

if (transformed.Nominal_bayar && transformed.Nominal_bayar < 100000) {
  issues.push("UNUSUALLY_LOW_PAYMENT")
}

if (issues.length > 0) {
  transformed.Data_Quality_Flag = "WARNING"
  transformed.Data_Quality_Issues = issues.join(", ")
}

```



```

// Add audit fields
transformed.Created_Date = new Date().toISOString().split("T")[0]
transformed.Updated_Date = new Date().toISOString().split("T")[0]
transformed.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
""').split("T")[0]}_INC`

return transformed
})
}

// Transform updated records
function transformUpdatedFactBayarFRSRecords(updatedRecords, existingData) {
  const transformedRecords = []

  updatedRecords.forEach((updatedRecord) => {
    const existingRecord = existingData.find(
      (record) =>
        record.Mahasiswa_NK == updatedRecord.Mahasiswa_SK &&
        record.Waktu_NK == updatedRecord.Waktu_SK &&
        record.Metode_Pembayaran_NK == updatedRecord.Metode_Pembayaran_SK,
    )

    if (existingRecord) {
      const updatedFactRecord = { ...existingRecord }

      // Update measures
      updatedFactRecord.Nominal_bayar = Number.parseFloat(updatedRecord.Nominal_bayar) ||
      null

      // Update Status_pembayaran
      if (updatedRecord.Status_pembayaran) {
        const status = updatedRecord.Status_pembayaran.toString().toLowerCase()
        if (status === "true" || status === "1" || status === "success" || status === "berhasil") {
          updatedFactRecord.Status_pembayaran = "Success"
          updatedFactRecord.Is_Payment_Success = "Y"
        } else {
          updatedFactRecord.Status_pembayaran = "Failed"
          updatedFactRecord.Is_Payment_Success = "N"
        }
      } else {
        updatedFactRecord.Status_pembayaran = "Unknown"
        updatedFactRecord.Is_Payment_Success = "N"
      }
    }
  })
}

```

```

// Update payment categorization
if (updatedFactRecord.Nominal_bayar) {
  if (updatedFactRecord.Nominal_bayar < 1000000) {
    updatedFactRecord.Payment_Category = "Low"
  } else if (updatedFactRecord.Nominal_bayar < 5000000) {
    updatedFactRecord.Payment_Category = "Medium"
  } else {
    updatedFactRecord.Payment_Category = "High"
  }
} else {
  updatedFactRecord.Payment_Category = "Unknown"
}

// Update audit fields
updatedFactRecord.Updated_Date = new Date().toISOString().split("T")[0]
updatedFactRecord.ETL_Batch_ID = `BATCH_${new Date().toISOString().replace(/[: -]/g,
"").split("T")[0]}_INC`

  transformedRecords.push(updatedFactRecord)
}
})

return transformedRecords
}

// Main incremental ETL function for F_Bayar_FRS
async function incrementalETLFactBayarFRS() {
  try {
    logStep("START", "Starting incremental ETL process for F_Bayar_FRS")

    const lastETLRunTimestamp = getLastETLRunTimestamp()
    logStep("INFO", `Last ETL run timestamp: ${lastETLRunTimestamp}`)

    // 1. EXTRACTION
    logStep("EXTRACTION", "Starting extraction from bayarFRS CSV")
    const sourceData = await fetchCSVData(
      "https://hebbkx1anhila5yf.public.blob.vercel-storage.com/Bayar_FRS-
lagNftDgjFRCHST5Ov2SWnbH2JnBJe.csv",
    )
    logStep("EXTRACTION", `Extracted ${sourceData.length} records from source`)

    // Simulate existing fact data
    const existingFactData = [

```

```
{
  BayarFRS_SK: 1,
  Mahasiswa_SK: 1,
  Waktu_SK: 1,
  Metode_Pembayaran_SK: 1,
  Mahasiswa_NK: 1,
  Waktu_NK: "1",
  Metode_Pembayaran_NK: "1",
  Nominal_bayar: 2500000.0,
  Status_pembayaran: "Success",
  Is_Payment_Success: "Y",
  Payment_Category: "Medium",
  Data_Quality_Flag: "VALID",
  Created_Date: "2023-05-01",
  Updated_Date: "2023-05-01",
  ETL_Batch_ID: "BATCH_20230501_001",
},
]
```

```
logStep("EXTRACTION", `Read ${existingFactData.length} existing fact records from data
warehouse`)
```

```
// 2. CHANGE DATA CAPTURE
```

```
logStep("CDC", "Identifying changed records")
const { newRecords, updatedRecords, unchangedRecords } =
identifyChangedFactBayarFRSRecords(
  sourceData,
  existingFactData,
)
logStep(
  "CDC",
  `Identified ${newRecords.length} new records, ${updatedRecords.length} updated records,
and ${unchangedRecords.length} unchanged records`,
)
```

```
// 3. TRANSFORMATION
```

```
logStep("TRANSFORMATION", "Starting data transformation for new and updated records")

const transformedNewRecords = transformNewFactBayarFRSRecords(newRecords,
existingFactData.length)
logStep("TRANSFORMATION", `Transformed ${transformedNewRecords.length} new
records`)
```

```

    const transformedUpdatedRecords =
transformUpdatedFactBayarFRSRecords(updatedRecords, existingFactData)
    logStep("TRANSFORMATION", `Transformed ${transformedUpdatedRecords.length}
updated records`)

// 4. LOADING
logStep("LOADING", "Starting data loading")

if (transformedNewRecords.length > 0) {
    logStep("LOADING", `Inserting ${transformedNewRecords.length} new fact records`)
}

if (transformedUpdatedRecords.length > 0) {
    logStep("LOADING", `Updating ${transformedUpdatedRecords.length} existing fact
records`)
}

// 5. EXPORT TO CSV
logStep("EXPORT", "Exporting transformed data to CSV")

if (transformedNewRecords.length > 0) {
    await exportToCSV(transformedNewRecords, "F_Bayar_FRS_New_Records.csv")
}

if (transformedUpdatedRecords.length > 0) {
    await exportToCSV(transformedUpdatedRecords, "F_Bayar_FRS_Updated_Records.csv")
}

const newTimestamp = updateETLRunTimestamp()

logStep("COMPLETE", "Incremental ETL process for F_Bayar_FRS completed successfully")

return {
    status: "success",
    timestamp: newTimestamp,
    newRecords: transformedNewRecords.length,
    updatedRecords: transformedUpdatedRecords.length,
    unchangedRecords: unchangedRecords.length,
    sampleNewRecords: transformedNewRecords.slice(0, 2),
    sampleUpdatedRecords: transformedUpdatedRecords.slice(0, 2),
}
} catch (error) {
    logStep("ERROR", `Incremental ETL process failed: ${error.message}`)
    console.error(error)
}

```

```
    return {  
      status: "error",  
      message: error.message,  
    }  
  }  
}  
}  
  
incrementalETLFactBayarFRS()  
  .then((result) => {  
    console.log("\nIncremental ETL Process Result:", result)  
  })  
  .catch((error) => {  
    console.error("Incremental ETL Process Failed:", error)  
  })
```

Dashboard

Dashboard design

Dashboard Implementation

Penggunaan Dashboard